SOFTWARE REQUIREMENTS SPECIFICATION

Zanzibar Tourism Journey Orchestration Platform

A Mobile-First Tourism Planning, Reservation, Transportation, Accommodation and Travel-Management System

Document Reference: SRS-ZTJOP-001

Version 1.0

Status: Approved for Development

Classification: Internal — Engineering Distribution

Document Control

Document Identification

Attribute	Value
Document Title	Software Requirements Specification — Zanzibar Tourism Journey Orchestration Platform
Document Reference	SRS-ZTJOP-001
Version	1.0
Document Type	Software Requirements Specification (IEEE 830 / ISO-IEC-IEEE 29148 aligned)
Product Name	Zanzibar Tourism Journey Orchestration Platform (referred to throughout as “the Platform”)
Initial Market	Zanzibar, United Republic of Tanzania
Target Users	International inbound tourists, transport providers, accommodation providers, activity providers, platform administrators
Intended Audience	Software architects, backend engineers, mobile engineers, database engineers, QA engineers, DevOps engineers, product owners, project managers

Revision History

Version	Date	Author	Description of Change
0.1	—	Systems Analysis Team	Initial concept capture and stakeholder interviews
0.5	—	Architecture Team	Draft architecture, domain model and data design
0.9	—	Architecture Team	API specification, diagrams, test strategy added
1.0	—	Architecture Team	Consolidated baseline release for development

Approvals

Role	Responsibility	Signature	Date
Product Owner	Business scope, MVP boundary, prioritisation		
Solution Architect	Architecture, technology stack, integration design		
Lead Backend Engineer	Backend, API and data implementation feasibility		
Lead Mobile Engineer	Tourist and Driver application feasibility		
QA Lead	Testability and acceptance criteria		
DevOps Lead	Deployment, environments, operability		

How to Read This Document

This specification is written to be executable by an engineering team that has no prior exposure to the project. Sections 1 to 5 establish business context. Sections 6 to 9 establish the technical foundation (architecture, data, backend, API) and should be read before any implementation begins. Sections 10 to 22 specify the functional subsystems. Sections 23 to 27 specify the client applications. Section 28 contains the formal UML and data-flow models. Sections 29 to 44 cover quality attributes, delivery process, and traceability.

Requirement identifiers follow the convention below and are referenced throughout the traceability matrix in Section 42.

Prefix	Meaning
FR-xxx	Functional requirement
NFR-xxx	Non-functional requirement
BR-xxx	Business rule
UC-xxx	Use case
TC-xxx	Test case
API-xxx	API endpoint group

Table of Contents

1 Executive Summary

The Platform is a mobile-first tourism planning, reservation, and travel-management system initially designed for international tourists visiting Zanzibar, Tanzania. The platform allows tourists to plan and reserve their Zanzibar journey before travelling, including airport transfers, accommodation, transportation, attractions, activities, and other tourism services. The system provides an integrated itinerary, calculates distances and estimated travel times, presents service costs, coordinates bookings with verified service providers, facilitates payments, and manages the tourist's journey from arrival at Zanzibar International Airport through completion of the planned trip. The underlying architecture is designed for future expansion to mainland Tanzania and potentially other tourism destinations.

The commercial premise is a two-sided marketplace. On the demand side, an international tourist assembles a complete, priced, confirmed journey while still in their home country. On the supply side, verified Zanzibar service providers — drivers and transport operators, hotels and guesthouses, activity and excursion operators — receive pre-paid, pre-scheduled demand without operating their own booking technology. The Platform earns revenue by taking a configurable commission on each settled booking.

The distinguishing technical characteristic of the product is the **itinerary orchestration layer**. Competing products in the region sell either transport (ride-hailing) or accommodation (OTA listings) or excursions (tour aggregation) as isolated transactions. This Platform treats those as component bookings assembled under a single Trip aggregate, sequenced on a day-by-day timeline, validated for temporal and geographic feasibility, priced as one basket, paid for in one transaction, and settled to many providers. That aggregate — Trip → Itinerary → ItineraryItem → Booking — is the architectural centre of the system and the source of most of its non-trivial business logic.

Three constraints shape every decision in this document:

1. **Zanzibar is the entire Version 1 market.** No feature may be deferred on the grounds that it is "needed for mainland later", and no logic may be written that assumes Zanzibar.
2. **The architecture must be destination-independent.** Destinations, attractions, activities, pricing rules, transfer corridors and policies are configuration and data, never code branches.
3. **The delivered product contains no artificial-intelligence features.** All recommendation, pricing, matching, scheduling, ranking and fraud-control behaviour is implemented as deterministic, auditable, rule-driven and query-driven logic.

The recommended implementation is a **modular monolith** backend (Django 5 + Django REST Framework, PostgreSQL 16 with PostGIS, Redis, Celery) exposing a versioned REST API to three clients: a Flutter tourist application, a Flutter driver application, and a React web application serving both the provider portal and the administration console. Section 6 justifies this choice against microservices and defines the seam-preserving rules that allow selective extraction later.

The MVP is delivered in fourteen phases over an estimated 32 to 38 engineering weeks with a team of six. Section 37 gives the phase plan; Section 38 gives the MUST/SHOULD/FUTURE boundary.

2 Introduction

2.1 Purpose of This Document

This document specifies, completely and unambiguously, the software to be built. It is the primary technical reference for design, estimation, task breakdown, implementation, testing, deployment and maintenance. Where this document conflicts with any earlier concept note, presentation or verbal brief, this document prevails.

It is written to satisfy four distinct readers:

- A **developer** must be able to open it at any subsystem section and implement that subsystem without further clarification.
- A **project manager** must be able to derive a work breakdown structure, dependencies and a delivery plan from Sections 37 and 38.

- A **QA engineer** must be able to derive test cases from Sections 41 and 43 and trace them to requirements via Section 42.
- An **operations engineer** must be able to build the environments and pipelines described in Section 35.

2.2 Scope of the Product

2.2.1 In Scope for Version 1

The Platform in its first release covers the complete pre-arrival and in-destination journey of an international tourist in Zanzibar:

- Account creation, identity capture and authentication for tourists and providers.
- Browsing of a curated, administrator-managed catalogue of Zanzibar destinations, attractions, activities and accommodation.
- Construction of a multi-day trip: dates, party size, inbound and outbound flight details, accommodation nights, activity selections, and inter-location transport legs.
- Deterministic computation of distance, estimated travel duration and price for every transport leg using an external routing provider behind an internal abstraction.
- Generation of a structured, day-sequenced itinerary that is persisted, versioned and re-priceable.
- Single-basket checkout covering all component bookings, paid by international card or mobile money, in USD or TZS.
- Availability enforcement, capacity enforcement and inventory holding across accommodation, activities and transport.
- Driver assignment for airport transfers and scheduled transport legs, including offer dispatch, acceptance, and status progression.
- Real-time driver location sharing during an active transport leg only.
- Booking-scoped messaging between tourist and provider.
- Push, email and SMS notification of all material journey events.
- Cancellation and refund processing under configurable, per-service-type policies.
- Commission calculation, provider ledger accrual and payout batch generation.
- Post-trip ratings and moderated reviews.
- A web provider portal for listing, availability, pricing, booking and earnings management.
- A web administration console covering catalogue, users, verification, bookings, payments, refunds, commissions, disputes, reporting and audit.

2.2.2 Explicitly Out of Scope for Version 1

The following are deliberately excluded. Each is excluded because it adds material regulatory, integration or operational risk without which the MVP still delivers its core value.

Excluded item	Rationale
Flight search, flight booking or flight ticketing	Requires GDS/IATA accreditation; the Platform records flight details supplied by the tourist for scheduling purposes only
Visa, immigration or e-permit processing	Government integration; informational content only in V1
Travel insurance sale	Regulated intermediation; deferred
On-demand hailing of an unplanned ride	V1 transport is scheduled and pre-booked; on-demand is a Version 2 candidate (Section 44)
Dynamic/surge pricing	Prices are set by providers and administrator-configured rules; no algorithmic price movement
Multi-language user interface beyond English	Localisation framework is built in V1 (Section 29.10); additional locales load as data
Provider-side mobile applications for hotels and activity operators	Providers use the responsive web portal in V1
Any AI/ML capability of any kind	Explicit product constraint (Section 50 of the concept brief; restated in

Excluded item	Rationale
	Section 3.5)

2.3 Definitions, Acronyms and Abbreviations

Term	Definition
Trip	The top-level aggregate representing one tourist journey to one destination between two dates
Itinerary	The ordered, day-partitioned plan belonging to exactly one Trip
Itinerary Item	A single scheduled element of an itinerary (a stay, an activity, a transfer, or a free block)
Booking	A commercial reservation of a specific service from a specific provider, referenced by an itinerary item
Component Booking	A Booking that is one part of a multi-booking Trip basket
Transfer	A point-to-point transport leg performed by an assigned driver
Provider	Any supply-side commercial entity: transport operator, accommodation, or activity operator
Driver	An individual authorised to perform transfers, employed by or acting as a transport provider
Inventory Hold	A short-lived exclusive reservation of capacity placed during checkout
Settlement	The transfer of the provider's share of a completed booking to the provider's ledger
Payout	The batched disbursement of a provider's settled ledger balance
GDS, OTA	Global Distribution System / Online Travel Agency
ZNZ	Abeid Amani Karume International Airport (Zanzibar International Airport), IATA code ZNZ
DRF	Django REST Framework
RBAC	Role-Based Access Control
RTO / RPO	Recovery Time Objective / Recovery Point Objective
PII	Personally Identifiable Information
TZS / USD	Tanzanian Shilling / United States Dollar
PCI DSS	Payment Card Industry Data Security Standard
SAQ A	PCI DSS Self-Assessment Questionnaire A, applicable to merchants who fully outsource cardholder data handling

2.4 References

Ref	Source	Use in this document
R1	ISO/IEC/IEEE 29148:2018	Requirements specification structure
R2	IEEE 830-1998	Legacy SRS section conventions
R3	OMG UML 2.5.1	Use case, sequence, activity and state diagram notation
R4	RFC 7231 / RFC 9110	HTTP semantics and status code usage
R5	RFC 7519	JSON Web Token structure
R6	OWASP ASVS 4.0	Security verification requirements (Section 30)
R7	OWASP API Security Top 10 (2023)	API threat controls
R8	PCI DSS v4.0 SAQ A	Payment scope minimisation
R9	Tanzania Personal Data Protection Act, 2022	Data privacy obligations (Section 31)
R10	EU GDPR 2016/679	Applicable to EU-resident tourists; privacy baseline
R11	ISO 4217	Currency codes
R12	ISO 8601	Date, time and duration representation
R13	ISO 3166-1	Country codes

3 Problem Statement, Objectives and Value Proposition

3.1 Problem Statement

An international tourist planning a Zanzibar holiday today assembles the journey from disconnected, largely offline sources. Accommodation is booked through a global OTA. Airport pickup is arranged by WhatsApp with a driver recommended by the hotel, or negotiated on arrival at the terminal. Excursions are sold by beach-front intermediaries at variable prices with no written terms. Inter-town transport is priced ad hoc. Payment is fragmented across card, cash USD, and mobile money.

This produces six concrete failures:

#	Failure	Consequence
P1	No single view of the journey	The tourist cannot verify that the plan is coherent before travelling
P2	Arrival uncertainty	The tourist does not know who will meet them, in what vehicle, or at what cost until arrival
P3	Opaque and inconsistent pricing	Identical services are quoted at materially different prices; the tourist cannot budget
P4	No temporal feasibility checking	Activities are booked that cannot physically be reached in time from the accommodation
P5	No accountability or recourse	Cash transactions with unverified operators leave no record for dispute resolution
P6	Provider demand volatility	Verified, capable local operators have no channel to reach pre-arrival demand and rely on intermediaries who capture most of the margin

3.2 Objectives

ID	Objective	Measure of success
OBJ-1	Enable a tourist to complete a full Zanzibar journey plan and payment before departure from their home country	≥ 80% of confirmed trips have all component bookings confirmed more than 72 hours before arrival
OBJ-2	Guarantee arrival certainty	100% of confirmed airport transfers display assigned driver identity, vehicle, plate and pickup time to the tourist at least 24 hours before the flight arrival time
OBJ-3	Present a single, complete, itemised trip cost before payment	Every trip summary reconciles component prices, fees and taxes to the charged total to the minor currency unit
OBJ-4	Enforce temporal and capacity feasibility of every itinerary	Zero confirmed itineraries containing an overlapping or unreachable item, verified by the validation rules in Section 10.6
OBJ-5	Give verified local providers a direct, accountable demand channel	Provider onboarding to first booking within 10 working days; provider payout within the configured settlement window with a full audit trail
OBJ-6	Build a destination-independent platform	Adding a new destination requires zero application code changes, demonstrated by the acceptance test in Section 41.12

3.3 Core Value Proposition

The tourist replaces a set of disconnected arrangements with one orchestrated journey.

```
BEFORE                                AFTER
-----------------------------------   -----------------------------------
OTA account          -> hotel         One account
WhatsApp             -> driver        One planned itinerary
Beach intermediary   -> excursions    One price, itemised
Cash negotiation     -> transfers     One payment
No record            -> disputes      One record, one support channel
Arrival uncertainty                   Named driver, known vehicle, known time
```

The tourist-facing outcome is a single artefact — the confirmed itinerary — of the following shape. The exact layout is specified in Section 24.20; the content model is specified in Section 10.

```
MY ZANZIBAR TRIP                                  Ref: TRP-2027-0000481
Party: 2 adults                                   Status: CONFIRMED

ARRIVAL
  Abeid Amani Karume Intl (ZNZ)   10 Aug 2027  14:30   Flight PW 412

DAY 1 - 10 Aug
  14:45  Airport transfer   ZNZ -> Ocean Breeze Hotel, Nungwi
         Driver: John M.  Toyota Noah  T 123 ABC  Seats 6
         57.4 km - est. 1 h 25 min                        USD  38.00
  16:30  Check-in          Ocean Breeze Hotel, Deluxe Sea View
         10 Aug -> 15 Aug, 5 nights, 1 room            USD 465.00

DAY 2 - 11 Aug
  08:30  Transfer          Hotel -> Stone Town             USD  42.00
  10:00  Stone Town Heritage Walking Tour  (3 h)           USD  60.00
  14:00  Transfer          Stone Town -> Hotel             USD  42.00

DAY 3 - 12 Aug
  09:00  Mnemba Atoll Snorkelling  (5 h, incl. transfer)   USD 110.00

DEPARTURE
  15 Aug  09:00  Transfer  Hotel -> ZNZ                    USD  38.00

COST SUMMARY
  Accommodation                                           USD 465.00
  Transportation                                          USD 160.00
  Activities                                              USD 170.00
  Service fee (5%)                                        USD  39.75
  ------------------------------------------------------------------
  TOTAL PAID                                              USD 834.75
```

3.4 Non-Goals

The Platform is not a ride-hailing application. Transport is one of four bookable service classes and is subordinate to the itinerary. The Platform is not an OTA; it does not aggregate global inventory but curates a verified local supply base. The Platform is not a payment institution; it never holds card data and uses a licensed payment service provider for all money movement.

3.5 The No-AI Constraint

This is a hard product constraint and a design constraint, not a preference. The delivered application must contain no artificial intelligence, machine learning, or generative capability. Specifically prohibited: AI chatbots, AI trip planners, AI or ML recommendation engines, AI travel assistants, generative content, algorithmic/AI pricing, ML fraud detection, and ML personalisation.

Every behaviour that a reader might expect to be delivered by AI is delivered instead by an explicit, deterministic mechanism, as follows. This mapping is binding on implementers.

Behaviour	Prohibited approach	Required implementation
“Suggested activities for you”	ML recommender	SQL ranking over `activity` filtered by destination proximity, tourist-selected interest tags, availability window, and administrator-set `feature_rank`; ordering is deterministic and reproducible (Section 16.5)
“Best driver for this booking”	ML matching	Deterministic scoring function with published, administrator-tunable weights over eligibility, proximity, vehicle capacity, acceptance rate and rating (Section 11.6)
“Optimal itinerary ordering”	AI planner	Constraint validation plus greedy chronological insertion against opening hours and travel-time matrices (Section 10.6)

Behaviour	Prohibited approach	Required implementation
“Price for this transfer”	AI/surge pricing	Tariff table lookup: base fare + per-km rate + per-minute rate + zone surcharge + time-of-day surcharge, all rows administrator-managed (Section 12.4)
“Fraud detection”	ML anomaly detection	Rule engine over explicit thresholds (velocity, mismatch, chargeback history) producing reviewable flags for human decision (Section 30.14)
“Support assistant”	LLM chatbot	Structured help centre articles, categorised ticket forms, and human agent queue (Section 24.32)

Verification of this constraint is a release gate: see TC-901 in Section 33 and item 17 of the consistency checklist in Section 43.

4 Geographic Scope and Expansion Model

4.1 Version 1 Scope: Zanzibar Only

Version 1 operates in Zanzibar (Unguja and, where administratively enabled, Pemba) exclusively. The catalogue is seeded with the destinations below. These are **data records**, not code constants, and are managed entirely through the administration console (Section 27.8).

Destination	Type	Typical role in a trip	Approx. road distance from ZNZ
Abeid Amani Karume Intl. Airport (ZNZ)	Airport	Arrival / departure node	0 km
Stone Town	Urban heritage	Attraction cluster, accommodation	7 km
Nungwi	North coast beach	Accommodation, water activities	57 km
Kendwa	North coast beach	Accommodation, sunset excursions	55 km
Paje	South-east coast	Kitesurfing, accommodation	48 km
Jambiani	South-east coast	Accommodation, village tours	55 km
Matemwe	North-east coast	Mnemba Atoll access, accommodation	55 km
Kiwengwa	East coast	Resort accommodation	40 km
Michamvi	South-east peninsula	Accommodation, sunset points	55 km
Pemba (Chake Chake)	Secondary island	Deferred; record created but `is_active = false`	

Distances shown are indicative seed values used to bootstrap the travel-time matrix; authoritative values are obtained from the routing provider at runtime (Section 12.3) and cached.

4.2 Architectural Consequence: Destination Independence

The single most damaging mistake an implementer can make is to encode Zanzibar-specific behaviour in application logic. The following are **prohibited** in the codebase:

```
PROHIBITED                            REQUIRED INSTEAD
-----------------------------------   -----------------------------------
if destination == "Zanzibar":         destination = Destination.objects.get(...)
ZANZIBAR_AIRPORT_ID = 1               destination.is_gateway == True
NUNGWI_TRANSFER_PRICE = 38            TransferTariff lookup by corridor
if region in ("Unguja","Pemba"):      region.country.code / region.is_active
hard-coded activity list              Activity table filtered by destination
hard-coded currency "TZS"             Currency resolved from destination.country
```

The geographic model is a five-level hierarchy. Every catalogue entity attaches to it, and every query filters by it.

```
Country            (ISO 3166-1; e.g. TZ - Tanzania)
   |  1..*
Region             (e.g. Zanzibar Urban/West, Zanzibar North, Arusha)
   |  1..*
Destination        (e.g. Nungwi, Stone Town, ZNZ Airport)
   |  1..*        \
Attraction          \ 1..*
   |  1..*           Accommodation
Activity            (attaches to Destination directly)
```

A destination carries the flags that drive behaviour, so behaviour changes by data, not by code:

Flag / field	Effect
`is_active`	Controls catalogue visibility and bookability
`is_gateway`	Marks an arrival/departure node (airport, ferry terminal); enables flight capture and transfer scheduling
`gateway_type`	`AIRPORT` | `SEAPORT` | `LAND_BORDER`
`centroid` (PostGIS Point)	Basis of proximity queries and travel-time matrix keys
`timezone`	IANA zone (`Africa/Dar_es_Salaam`); all local-time rendering derives from this
`default_currency`	Presentment currency for catalogue prices in this destination
`launch_date`	Enables scheduled market launch without a deployment

4.3 Expansion Path

```
VERSION 1 ......... ZANZIBAR (Unguja)
      |            single Region set, one gateway (ZNZ), full feature set
      v
VERSION 2 ......... MAINLAND TANZANIA
      |            Dar es Salaam, Arusha, Serengeti, Ngorongoro,
      |            Kilimanjaro, Tarangire, Manyara
      |            NEW CAPABILITY NEEDED: multi-day safari packages,
      |            inter-region domestic flights, park permit fees
      v
VERSION 3 ......... EAST AFRICA
      |            Kenya, Rwanda, Uganda
      |            NEW CAPABILITY NEEDED: multi-country trips,
      |            cross-border transfer compliance, second country tax model
      v
VERSION 4+ ........ FURTHER INTERNATIONAL MARKETS
```

The table below records what expansion costs under this design. Anything in the “code change” column represents accepted, bounded future work — not rework.

Expansion activity	Configuration only	Code change required
Add a destination, attraction, activity, accommodation	Yes	No
Add a transfer corridor and tariff	Yes	No
Add a currency and presentment rules	Yes	No
Add a country with new tax treatment	Partly (tax rule rows)	Only if a new tax *type* is needed
Add a new service class (e.g. domestic flight seat)	No	Yes — new booking subtype and provider type
Multi-country single trip	No	Yes — Trip gains a destination set rather than a single destination
New payment provider	Partly (config)	Yes — new adapter implementing the gateway port

Multi-country trips are the one known structural limitation of the V1 model: `trip.destination_id` is a single foreign key. Section 44.4 specifies the migration path (introduce `trip_destination` as a join table, retaining a `primary_destination_id` for compatibility).

5 Stakeholders, Actors and User Roles

5.1 Actor Catalogue

Actor	Type	Channel	Authentication	Volume assumption (Year 1)
Tourist	Primary, human	Tourist mobile app (iOS/Android)	Email + password, or OAuth social login; JWT	15,000 registered; 3,000 trips
Driver	Primary, human	Driver mobile app (Android-first)	Phone + password + OTP; JWT	250 verified
Transport Provider (company)	Primary, human	Provider web portal	Email + password + mandatory TOTP	40
Accommodation Provider	Primary, human	Provider web portal	Email + password + mandatory TOTP	180 properties
Activity Provider	Primary, human	Provider web portal	Email + password + mandatory TOTP	120
Administrator	Primary, human	Admin web console	Email + password + mandatory TOTP, IP allow-list	12 staff
Support Agent	Secondary, human	Admin web console (restricted role)	As administrator	6 staff
Finance Officer	Secondary, human	Admin web console (restricted role)	As administrator	3 staff
Payment Service Provider	External system	Server-to-server + webhook	mTLS or signed webhook	—
Routing/Maps Provider	External system	Server-to-server HTTPS	API key held server-side	—
Push Notification Service	External system	Server-to-server	Service account	—
Email/SMS Gateway	External system	Server-to-server	API key	—
Scheduler (system actor)	Internal system	Celery Beat	Internal	—

5.2 Role-Permission Model

Authorisation is RBAC with resource-level ownership checks. A role grants permission to an *operation class*; an ownership predicate then restricts it to rows the principal owns. Both checks must pass.

Role	Key permissions	Ownership predicate
`TOURIST`	Manage own profile; create/read/update own trips; create bookings; pay; cancel own bookings; message on own bookings; review completed bookings	`row.tourist_id == principal.tourist_id`
`DRIVER`	Read assigned bookings; set availability; accept/decline offers; progress trip status; read own earnings	`row.driver_id == principal.driver_id`
`PROVIDER_OWNER`	Full management of own provider org, listings, availability, pricing, bookings, staff, payout details	`row.provider_id == principal.provider_id`
`PROVIDER_STAFF`	Manage listings, availability and bookings for own provider; no payout or banking access	`row.provider_id == principal.provider_id`
`SUPPORT_AGENT`	Read all users, trips, bookings; create/annotate tickets; issue goodwill credits up to configured cap; cannot alter payments or catalogue	Global read, scoped write
`FINANCE_OFFICER`	Read all financial records; approve refunds; approve and release payout batches; export reports	Global
`CATALOGUE_ADMIN`	Manage countries, regions, destinations, attractions, activities, tariffs, policies	Global

Role	Key permissions	Ownership predicate
`COMPLIANCE_ADMIN`	Approve/reject provider and driver verification; suspend accounts; moderate reviews	Global
`SUPER_ADMIN`	All of the above; manage roles and system configuration; read audit log	Global

Enforcement points are specified in Section 30.3. Every permission in this table maps to a DRF permission class and is covered by an authorisation test (TC-2xx series).

5.3 Actor Responsibilities

5.3.1 Tourist

Register and verify email; maintain profile including party details and passport-adjacent travel data where collected; browse destinations, attractions, activities and accommodation; create and edit a trip; supply flight information; select accommodation, nights and room types; select attractions and activities; select and price transport legs; review computed distances, travel times and prices; generate and confirm an itinerary; pay; view booking status; track an assigned driver during an active transfer; message providers within booking context; receive notifications; cancel eligible bookings; request refunds; rate and review providers; view trip history.

5.3.2 Driver

Register; submit national identity document, driving licence and photograph; register a vehicle and submit registration, insurance and inspection documents; complete verification; set online/offline availability and service area; receive and accept or decline booking offers within the offer TTL; view pickup location, tourist first name, masked contact and flight information; navigate to pickup; progress trip status through the defined states; complete the trip; view earnings and trip history; receive ratings.

5.3.3 Accommodation Provider

Register the business; submit business licence and tax registration; create one or more properties; create room types with capacity, amenities and photographs; define nightly rates by date range and rate plan; publish availability by room-type-day; manage inbound bookings; manage cancellations under policy; view earnings, statements and payouts; view and respond to reviews.

5.3.4 Activity Provider

Register; submit licence and, where applicable, operating permits; create activities with description, duration, meeting point, requirements and inclusions; define price per person and any per-group price; define capacity per departure; define schedules (fixed departures or on-request windows); manage availability and blackout dates; manage bookings; view earnings; receive ratings.

5.3.5 Transport Provider

All driver-management functions for a fleet: onboard drivers and vehicles, monitor assignment acceptance, view consolidated earnings, and manage payout details.

5.3.6 Administrator

Complete system management as enumerated in Section 27.

5.4 User Characteristics and Their Design Implications

Characteristic	Implication for the design
Tourists are outside Tanzania when planning, on high-latency intercontinental links	API responses must be paginated and small; images served from CDN with responsive variants; no chatty multi-round-trip flows in the planner
Tourists arrive on aircraft with no local SIM	Confirmed itinerary, driver details and vouchers must be available offline in the app (Section 23.8)

Characteristic	Implication for the design
Drivers use low-to-mid-range Android devices on 3G/4G with intermittent coverage	Driver app must queue status transitions locally and reconcile on reconnect; location batching, not per-second streaming
Providers are small businesses with limited IT capability	Provider portal must be usable on a mobile browser, require no installation, and use plain language
Payments cross currencies and jurisdictions	Presentment currency, settlement currency and FX capture must be modelled explicitly (Section 21.7)

6 System Architecture

6.1 Architectural Drivers

Driver	Weight	Consequence
Small founding team (≤ 8 engineers)	High	Minimise operational surface; one deployable backend
Strong transactional consistency across a multi-service basket	High	A single relational database with real transactions; no distributed saga in V1
Unknown, likely modest initial load	High	Vertical scaling first; horizontal read scaling second
Requirement for future service extraction	Medium	Enforce module seams and forbid cross-module ORM reach-through
Real-time driver location and offer dispatch	Medium	WebSocket layer alongside REST, backed by Redis pub/sub
Regulatory and audit obligations on money	High	Immutable financial ledger; append-only audit log

6.2 Architecture Decision: Modular Monolith

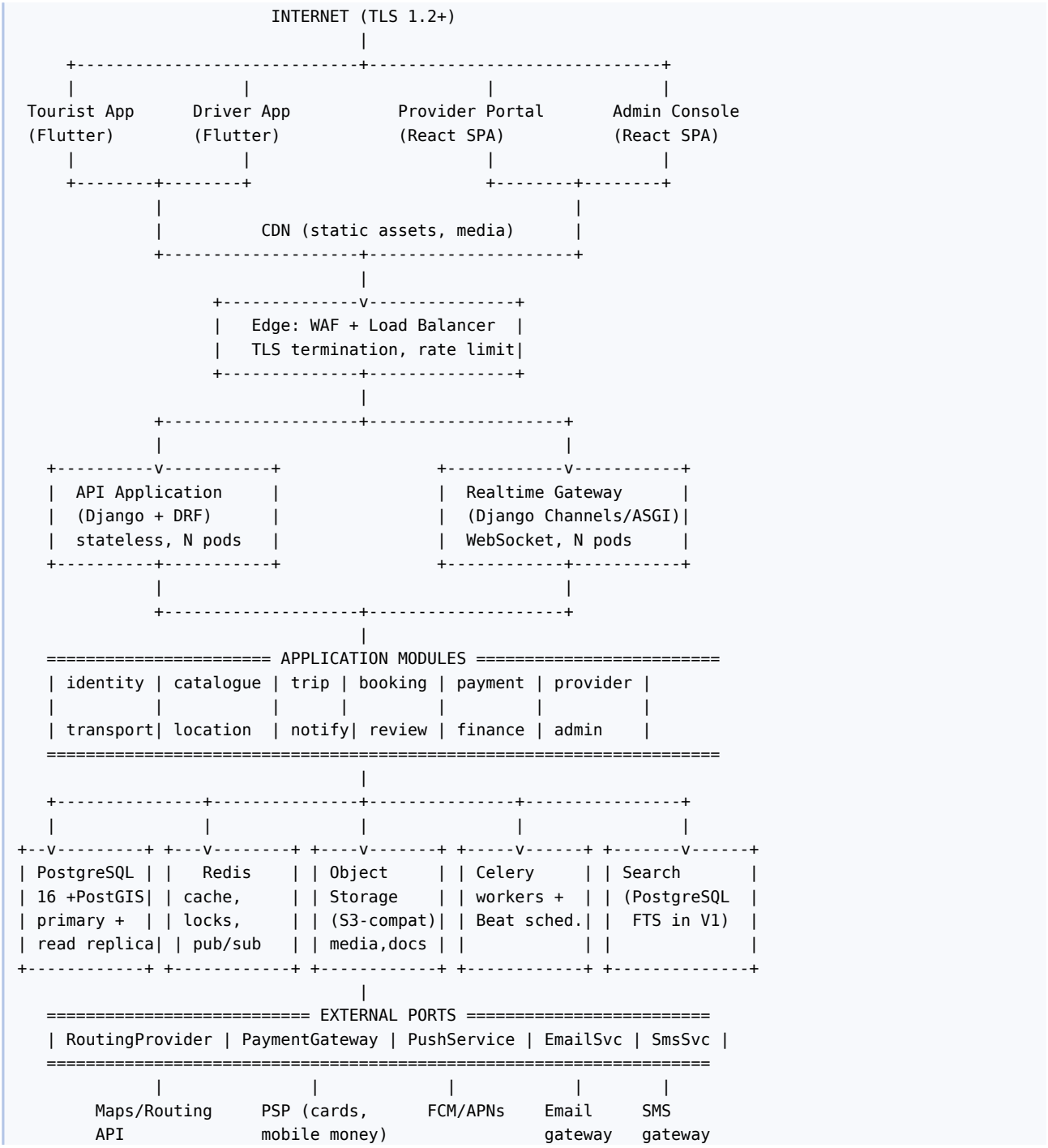
Decision. The backend is implemented as a single deployable **modular monolith** with strictly enforced internal module boundaries, deployed as multiple identical stateless instances behind a load balancer, with asynchronous work handled by separate worker processes running the same codebase.

Alternatives considered.

Option	Assessment
Microservices (per the indicative diagram in the concept brief)	Rejected for V1. The core transaction — confirm a trip containing accommodation, activity and transport bookings and a single payment — requires atomic consistency across all of them. In a microservice topology this becomes a distributed saga with compensating transactions, doubling the complexity of the most business-critical path. The team size does not support 8–10 independently deployed services, their pipelines, their observability and their on-call burden. Network partition failure modes would dominate early incident load.
Monolith without module discipline	Rejected. Guarantees an unextractable ball of mud within 18 months and violates OBJ-6's implied modularity.
Hybrid: monolith + selectively extracted services	**Adopted as the target end-state.** V1 ships as one deployable; the *notification dispatcher* and *location/tracking gateway* are already isolated behind interfaces and are the first extraction candidates because they are I/O-bound, low-consistency and high-volume.

Consequences. Deployment is simple and rollback is atomic. Cross-module calls are in-process and cheap. The cost is that a single bad deployment affects all functions, mitigated by canary deployment (Section 35.6). Module discipline must be mechanically enforced, not merely documented — see Section 6.5.

6.3 Logical Architecture



Note the deliberate deviations from the indicative diagram in the concept brief: there is no separate API gateway product (the load balancer plus DRF middleware performs routing, TLS termination, rate limiting and authentication), and the services shown there as separate deployables are internal modules here. The external ports are preserved exactly, because they are genuine integration boundaries.

6.4 Module Catalogue

Each module owns its tables. No module may query another module's tables directly; it calls the owning module's service interface.

Module	Owns (principal tables)	Public service interface (illustrative)	Depends on
`identity`	`user`, `role`, `user_role`, `tourist_profile`, `session`, `device`	`authenticate()`, `issue_tokens()`, `get_principal()`	—

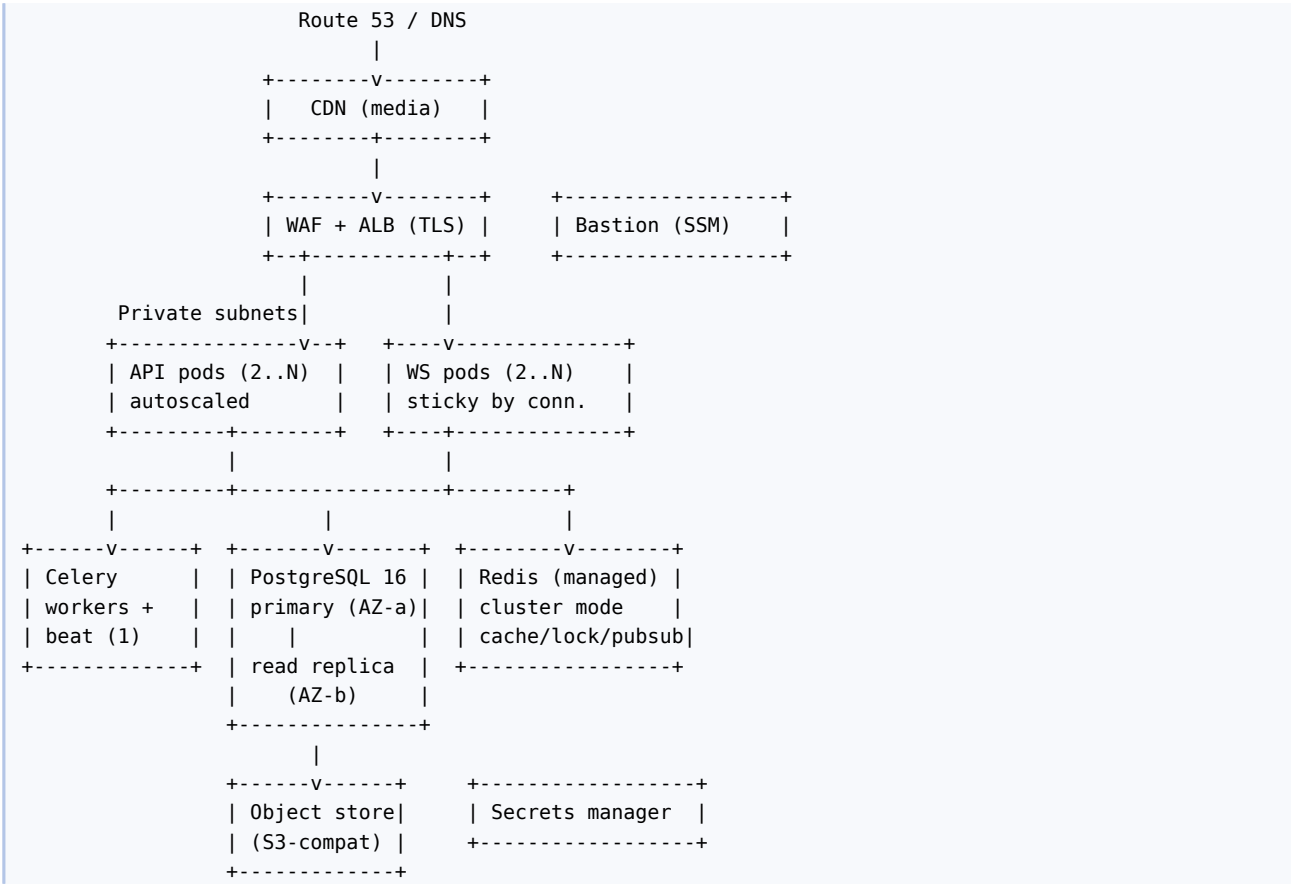
Module	Owns (principal tables)	Public service interface (illustrative)	Depends on
`catalogue`	`country`, `region`, `destination`, `attraction`, `activity`, `activity_schedule`, `accommodation`, `room_type`, `media`	`search_activities()`, `get_destination()`, `list_room_types()`	`location`
`provider`	`provider`, `provider_document`, `provider_staff`, `driver`, `vehicle`	`is_verified()`, `eligible_drivers()`, `get_payout_account()`	`identity`
`inventory`	`room_availability`, `activity_departure`, `inventory_hold`	`check_availability()`, `hold()`, `commit()`, `release()`	`catalogue`
`trip`	`trip`, `itinerary`, `itinerary_item`, `trip_flight`	`create_trip()`, `regenerate_itinerary()`, `validate_itinerary()`	`catalogue`, `transport`
`booking`	`booking`, `booking_accommodation`, `booking_activity`, `booking_transfer`, `booking_status_history`	`create_basket()`, `confirm()`, `cancel()`, `transition()`	`inventory`, `trip`, `provider`
`transport`	`transfer_corridor`, `transfer_tariff`, `driver_assignment`, `driver_offer`	`quote_transfer()`, `assign_driver()`, `dispatch_offer()`	`location`, `provider`
`location`	`route_cache`, `driver_location`, `geofence`	`route()`, `distance_matrix()`, `geocode()`, `last_known()`	— (external port)
`payment`	`payment`, `payment_transaction`, `refund`, `payment_webhook_event`	`initiate()`, `verify()`, `refund()`	`booking` (via events)
`finance`	`commission_rule`, `ledger_entry`, `provider_balance`, `payout`, `payout_item`, `fx_rate`	`accrue()`, `settle()`, `build_payout_batch()`	`payment`, `booking`
`notify`	`notification`, `notification_template`, `notification_delivery`	`emit(event, recipients, context)`	— (external ports)
`messaging`	`conversation`, `message`, `message_report`	`post()`, `thread()`	`booking`
`review`	`review`, `review_response`, `review_flag`, `rating_aggregate`	`submit()`, `moderate()`, `aggregate_for()`	`booking`
`admin`	`audit_log`, `system_setting`, `feature_flag`, `support_ticket`	`record_audit()`, `get_setting()`	all (read via interfaces)

6.5 Enforcing Module Boundaries

Discipline that is only documented is not discipline. The following controls are mandatory and are enforced in CI:

1. **Package layout.** Each module is a Python package under `apps/`. Cross-module imports are permitted only from `apps.<module>.services` and `apps.<module>.dto` — never from `apps.<module>.models`.
2. **Import linter.** `import-linter` contracts encode the dependency table above; a forbidden import fails the build.
3. **Foreign keys across modules** are permitted (single database) but the *reading* module must not traverse the relation to read the other module's columns; it calls the service.
4. **Domain events.** Modules that need to react to another module's state changes subscribe to domain events (Section 8.9) rather than calling back synchronously. `booking` emits `BookingConfirmed`; `notify`, `finance` and `review` consume it.
5. **Architecture test.** A test asserts that every module's `services.py` exposes only DTOs and primitives — never ORM instances — across module boundaries.

6.6 Physical Deployment Topology (Production)



All compute is in private subnets. Only the ALB is internet-facing. Egress to external ports (payment, routing, push) traverses a NAT gateway with a fixed IP so that providers can allow-list it.

6.7 Key Architectural Principles

#	Principle	Practical rule
A1	The itinerary is the aggregate root of a journey	Nothing may confirm a component booking except through the trip checkout flow, so the basket cannot become internally inconsistent
A2	Money is append-only	No `UPDATE` on ledger rows; corrections are new, signed, reversing entries
A3	External providers are ports	Every third-party integration sits behind an interface with at least two conceivable implementations, plus a fake for tests
A4	State machines are explicit	Booking, payment, trip and assignment status transitions are declared in one table per machine and validated centrally
A5	Time is stored in UTC and rendered in destination-local time	`TIMESTAMPTZ` everywhere; never a naive datetime
A6	Idempotency for every mutating external-facing operation	`Idempotency-Key` header required on POST to payments, bookings and assignments
A7	Deterministic behaviour, always	Any ordering exposed to a user must have a total order and a stable tie-break
A8	No destination-specific code	Section 4.2

7 Domain Model and Database Architecture

7.1 Database Technology Decision

PostgreSQL 16 with the PostGIS extension is the single system of record.

Requirement	Why PostgreSQL satisfies it
Atomic multi-booking checkout	Full ACID transactions with `SERIALIZABLE` / `REPEATABLE READ` isolation where needed
Overlap-free availability	Native `daterange`/`tstzrange` types plus `EXCLUDE USING gist` constraints prevent double-booking at the storage layer, not just in application code
Geospatial proximity and containment	PostGIS `geography(Point,4326)`, `ST_DWithin`, GiST indexes
Financial correctness	`NUMERIC(14,2)` exact decimal arithmetic; no floating point for money
Flexible provider attributes	`JSONB` for amenity sets and provider-defined fields, with GIN indexes
Full-text catalogue search in V1	Built-in `tsvector` FTS removes the need for a separate search cluster until scale demands it
Audit and history	Table partitioning for `audit_log` and `driver_location`

Redis is used for cache, distributed locks, rate-limit counters, WebSocket pub/sub and Celery brokering. It is **never** the system of record for anything.

7.2 Design Conventions

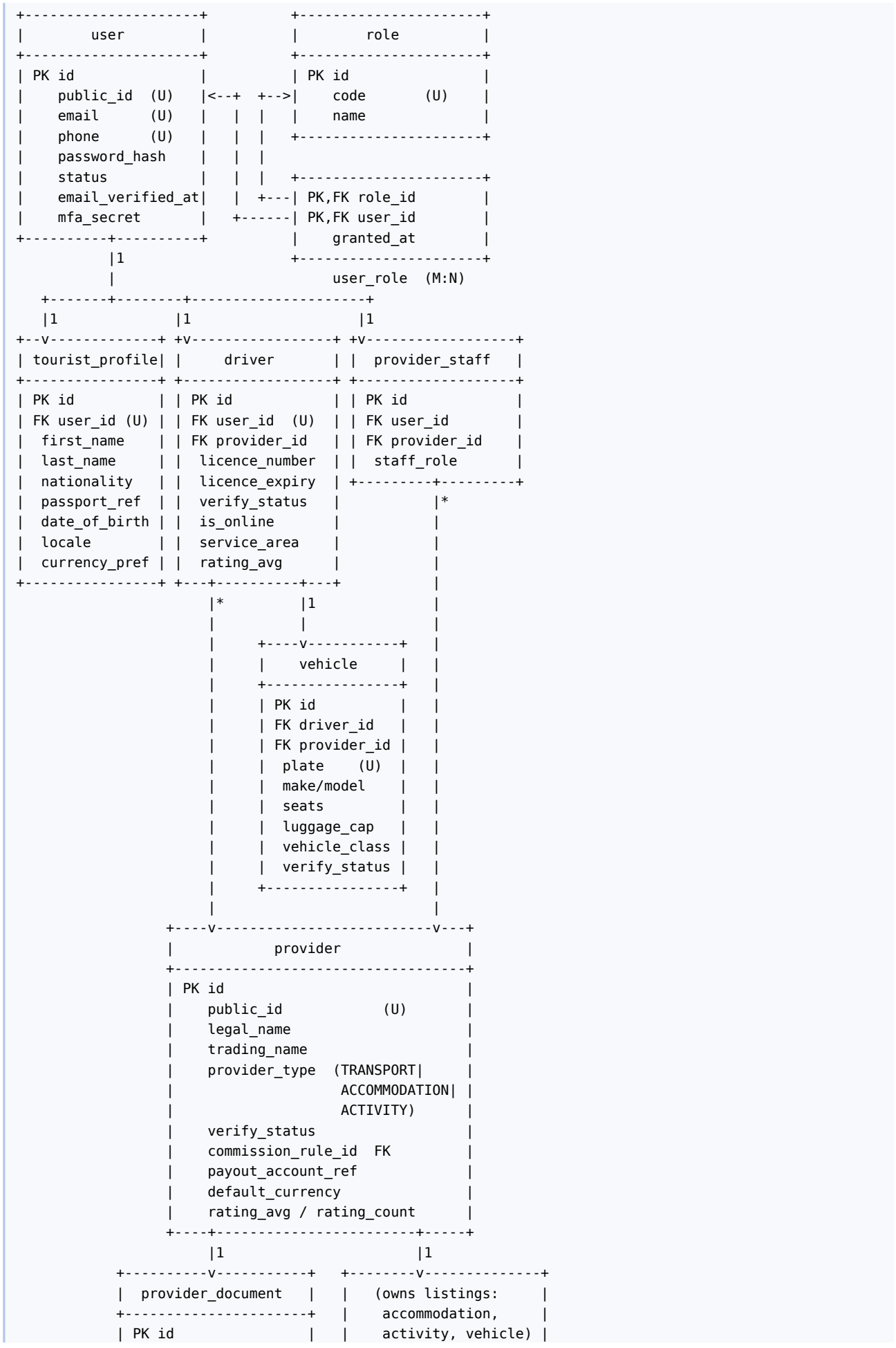
These conventions are mandatory across every table.

Convention	Rule
Primary keys	`id BIGSERIAL PRIMARY KEY` internally; every externally exposed entity also carries `public_id UUID NOT NULL UNIQUE DEFAULT gen_random_uuid()`. APIs expose `public_id` only. Sequential integers are never returned to clients.
Naming	`snake_case`, singular table names, FK named `<referenced_table>_id`
Timestamps	`created_at TIMESTAMPTZ NOT NULL DEFAULT now()`, `updated_at TIMESTAMPTZ NOT NULL DEFAULT now()` on every table; maintained by trigger
Soft deletion	`deleted_at TIMESTAMPTZ NULL` on catalogue and user-facing entities. Financial and booking records are **never** soft-deleted or hard-deleted. Default managers exclude soft-deleted rows.
Money	`amount_minor BIGINT` (minor units, e.g. cents) **or** `NUMERIC(14,2)`; the schema below uses `NUMERIC(14,2)` for readability. Every money column is accompanied by a `currency CHAR(3)` column. Never store money without its currency.
Enumerations	Stored as `VARCHAR` with a `CHECK` constraint, mirrored by a Python enum. Not native PG enums, because adding a value must not require an exclusive lock.
Geography	`geography(Point, 4326)` for all coordinates
Concurrency	`version INTEGER NOT NULL DEFAULT 0` on `booking`, `inventory_hold`, `room_availability`, `activity_departure`, `driver_assignment` for optimistic locking
Audit fields	`created_by_user_id`, `updated_by_user_id` on all administrator-mutable tables

7.3 Entity-Relationship Diagram

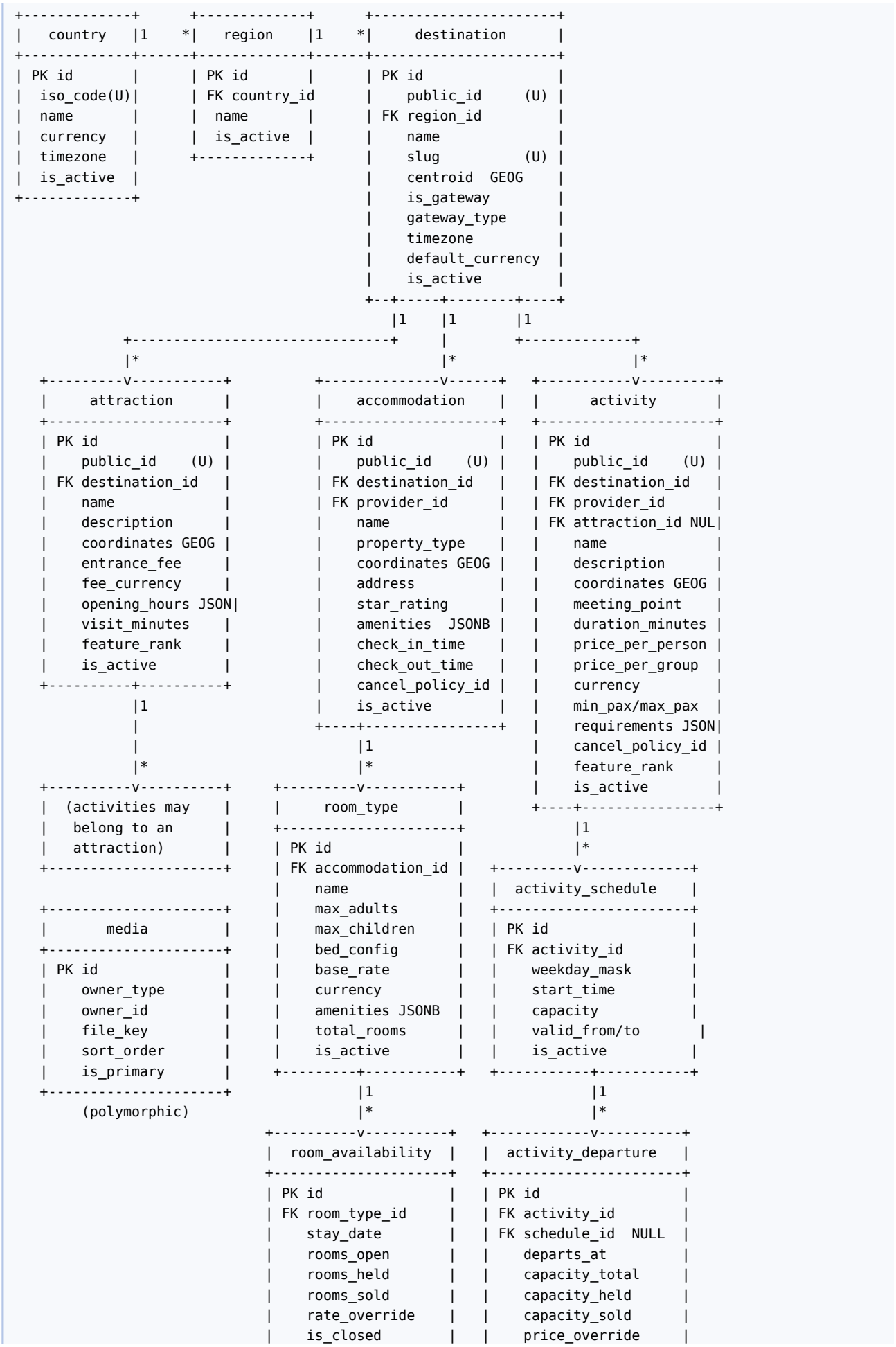
The ERD is presented in four connected views to remain legible. Together they are the complete model, and they match the schema in Section 7.5 exactly.

7.3.1 View 1 — Identity, Providers and Supply


user
PK id
public_id (U)
email (U)
phone (U)
password_hash
status
email_verified_at
mfa_secret
role
PK id
code (U)
name
user_role (M:N)
PK,FK role_id
PK,FK user_id
granted_at
tourist_profile
PK id
FK user_id (U)
first_name
last_name
nationality
passport_ref
date_of_birth
locale
currency_pref
driver
PK id
FK user_id (U)
FK provider_id
licence_number
licence_expiry
verify_status
is_online
service_area
rating_avg
provider_staff
PK id
FK user_id
FK provider_id
staff_role
vehicle
PK id
FK driver_id
FK provider_id
plate (U)
make/model
seats
luggage_cap
vehicle_class
verify_status
provider
PK id
public_id (U)
legal_name
trading_name
provider_type (TRANSPORT| ACCOMMODATION| ACTIVITY)
verify_status
commission_rule_id FK
payout_account_ref
default_currency
rating_avg / rating_count
provider_document
PK id
(owns listings: accommodation, activity, vehicle)

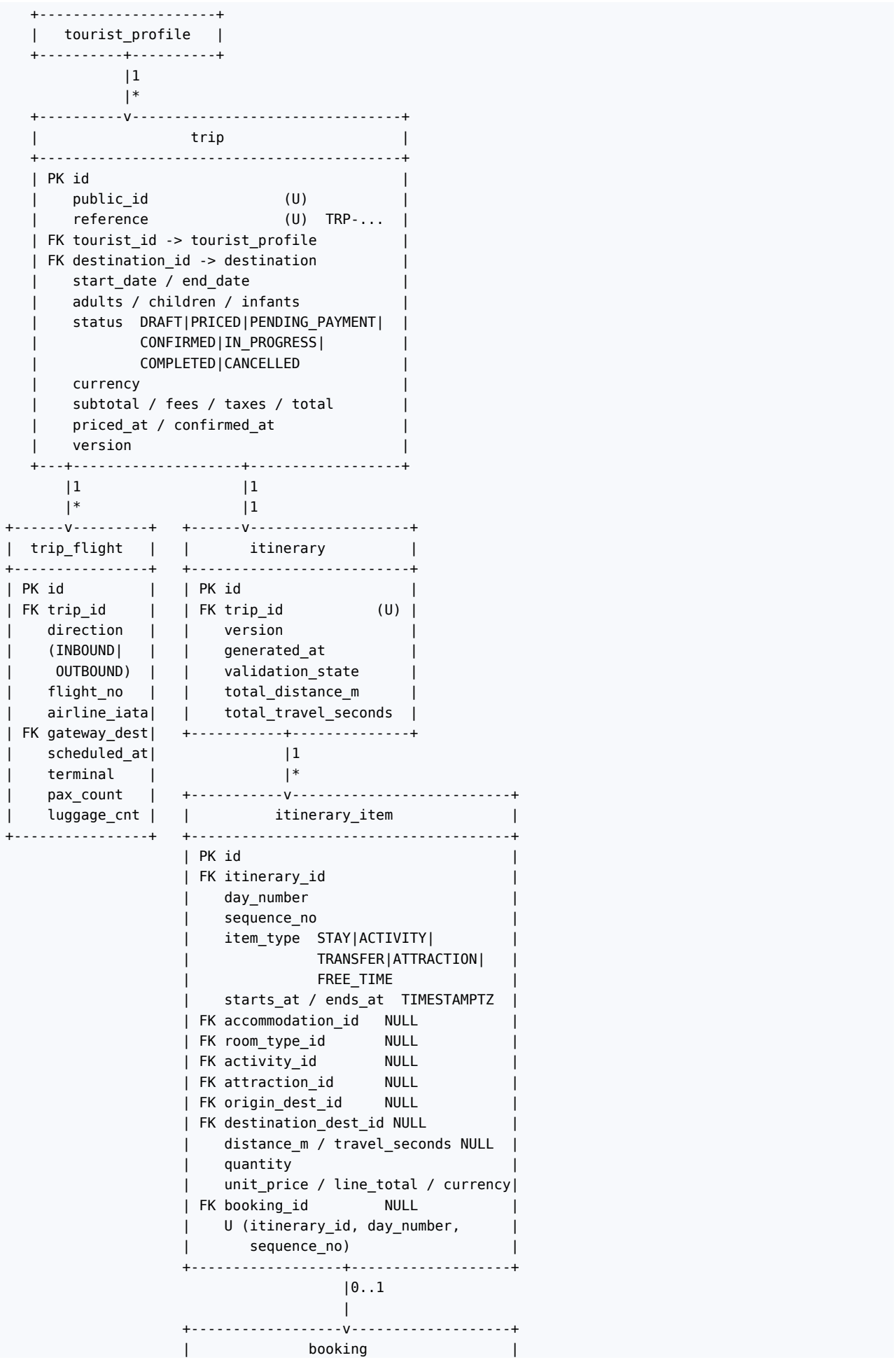
```
            | FK provider_id       |   +----------------------+
            |    doc_type          |
            |    file_key          |
            |    expires_on        |
            |    review_status     |
            +----------------------+
```

7.3.2 View 2 — Geography and Catalogue


country
PK id
iso_code(U)
name
currency
timezone
is_active
region
PK id
FK country_id
name
is_active
destination
PK id
public_id (U)
FK region_id
name
slug (U)
centroid GEOG
is_gateway
gateway_type
timezone
default_currency
is_active
attraction
PK id
public_id (U)
FK destination_id
name
description
coordinates GEOG
entrance_fee
fee_currency
opening_hours JSON
visit_minutes
feature_rank
is_active
accommodation
PK id
public_id (U)
FK destination_id
FK provider_id
name
property_type
coordinates GEOG
address
star_rating
amenities JSONB
check_in_time
check_out_time
cancel_policy_id
is_active
activity
PK id
public_id (U)
FK destination_id
FK provider_id
FK attraction_id NUL
name
description
coordinates GEOG
meeting_point
duration_minutes
price_per_person
price_per_group
currency
min_pax/max_pax
requirements JSON
cancel_policy_id
feature_rank
is_active
(activities may belong to an attraction)
media
PK id
owner_type
owner_id
file_key
sort_order
is_primary
(polymorphic)
room_type
PK id
FK accommodation_id
name
max_adults
max_children
bed_config
base_rate
currency
amenities JSONB
total_rooms
is_active
activity_schedule
PK id
FK activity_id
weekday_mask
start_time
capacity
valid_from/to
is_active
room_availability
PK id
FK room_type_id
stay_date
rooms_open
rooms_held
rooms_sold
rate_override
is_closed
activity_departure
PK id
FK activity_id
FK schedule_id NULL
departs_at
capacity_total
capacity_held
capacity_sold
price_override

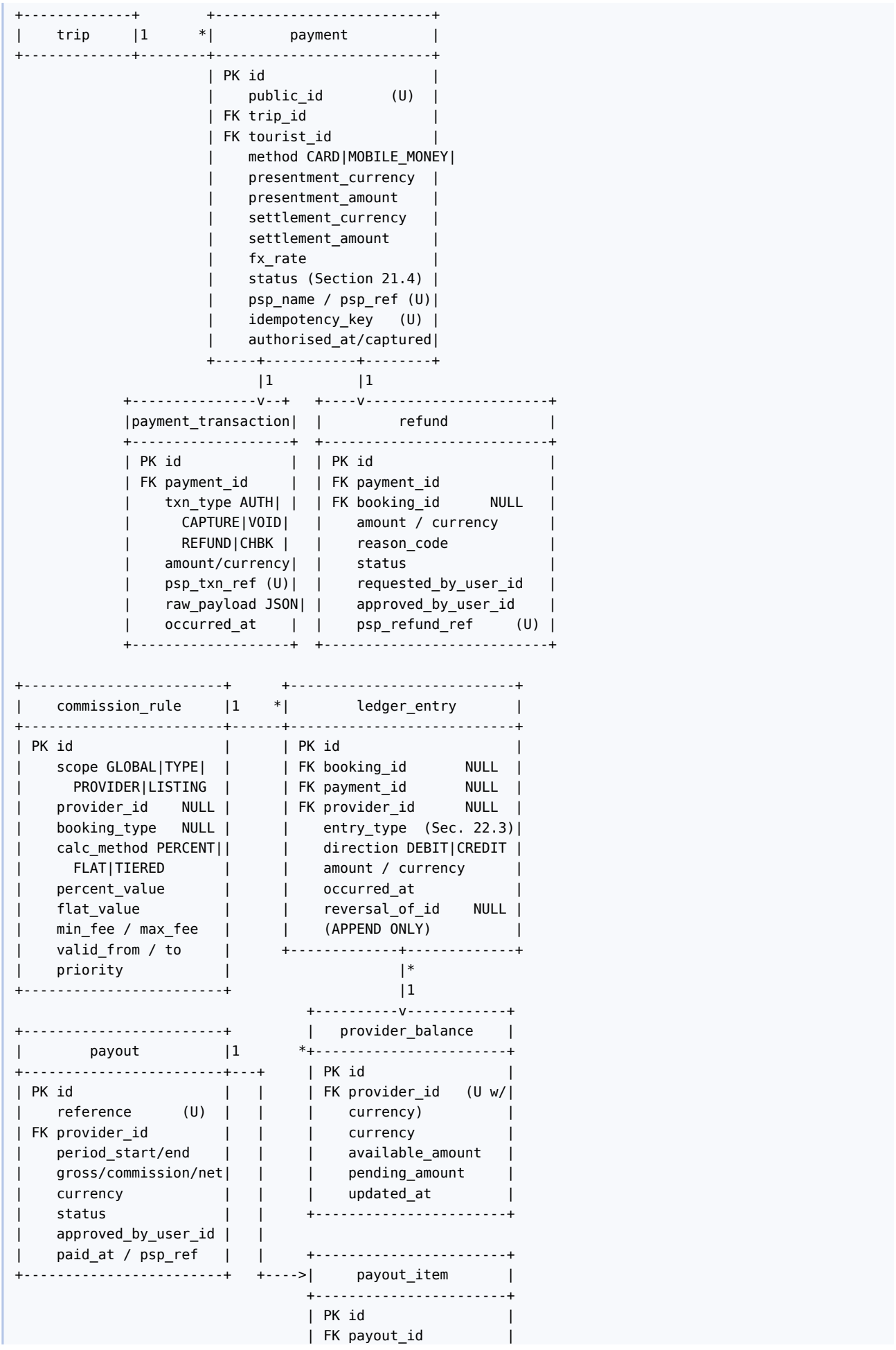
```
                              | U (room_type_id,    |   |    status             |
                              |    stay_date)       |   | U (activity_id,       |
                              +---------------------+   |    departs_at)        |
                                                        +-----------------------+
```

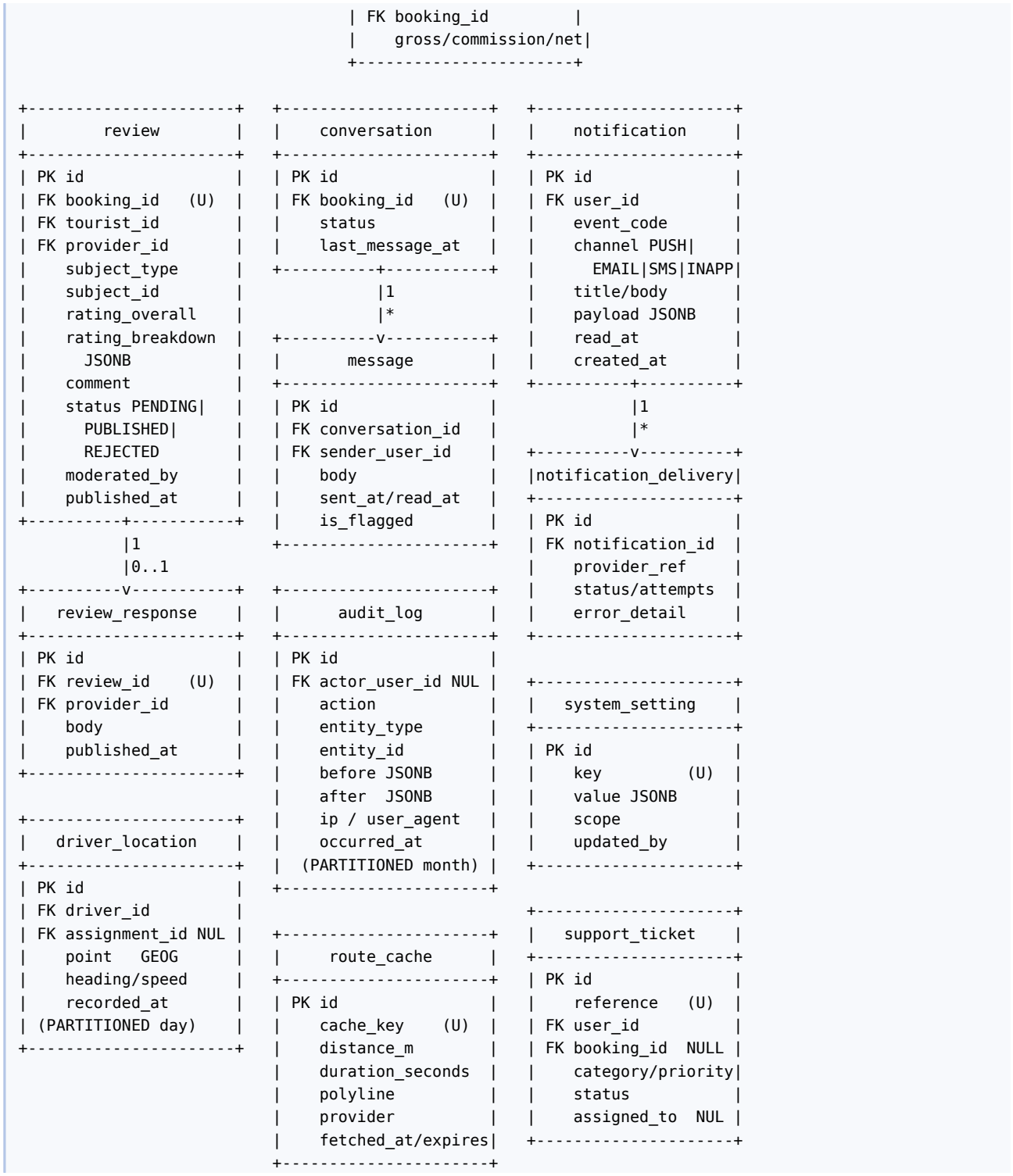
7.3.3 View 3 — Trip, Itinerary and Booking (the transactional core)


tourist_profile
trip
PK id
public_id (U)
reference (U) TRP-...
FK tourist_id -> tourist_profile
FK destination_id -> destination
start_date / end_date
adults / children / infants
status DRAFT|PRICED|PENDING_PAYMENT|
CONFIRMED|IN_PROGRESS|
COMPLETED|CANCELLED
currency
subtotal / fees / taxes / total
priced_at / confirmed_at
version
trip_flight
PK id
FK trip_id
direction
(INBOUND|
OUTBOUND)
flight_no
airline_iata
FK gateway_dest
scheduled_at
terminal
pax_count
luggage_cnt
itinerary
PK id
FK trip_id (U)
version
generated_at
validation_state
total_distance_m
total_travel_seconds
itinerary_item
PK id
FK itinerary_id
day_number
sequence_no
item_type STAY|ACTIVITY|
TRANSFER|ATTRACTION|
FREE_TIME
starts_at / ends_at TIMESTAMPTZ
FK accommodation_id NULL
FK room_type_id NULL
FK activity_id NULL
FK attraction_id NULL
FK origin_dest_id NULL
FK destination_dest_id NULL
distance_m / travel_seconds NULL
quantity
unit_price / line_total / currency
FK booking_id NULL
U (itinerary_id, day_number,
sequence_no)
booking


booking
PK id
public_id / reference (U)
FK trip_id
FK tourist_id
FK provider_id
booking_type ACCOMMODATION|ACTIVITY|TRANSFER
status (see Section 20)
starts_at / ends_at
pax_count
gross_amount / currency
commission_amount / net_amount
cancel_policy_id FK
cancelled_at / cancel_reason
version
booking_accommodation
PK,FK booking_id
FK room_type
check_in
check_out
nights
rooms
guests
booking_activity
PK,FK booking_id
FK activity
FK departure
pax_adult
pax_child
meeting_at
booking_transfer
PK,FK booking_id
FK origin_dest_id
FK dest_dest_id
pickup_lat/lng
dropoff_lat/lng
pickup_at
distance_m
travel_seconds
vehicle_class
FK tariff_id
luggage_count
is_airport_transfer
FK trip_flight_id NUL
driver_assignment
PK id
FK booking_id (U)
FK driver_id
FK vehicle_id
status
offered_at/accepted_at
arrived_at/started_at
completed_at
start_odo / end_odo
version
booking_status_history
PK id
FK booking_id
from_status/to_status
actor_user_id FK
reason
occurred_at
inventory_hold
PK id
hold_token (U)
FK trip_id
resource_type
resource_id
date_from / date_to
quantity
expires_at
status HELD|COMMITTED|RELEASED|EXPIRED

7.3.4 View 4 — Payment, Finance, Engagement and Operations


trip
payment
PK id
public_id (U)
FK trip_id
FK tourist_id
method CARD|MOBILE_MONEY
presentment_currency
presentment_amount
settlement_currency
settlement_amount
fx_rate
status (Section 21.4)
psp_name / psp_ref (U)
idempotency_key (U)
authorised_at/captured
payment_transaction
PK id
FK payment_id
txn_type AUTH|CAPTURE|VOID|REFUND|CHBK
amount/currency
psp_txn_ref (U)
raw_payload JSON
occurred_at
refund
PK id
FK payment_id
FK booking_id NULL
amount / currency
reason_code
status
requested_by_user_id
approved_by_user_id
psp_refund_ref (U)
commission_rule
PK id
scope GLOBAL|TYPE|PROVIDER|LISTING
provider_id NULL
booking_type NULL
calc_method PERCENT|FLAT|TIERED
percent_value
flat_value
min_fee / max_fee
valid_from / to
priority
ledger_entry
PK id
FK booking_id NULL
FK payment_id NULL
FK provider_id NULL
entry_type (Sec. 22.3)
direction DEBIT|CREDIT
amount / currency
occurred_at
reversal_of_id NULL
(APPEND ONLY)
provider_balance
PK id
FK provider_id (U w/ currency)
currency
available_amount
pending_amount
updated_at
payout
PK id
reference (U)
FK provider_id
period_start/end
gross/commission/net
currency
status
approved_by_user_id
paid_at / psp_ref
payout_item
PK id
FK payout_id



7.4 Relationship and Cardinality Register

#	Parent	Child	Cardinality	Delete rule	Note
R1	user	tourist_profile	1 : 0..1	RESTRICT	A user is a tourist only if this row exists
R2	user	driver	1 : 0..1	RESTRICT	
R3	user	provider_staff	1 : 0..*	CASCADE	A person may work for several providers
R4	provider	driver	1 : 0..*	RESTRICT	Independent drivers are their own provider org

#	Parent	Child	Cardinality	Delete rule	Note
R5	driver	vehicle	1 : 1..*	RESTRICT	At least one vehicle required to go online
R6	country	region	1 : 1..*	RESTRICT	
R7	region	destination	1 : 1..*	RESTRICT	
R8	destination	attraction	1 : 0..*	RESTRICT	
R9	destination	accommodation	1 : 0..*	RESTRICT	
R10	destination	activity	1 : 0..*	RESTRICT	
R11	attraction	activity	0..1 : 0..*	SET NULL	Activity may be standalone
R12	accommodation	room_type	1 : 1..*	CASCADE	
R13	room_type	room_availability	1 : 0..*	CASCADE	One row per room-type per date
R14	activity	activity_schedule	1 : 0..*	CASCADE	
R15	activity	activity_departure	1 : 0..*	CASCADE	Materialised departures
R16	tourist_profile	trip	1 : 0..*	RESTRICT	
R17	destination	trip	1 : 0..*	RESTRICT	Single destination in V1 (see 4.3)
R18	trip	itinerary	1 : 1	CASCADE	Exactly one current itinerary per trip
R19	trip	trip_flight	1 : 0..2	CASCADE	Inbound and/or outbound
R20	itinerary	itinerary_item	1 : 0..*	CASCADE	
R21	itinerary_item	booking	0..* : 0..1	SET NULL	An activity booking may cover 1 item; a stay booking covers N nightly items
R22	trip	booking	1 : 0..*	RESTRICT	The basket
R23	provider	booking	1 : 0..*	RESTRICT	
R24	booking	booking_accommodation / booking_activity / booking_transfer	1 : 0..1	CASCADE	Exactly one subtype must exist, matching `booking_type`
R25	booking_transfer	driver_assignment	1 : 0..1	RESTRICT	Assignment created at confirmation or by dispatch
R26	booking	booking_status_history	1 : 1..*	CASCADE	Append-only
R27	trip	payment	1 : 0..*	RESTRICT	Retries create new payment rows
R28	payment	payment_transaction	1 : 1..*	RESTRICT	Append-only
R29	payment	refund	1 : 0..*	RESTRICT	
R30	booking	ledger_entry	1 : 0..*	RESTRICT	Append-only
R31	provider	provider_balance	1 : 1..*	RESTRICT	One per currency
R32	provider	payout	1 : 0..*	RESTRICT	

#	Parent	Child	Cardinality	Delete rule	Note
R33	payout	payout_item	1 : 1..*	CASCADE	
R34	booking	review	1 : 0..1	RESTRICT	One review per completed booking
R35	review	review_response	1 : 0..1	CASCADE	
R36	booking	conversation	1 : 0..1	CASCADE	
R37	conversation	message	1 : 0..*	CASCADE	
R38	user	notification	1 : 0..*	CASCADE	
R39	notification	notification_delivery	1 : 1..*	CASCADE	One per channel attempt
R40	driver	driver_location	1 : 0..*	CASCADE	Partitioned, retention-limited
R41	commission_rule	provider	0..1 : 0..*	SET NULL	Provider-specific override
R42	booking	inventory_hold	0..* via trip	—	Holds are keyed to trip during checkout
R43	cancellation_policy	accommodation / activity / booking	1 : 0..*	RESTRICT	Policy snapshot copied onto booking

7.5 Table Specifications

Only columns with non-obvious semantics are annotated. Every table additionally carries `created_at`, `updated_at` and, where marked, `deleted_at`, per Section 7.2.

7.5.1 `user`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		Internal key
public_id	UUID			N	gen_random_uuid()	External identifier
email	CITEXT			N		Unique, case-insensitive
phone_e164	VARCHAR(20)			Y		E.164; unique when not null
password_hash	VARCHAR(255)			N		Argon2id
status	VARCHAR(20)			N	‘PENDING’	PENDING/ACTIVE/SUSPENDED/CLOSED
email_verified_at	TIMESTAMPTZ			Y		Null until verified
phone_verified_at	TIMESTAMPTZ			Y		
mfa_secret	BYTEA			Y		Encrypted TOTP seed; required for staff/provider roles
failed_login_count	SMALLINT			N	0	Lockout counter
locked_until	TIMESTAMPTZ			Y		Set by lockout policy
last_login_at	TIMESTAMPTZ			Y		
deleted_at	TIMESTAMPTZ			Y		Soft delete / erasure marker

Indexes: `UNIQUE(email) WHERE deleted_at IS NULL`; `UNIQUE(phone_e164) WHERE phone_e164 IS NOT NULL AND deleted_at IS NULL`; `INDEX(status)`.

7.5.2 `tourist_profile`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
user_id	BIGINT		Y	N		→ user.id, UNIQUE
first_name	VARCHAR(80)			N		
last_name	VARCHAR(80)			N		
nationality	CHAR(2)			Y		ISO 3166-1 alpha-2
date_of_birth	DATE			Y		Used for age-restricted activities
passport_reference	BYTEA			Y		Encrypted at rest; collected only when a booked service requires it (BR-072)
emergency_contact	JSONB			Y		Name and phone
locale	VARCHAR(10)			N	'en'	BCP 47
preferred_currency	CHAR(3)			N	'USD'	ISO 4217
interest_tags	VARCHAR(30)[]			Y	'{}'	Drives deterministic catalogue ordering (5.5)
marketing_opt_in	BOOLEAN			N	false	

7.5.3 `provider`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
public_id	UUID			N	gen_random_uuid()	
legal_name	VARCHAR(200)			N		Registered name
trading_name	VARCHAR(200)			N		Displayed name
provider_type	VARCHAR(20)			N		TRANSPORT/ACCOMMODATION/ACTIVITY
contact_email	CITEXT			N		
contact_phone	VARCHAR(20)			N		
region_id	BIGINT		Y	N		→ region.id, operating region
verify_status	VARCHAR(20)			N	'DRAFT'	DRAFT/SUBMITTED/UNDER_REVIEW/VERIFIED/REJECTED/SUSPENDED
verified_at	TIMESTAMPTZ			Y		
commission_rule_id	BIGINT		Y	Y		→ commission_rule.id; null uses resolution order (22.2)
payout_account_ref	VARCHAR(120)			Y		Token from PSP; never raw bank details
payout_currency	CHAR(3)			N	'TZS'	
rating_avg	NUMERIC(3,2)			N	0.00	Denormalised from rating_aggregate
rating_count	INTEGER			N	0	
deleted_at	TIMESTAMPTZ			Y		

Constraint: a provider may not own listings of a type inconsistent with `provider_type` (enforced by trigger and application service).

7.5.4 `driver`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
user_id	BIGINT		Y	N		→ user.id, UNIQUE
provider_id	BIGINT		Y	N		→ provider.id (TRANSPORT)
licence_number	VARCHAR(60)			N		Encrypted at rest
licence_expires_on	DATE			N		Blocks dispatch when past
national_id_ref	BYTEA			Y		Encrypted
verify_status	VARCHAR(20)			N	'DRAFT'	As provider
is_online	BOOLEAN			N	false	Availability toggle
service_area	geography(Polygon,4326)			Y		Optional operating boundary
home_destination_id	BIGINT		Y	Y		→ destination.id, dispatch anchor
languages	VARCHAR(10)[]			Y	'{en}'	Displayed to tourist
rating_avg	NUMERIC(3,2)			N	0.00	
acceptance_rate	NUMERIC(5,2)			N	100.00	Rolling 30-day; input to dispatch score
completed_trips	INTEGER			N	0	

7.5.5 `vehicle`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
driver_id	BIGINT		Y	N		→ driver.id
provider_id	BIGINT		Y	N		→ provider.id
plate_number	VARCHAR(20)			N		UNIQUE
make	VARCHAR(40)			N		
model	VARCHAR(40)			N		
year	SMALLINT			N		
colour	VARCHAR(20)			N		Aids airport identification
seat_capacity	SMALLINT			N		Excludes driver
luggage_capacity	SMALLINT			N	0	Large bags
vehicle_class	VARCHAR(20)			N	'STANDARD'	STANDARD/COMFORT/VAN/ MINIBUS
has_air_conditioning	BOOLEAN			N	true	
insurance_expires_on	DATE			N		Blocks dispatch when past
inspection_expires_on	DATE			N		Blocks dispatch when past
verify_status	VARCHAR(20)			N	'DRAFT'	
is_active	BOOLEAN			N	true	

7.5.6 `destination`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
public_id	UUID			N	gen_random_uuid()	
region_id	BIGINT		Y	N		→ region.id
name	VARCHAR(120)			N		
slug	VARCHAR(140)			N		UNIQUE, URL-safe
summary	TEXT			Y		Catalogue copy
centroid	geography(Point,4326)			N		Proximity anchor

Field	Data Type	PK	FK	Null	Default	Description
is_gateway	BOOLEAN			N	false	Arrival/departure node
gateway_type	VARCHAR(20)			Y		AIRPORT/SEAPORT/ LAND_BORDER
gateway_code	VARCHAR(10)			Y		e.g. ZNZ
timezone	VARCHAR(60)			N		IANA zone
default_currency	CHAR(3)			N		
launch_date	DATE			Y		Visibility gate
feature_rank	SMALLINT			N	100	Deterministic ordering
is_active	BOOLEAN			N	false	
deleted_at	TIMESTAMPTZ			Y		

Indexes: `GIST(centroid)`; `INDEX(region_id, is_active)`; `UNIQUE(slug)`; partial `UNIQUE(gateway_code) WHERE is_gateway`.

7.5.7 `accommodation` and `room_type`

`accommodation` fields of note: `provider_id`, `destination_id`, `property_type` (HOTEL/RESORT/LODGE/GUESTHOUSE/APARTMENT), `coordinates`, `address_line`, `star_rating SMALLINT NULL`, `amenities JSONB` (GIN indexed), `check_in_time TIME`, `check_out_time TIME`, `cancellation_policy_id`, `child_policy JSONB`, `is_active`, `deleted_at`.

`room_type`:

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
accommodation_id	BIGINT		Y	N		→ accommodation.id
name	VARCHAR(120)			N		e.g. Deluxe Sea View
max_adults	SMALLINT			N		Occupancy validation
max_children	SMALLINT			N	0	
bed_configuration	VARCHAR(80)			Y		
size_sqm	SMALLINT			Y		
base_rate	NUMERIC(14,2)			N		Nightly rate before overrides
currency	CHAR(3)			N		
total_rooms	SMALLINT			N		Physical inventory ceiling
amenities	JSONB			Y	'{}'	
is_active	BOOLEAN			N	true	

7.5.8 `room_availability`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
room_type_id	BIGINT		Y	N		→ room_type.id
stay_date	DATE			N		The night being sold
rooms_open	SMALLINT			N		Provider-published availability
rooms_held	SMALLINT			N	0	Active inventory holds
rooms_sold	SMALLINT			N	0	Confirmed bookings
rate_override	NUMERIC(14,2)			Y		Overrides room_type.base_rate
min_nights	SMALLINT			Y		Minimum length of stay
is_closed	BOOLEAN			N	false	Stop-sell

Field	Data Type	PK	FK	Null	Default	Description
version	INTEGER			N	0	Optimistic lock

Constraints: `UNIQUE(room_type_id, stay_date)`; `CHECK (rooms_held + rooms_sold <= rooms_open)`; `CHECK (rooms_open <= (SELECT total_rooms ...))` enforced by trigger.

7.5.9 `activity` and `activity_departure`

`activity` fields of note: `provider_id`, `destination_id`, `attraction_id NULL`, `name`, `description`, `coordinates`, `meeting_point_text`, `duration_minutes`, `price_per_person NUMERIC(14,2)`, `price_per_group NUMERIC(14,2) NULL`, `currency`, `min_pax`, `max_pax`, `min_age SMALLINT NULL`, `requirements JSONB`, `inclusions JSONB`, `exclusions JSONB`, `cancellation_policy_id`, `booking_cutoff_hours SMALLINT`, `feature_rank`, `is_active`, `deleted_at`.

`activity_departure`:

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
activity_id	BIGINT		Y	N		→ activity.id
schedule_id	BIGINT		Y	Y		→ activity_schedule.id; null for ad-hoc
departs_at	TIMESTAMPTZ			N		Exact departure instant
capacity_total	SMALLINT			N		
capacity_held	SMALLINT			N	0	
capacity_sold	SMALLINT			N	0	
price_override	NUMERIC(14,2)			Y		
status	VARCHAR(20)			N	'OPEN'	OPEN/FULL/CANCELLED/CLOSED
version	INTEGER			N	0	

Constraints: `UNIQUE(activity_id, departs_at)`; `CHECK (capacity_held + capacity_sold <= capacity_total)`.

7.5.10 `trip`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
public_id	UUID			N	gen_random_uuid()	
reference	VARCHAR(20)			N		UNIQUE, human-facing `TRP-YYYY-NNNNNNN`
tourist_id	BIGINT		Y	N		→ tourist_profile.id
destination_id	BIGINT		Y	N		→ destination.id
title	VARCHAR(140)			Y		Tourist-supplied label
start_date	DATE			N		First day
end_date	DATE			N		Last day
adults	SMALLINT			N	1	≥ 1
children	SMALLINT			N	0	
infants	SMALLINT			N	0	
status	VARCHAR(20)			N	'DRAFT'	See Section 20.5
currency	CHAR(3)			N		Presentment currency, locked at PRICED
subtotal_amount	NUMERIC(14,2)			N	0	Sum of line totals
fee_amount	NUMERIC(14,2)			N	0	Platform service fee
tax_amount	NUMERIC(14,2)			N	0	
total_amount	NUMERIC(14,2)			N	0	Charged amount

Field	Data Type	PK	FK	Null	Default	Description
priced_at	TIMESTAMPTZ			Y		Start of the quote validity window
quote_expires_at	TIMESTAMPTZ			Y		priced_at + configured TTL
confirmed_at	TIMESTAMPTZ			Y		
cancelled_at	TIMESTAMPTZ			Y		
version	INTEGER			N	0	

Constraints: `CHECK (end_date >= start_date)`; `CHECK (adults >= 1)`; `CHECK (total_amount = subtotal_amount + fee_amount + tax_amount)`.

7.5.11 `itinerary_item`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
itinerary_id	BIGINT		Y	N		→ itinerary.id
day_number	SMALLINT			N		1-based, relative to trip.start_date
sequence_no	SMALLINT			N		Order within the day
item_type	VARCHAR(20)			N		STAY/ACTIVITY/TRANSFER/ ATTRACTION/FREE_TIME
title	VARCHAR(160)			N		Rendered label
starts_at	TIMESTAMPTZ			N		
ends_at	TIMESTAMPTZ			N		
accommodation_id	BIGINT		Y	Y		Set when STAY
room_type_id	BIGINT		Y	Y		Set when STAY
activity_id	BIGINT		Y	Y		Set when ACTIVITY
activity_departure_id	BIGINT		Y	Y		Set when ACTIVITY
attraction_id	BIGINT		Y	Y		Set when ATTRACTION
origin_destination_id	BIGINT		Y	Y		Set when TRANSFER
target_destination_id	BIGINT		Y	Y		Set when TRANSFER
origin_point	geography(Point,4326)			Y		Precise pickup
target_point	geography(Point,4326)			Y		Precise drop-off
distance_m	INTEGER			Y		From routing provider
travel_seconds	INTEGER			Y		From routing provider
quantity	SMALLINT			N	1	Rooms, seats or vehicles
pax_count	SMALLINT			Y		
unit_price	NUMERIC(14,2)			Y		
line_total	NUMERIC(14,2)			Y		
currency	CHAR(3)			Y		
booking_id	BIGINT		Y	Y		Populated on basket creation
is_locked	BOOLEAN			N	false	True once the covering booking is confirmed

Constraints: `UNIQUE(itinerary_id, day_number, sequence_no)`; `CHECK (ends_at >= starts_at)`; a `CHECK` per `item_type` asserting that exactly the correct subset of nullable FKs is populated.

7.5.12 `booking`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
public_id	UUID			N	gen_random_uuid()	

Field	Data Type	PK	FK	Null	Default	Description
reference	VARCHAR(20)			N		UNIQUE, `BKG-YYYY-NNNNNNN`
trip_id	BIGINT		Y	N		→ trip.id
tourist_id	BIGINT		Y	N		→ tourist_profile.id (denormalised for query)
provider_id	BIGINT		Y	N		→ provider.id
booking_type	VARCHAR(20)			N		ACCOMMODATION/ACTIVITY/TRANSFER
status	VARCHAR(20)			N	'PENDING'	Section 20.2
starts_at	TIMESTAMPTZ			N		Service start
ends_at	TIMESTAMPTZ			N		Service end
pax_count	SMALLINT			N		
gross_amount	NUMERIC(14,2)			N		Tourist-paid amount for this component
currency	CHAR(3)			N		
commission_rate	NUMERIC(5,2)			Y		Snapshot of applied rate
commission_amount	NUMERIC(14,2)			Y		
net_amount	NUMERIC(14,2)			Y		gross − commission; provider entitlement
cancellation_policy_snapshot	JSONB			N		Frozen policy at confirmation (BR-041)
confirmed_at	TIMESTAMPTZ			Y		
cancelled_at	TIMESTAMPTZ			Y		
cancellation_reason	VARCHAR(40)			Y		Coded reason
completed_at	TIMESTAMPTZ			Y		
version	INTEGER			N	0	

Indexes: `INDEX(trip_id)`, `INDEX(provider_id, status, starts_at)`, `INDEX(tourist_id, status)`, `INDEX(status, starts_at)` for scheduler sweeps.

7.5.13 `payment`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
public_id	UUID			N	gen_random_uuid()	
trip_id	BIGINT		Y	N		→ trip.id
tourist_id	BIGINT		Y	N		
method	VARCHAR(20)			N		CARD/MOBILE_MONEY
presentment_currency	CHAR(3)			N		Currency shown to the tourist
presentment_amount	NUMERIC(14,2)			N		
settlement_currency	CHAR(3)			N		Currency the PSP settles in
settlement_amount	NUMERIC(14,2)			Y		Known after capture
fx_rate	NUMERIC(18,8)			Y		Rate applied, stored for audit
status	VARCHAR(20)			N	'INITIATED'	Section 21.4
psp_name	VARCHAR(40)			N		Adapter identity

Field	Data Type	PK	FK	Null	Default	Description
psp_reference	VARCHAR(120)			Y		UNIQUE when not null
idempotency_key	VARCHAR(64)			N		UNIQUE
failure_code	VARCHAR(40)			Y		Normalised failure taxonomy
authorised_at	TIMESTAMPTZ			Y		
captured_at	TIMESTAMPTZ			Y		

The Platform stores **no** primary account number, CVV, expiry or full cardholder name. Only the PSP token, last four digits and card brand may be stored, in `payment_instrument_summary JSONB`.

7.5.14 `ledger_entry` (append-only)

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
entry_type	VARCHAR(30)			N		Section 22.3
direction	VARCHAR(6)			N		DEBIT/CREDIT
amount	NUMERIC(14,2)			N		Always positive
currency	CHAR(3)			N		
booking_id	BIGINT		Y	Y		
payment_id	BIGINT		Y	Y		
provider_id	BIGINT		Y	Y		
payout_id	BIGINT		Y	Y		
reversal_of_id	BIGINT		Y	Y		Self-FK; set on correcting entries
memo	VARCHAR(200)			Y		
occurred_at	TIMESTAMPTZ			N	now()	

Rules: `UPDATE` and `DELETE` are revoked at the database role level. A correction is a new entry referencing `reversal_of_id`. A nightly job asserts that debits equal credits per booking and per currency (BR-064).

7.5.15 `audit_log`

Field	Data Type	PK	FK	Null	Default	Description
id	BIGSERIAL	Y		N		
actor_user_id	BIGINT		Y	Y		Null for system actions
actor_role	VARCHAR(30)			Y		Role in effect
action	VARCHAR(60)			N		e.g. `booking.cancel`
entity_type	VARCHAR(60)			N		
entity_id	BIGINT			N		
before_state	JSONB			Y		Redacted of secrets
after_state	JSONB			Y		
ip_address	INET			Y		
user_agent	VARCHAR(300)			Y		
request_id	UUID			Y		Correlates to application logs
occurred_at	TIMESTAMPTZ			N	now()	Partition key

Partitioned monthly by `occurred_at`. Retention: 7 years for financial actions, 2 years otherwise.

7.6 Indexing Strategy

Table	Index	Purpose
destination	GIST(centroid)	Nearest-destination and radius queries

Table	Index	Purpose
accommodation	GIST(coordinates); GIN(amenities)	Map search; amenity filters
activity	GIST(coordinates); GIN(to_tsvector(name || description))	Map search; catalogue full-text
room_availability	UNIQUE(room_type_id, stay_date); INDEX(stay_date) WHERE NOT is_closed	Availability scan; nightly sweeps
activity_departure	UNIQUE(activity_id, departs_at); INDEX(departs_at) WHERE status='OPEN'	Departure search
booking	INDEX(provider_id, status, starts_at); INDEX(status, starts_at)	Provider console; scheduler
itinerary_item	UNIQUE(itinerary_id, day_number, sequence_no)	Deterministic ordering
inventory_hold	INDEX(expires_at) WHERE status='HELD'	Expiry sweeper
driver_location	BRIN(recorded_at); GIST(point)	Time-series scan; proximity
payment	UNIQUE(idempotency_key); UNIQUE(psp_reference)	Idempotency; webhook matching
ledger_entry	INDEX(provider_id, currency, occurred_at); INDEX(booking_id)	Statement and reconciliation
audit_log	INDEX(entity_type, entity_id); INDEX(actor_user_id, occurred_at)	Investigation
message	INDEX(conversation_id, sent_at DESC)	Thread paging
review	INDEX(subject_type, subject_id, status)	Rating aggregation

7.7 Data Integrity Mechanisms

Mechanism	Applied to	Effect
Foreign keys with explicit delete rules	All relations (Section 7.4)	No orphan rows; financial rows cannot be cascaded away
`CHECK` constraints	Money non-negativity, date ordering, capacity sums, enum domains	Invalid states impossible at storage layer
Partial unique indexes	Soft-deleted rows excluded from uniqueness	Re-registration after account closure remains possible
`EXCLUDE USING gist`	`driver_assignment` over `(driver_id WITH =, tstzrange(starts_at, ends_at) WITH &&)`	A driver cannot hold two overlapping assignments
Optimistic locking (`version`)	booking, availability, departures, assignments	Lost-update prevention under concurrency
Row locking (`SELECT … FOR UPDATE`)	Availability rows during hold and commit	Serialises the critical section (Section 20.7)
Triggers	`updated_at` maintenance; capacity ceiling; ledger immutability	Invariants independent of application code
Advisory locks	Payout batch generation per provider	Prevents duplicate batches

7.8 Data Retention and Archival

Data class	Live retention	Archive	Basis
`driver_location`	30 days	Aggregated trip polyline retained with booking; raw points purged	Location minimisation (Section 31.5)
`audit_log` (financial)	7 years	Cold object storage after 13 months	Tax and accounting obligations
`audit_log` (other)	24 months	Purged	Data minimisation
Bookings, payments, ledger, payouts	Indefinite	Partitioned by year after 3 years	Financial record-keeping
Messages	24 months after trip completion	Purged	Minimisation
Closed tourist accounts	Anonymised at 30 days after closure request; financial rows retain a pseudonymous key	—	Erasure vs. legal retention balance (Section 31.7)
Notification bodies	90 days	Metadata retained	Volume control

8 Backend Architecture

8.1 Technology Selection

Concern	Selection	Justification
Language / runtime	Python 3.12	Mature ecosystem for geospatial, financial and background-job work; large local hiring pool
Web framework	Django 5.x	Batteries-included ORM, migrations, admin scaffolding, mature auth; the built-in admin accelerates internal tooling without becoming the customer-facing console
API layer	Django REST Framework 3.15	Serializer/validation model, permission classes, schema generation via `drf-spectacular`
Async/WebSocket	Django Channels 4 (ASGI)	Shares models and auth with the REST app; avoids a second stack
Background jobs	Celery 5 + Redis broker	Retries, rate limiting, scheduled beats, dead-letter routing
Database	PostgreSQL 16 + PostGIS 3.4	Section 7.1
Cache / locks / pub-sub	Redis 7	
Object storage	S3-compatible	Media and provider documents
Schema/API contract	OpenAPI 3.1 generated from code	Contract is generated, never hand-maintained

Alternatives assessed and rejected for V1: Node.js/NestJS (equally viable; rejected because the team's stronger competence and the richer geospatial/financial library set favour Python), Go (best runtime performance; rejected because development velocity matters more than throughput at this scale), Spring Boot (heavier operational and cognitive overhead than the team can carry).

8.2 Layering

Every module follows the same five-layer structure. Layer rules are enforced by review and by the import linter.

```
 HTTP / WebSocket
      |
+-----v-----------------------------------------------+
| 1. Interface layer   views.py, consumers.py,        |
|    serializers.py, permissions.py, urls.py          |
|    - request parsing, authn/authz, response shaping |
|    - NO business logic, NO ORM queries              |
+-----v-----------------------------------------------+
| 2. Application layer  services.py, use_cases/       |
|    - orchestrates a use case in ONE transaction     |
|    - emits domain events                            |
|    - the only layer other modules may call          |
+-----v-----------------------------------------------+
| 3. Domain layer  domain/rules.py, domain/pricing.py,|
|    domain/state_machine.py, domain/policies.py      |
|    - pure functions over value objects              |
|    - NO I/O, fully unit-testable                    |
+-----v-----------------------------------------------+
| 4. Data-access layer  repositories.py, models.py,   |
|    selectors.py                                     |
|    - all ORM access; returns DTOs to layer 2        |
+-----v-----------------------------------------------+
| 5. Infrastructure  adapters/ (routing, payment,     |
|    push, email, sms, storage), tasks.py             |
|    - implements ports declared in layer 3           |
+-----------------------------------------------------+
```

The strict rule that pays for itself: **pricing, policy, validation and state-transition logic live in layer 3 as pure functions.** They take primitives and value objects and return results. This makes the most audit-sensitive logic in the system testable without a database and reviewable without a debugger.

8.3 Request Lifecycle

```
Client
  |  HTTPS, Authorization: Bearer <access JWT>, X-Request-Id, Idempotency-Key
  v
[ALB]  TLS termination, connection limits
  v
[SecurityMiddleware]        HSTS, referrer policy, host validation
[RequestIdMiddleware]       generates/propagates X-Request-Id into log context
[RateLimitMiddleware]       Redis token bucket, keyed by principal or IP
[AuthenticationMiddleware]  JWT verify -> Principal(user, roles, provider_id...)
[LocaleMiddleware]          resolves locale + presentment currency
[AuditContextMiddleware]    binds actor to thread-local for audit writes
  v
[DRF Router] -> [ViewSet]
     - Permission classes: role check, then object ownership check
     - Serializer: syntactic + semantic validation
     - Idempotency check (POST): replay stored response if key seen
  v
[Application service]  @transaction.atomic
     - domain rules -> repositories -> events queued
  v
[Response serializer] -> envelope -> JSON
  v
[After commit] domain events dispatched to Celery
```

Domain events are dispatched **after** commit (`transaction.on_commit`) so that no notification, ledger accrual or webhook can describe a state that was rolled back.

8.4 Transaction and Concurrency Policy

Operation	Isolation	Locking	Rationale
Read catalogue	READ COMMITTED	none	Tolerates staleness
Place inventory hold	READ COMMITTED	`SELECT … FOR UPDATE` on the affected `room_availability` / `activity_departure` rows, ordered by primary key	Serialises the capacity check-and-decrement; PK ordering prevents deadlock
Confirm trip (basket)	READ COMMITTED	Row locks on all held inventory + optimistic `version` on `trip` and each `booking`	Atomic multi-booking confirmation
Payment webhook processing	READ COMMITTED	Advisory lock on `payment.id`; unique constraint on `psp_reference`	Exactly-once effect under duplicate webhooks
Driver assignment acceptance	READ COMMITTED	`SELECT … FOR UPDATE` on `driver_assignment` + `EXCLUDE` constraint	First acceptance wins; overlap impossible
Payout batch generation	READ COMMITTED	Advisory lock per `(provider_id, period)`	No duplicate batches

Deadlock avoidance rule: **always acquire row locks in ascending primary-key order**, and never hold a database transaction open across an external HTTP call.

8.5 Business Logic Placement

Logic	Location	Reason
Transfer price calculation	`apps/transport/domain/pricing.py`	Must be unit-testable and auditable

Logic	Location	Reason
	(pure)	
Accommodation nightly pricing	`apps/catalogue/domain/rates.py` (pure)	Same
Commission resolution	`apps/finance/domain/commission.py` (pure)	Money correctness
Cancellation and refund calculation	`apps/booking/domain/policies.py` (pure)	Disputes require reproducibility
Itinerary feasibility validation	`apps/trip/domain/validation.py` (pure)	Complex rule set, heavy test coverage
Booking/payment/trip state transitions	`apps/*/domain/state_machine.py` (pure)	One declaration per machine
Driver dispatch scoring	`apps/transport/domain/dispatch.py` (pure)	Must be explainable to providers and regulators
Availability mutation	`apps/inventory/services.py` (transactional)	Requires locks; not pure

8.6 Validation Strategy

Validation is applied in three tiers, and each tier has a distinct responsibility. Skipping a tier is a defect.

Tier	Where	Examples
Syntactic	DRF serializer	Types, formats, lengths, enum membership, required fields, ISO-8601 parsing
Semantic / business	Domain layer	Trip dates within booking horizon; party size ≤ room capacity; activity booked after its cutoff; transfer origin ≠ destination; policy allows the requested cancellation
Structural / storage	Database constraints	Uniqueness, referential integrity, capacity sums, date ordering, assignment overlap

8.7 Error Handling

A single exception hierarchy maps to a single response envelope. See Section 32 for the full catalogue and client behaviour.

```
PlatformError
  ├─ ValidationError            -> 422
  ├─ AuthenticationError        -> 401
  ├─ PermissionDeniedError      -> 403
  ├─ NotFoundError              -> 404
  ├─ ConflictError              -> 409   (state conflict, version conflict)
  ├─ InventoryUnavailableError  -> 409   (subtype, with alternatives payload)
  ├─ RateLimitedError           -> 429
  ├─ ExternalServiceError       -> 502 / 503
  └─ InternalError              -> 500
```

8.8 Background and Scheduled Jobs

Job	Trigger	Queue	Purpose	Failure handling
`dispatch_driver_offer`	Event: transfer booking confirmed	`realtime`	Send offer to ranked driver	Retry to next candidate on TTL expiry
`expire_driver_offer`	Countdown (offer TTL)	`realtime`	Mark offer expired, advance candidate list	—
`release_expired_holds`	Beat, every 60 s	`default`	Release `inventory_hold` past `expires_at`	Idempotent
`send_notification`	Event	`notify`	Deliver via push/email/SMS	Exponential backoff, 5 attempts, DLQ

Job	Trigger	Queue	Purpose	Failure handling
`verify_payment_status`	Countdown after initiate	`payments`	Poll PSP if webhook not received	6 attempts over 30 min, then mark for review
`accrue_commission`	Event: booking completed	`finance`	Write ledger entries	Idempotent by `(booking_id, entry_type)`
`build_payout_batches`	Beat, weekly	`finance`	Assemble provider payouts	Advisory-locked
`materialise_activity_departures`	Beat, nightly	`default`	Expand schedules into departures for the horizon	Idempotent upsert
`refresh_route_cache`	Beat, nightly	`default`	Recompute corridor matrix entries near expiry	—
`send_trip_reminders`	Beat, hourly	`notify`	T-72 h, T-24 h, T-3 h reminders	Idempotent by `(booking_id, reminder_code)`
`auto_complete_bookings`	Beat, hourly	`default`	Move past-end-time bookings to COMPLETED	—
`reconcile_payments`	Beat, daily	`finance`	Compare PSP settlement report against ledger	Alerts on mismatch
`purge_driver_locations`	Beat, daily	`default`	Drop partitions past retention	—
`rebuild_rating_aggregates`	Beat, hourly	`default`	Recompute `rating_aggregate`	—

Queue isolation matters: a burst of notifications must never delay payment verification. Queues `payments` and `finance` have dedicated workers and their own concurrency limits.

8.9 Domain Events

Events are the module decoupling mechanism. They are published in-process after commit and consumed by Celery tasks.

Event	Publisher	Consumers
`TripPriced`	trip	notify (quote expiry reminder)
`TripConfirmed`	booking	notify, trip (status sync), finance
`BookingConfirmed`	booking	notify, transport (dispatch if transfer), messaging (open conversation)
`BookingCancelled`	booking	inventory (release), payment (refund evaluation), notify, transport (unassign)
`BookingCompleted`	booking	finance (accrue), review (invite), notify
`PaymentCaptured`	payment	booking (confirm basket), notify
`PaymentFailed`	payment	booking (release holds), notify
`RefundSettled`	payment	finance (reverse accrual), notify
`DriverAssigned`	transport	notify (tourist + driver)
`DriverArrived` / `TripStarted` / `TripEnded`	transport	notify, booking (status)
`ProviderVerified`	provider	notify, catalogue (allow publish)
`ReviewPublished`	review	provider (aggregate refresh), notify

8.10 Caching Policy

Cached item	Key	TTL	Invalidation
Destination list	`cat:dest:{locale}`	1 h	On catalogue write
Activity/accommodation detail	`cat:{type}:{public_id}:{locale}`	15 min	On entity write
Route between two points	`route:{provider}:{o_geohash}:{d_geohash}:{mode}`	30 days	Nightly refresh job
Destination-pair travel matrix	`matrix:{o_dest}:{d_dest}`	7 days	On tariff/corridor change

Cached item	Key	TTL	Invalidation
FX rate	`fx:{base}:{quote}`	1 h	Scheduled refresh
Availability search result	`avail:{room_type}:{from}:{to}`	60 s	Deliberately short; never used for the authoritative check
Session principal	`auth:{jti}`	Token TTL	On logout/revocation

Rule: caches may serve *search and browse* results; the **authoritative availability check always reads the database inside the holding transaction**. A cached availability figure may never confirm a booking.

8.11 Observability

Structured JSON logs carrying `request_id`, `user_id`, `trip_id`, `booking_id`, `payment_id` where in scope; secrets and PII redacted by a logging filter. Metrics exposed in Prometheus format: request rate/latency/error by endpoint, Celery queue depth and task duration, database connection saturation, external port latency and error rate, and business gauges (trips confirmed, payment success rate, offer acceptance rate, hold expiry rate). Distributed tracing via OpenTelemetry across HTTP → task → external call. Alert rules are listed in Section 29.14.

9 API Design and Specification

9.1 Conventions

Aspect	Rule
Base URL	`https://api.<domain>/api/v1`
Versioning	URI major version; additive changes only within a version; deprecation announced with a 180-day window and `Sunset` header
Format	JSON; UTF-8; `Content-Type: application/json`
Identifiers	UUID `public_id` only; internal integers are never exposed
Dates/times	ISO 8601 with offset (`2027-08-10T14:30:00+03:00`); durations in seconds; dates as `YYYY-MM-DD`
Money	`{"amount": "465.00", "currency": "USD"}` — decimal **string**, never a float
Authentication	`Authorization: Bearer <access token>`
Idempotency	`Idempotency-Key` header required on all POST that create bookings, payments or assignments; server stores key → response for 24 h
Pagination	Cursor-based: `?limit=&cursor=`; response returns `next_cursor`
Filtering / sorting	Explicit allow-listed query parameters only; `sort` accepts a fixed enum
Correlation	`X-Request-Id` echoed in every response
Rate limiting	`X-RateLimit-Limit`, `X-RateLimit-Remaining`, `Retry-After` on 429
Localisation	`Accept-Language` selects content locale; `X-Currency` selects presentment currency

9.2 Response Envelope

Success:

```
{
  "data": { "...resource or array..." },
  "meta": { "request_id": "0f0a...", "next_cursor": "eyJpZCI6NDJ9" }
}
```

Error:

```
{
  "error": {
    "code": "INVENTORY_UNAVAILABLE",
    "message": "The selected room is no longer available for 12-14 Aug 2027.",
    "details": [
      { "field": "items[1].room_type_id", "issue": "SOLD_OUT" }
    ],
    "request_id": "0f0a...",
    "retryable": false
  }
}
```

`message` is safe to display to an end user, localised by `Accept-Language`. `code` is stable and machine-readable. `details[].field` uses JSON-pointer-like paths into the request body.

9.3 Endpoint Catalogue

Legend for Auth column: – public, `T` tourist, `D` driver, `P` provider, `A` admin.

9.3.1 API-01 Authentication and Account

Method	Path	Auth	Purpose
POST	`/auth/register`	—	Create tourist account
POST	`/auth/login`	—	Obtain access + refresh tokens
POST	`/auth/refresh`	—	Rotate refresh token
POST	`/auth/logout`	T/D/P/A	Revoke refresh token
POST	`/auth/verify-email`	—	Consume email verification token
POST	`/auth/password/forgot`	—	Request reset link
POST	`/auth/password/reset`	—	Consume reset token
POST	`/auth/mfa/enrol` | `/auth/mfa/verify`	P/A	TOTP enrolment
GET/PATCH	`/me`	T/D/P/A	Read/update own profile
POST	`/me/devices`	T/D	Register push token
DELETE	`/me`	T	Request account closure

9.3.2 API-02 Catalogue

Method	Path	Auth	Purpose
GET	`/destinations`	—	List active destinations (`?region=&is_gateway=`)
GET	`/destinations/{id}`	—	Destination detail with attraction/activity counts
GET	`/attractions`	—	`?destination=&tags=&limit=&cursor=`
GET	`/attractions/{id}`	—	Detail incl. opening hours, fee, media
GET	`/activities`	—	`?destination=&date=&pax=&tags=&max_price=&sort=`
GET	`/activities/{id}`	—	Detail incl. inclusions, requirements, policy
GET	`/activities/{id}/departures`	—	`?from=&to=&pax=` → bookable departures with remaining capacity
GET	`/accommodations`	—	`?destination=&check_in=&check_out=&adults=&children=&rooms=&amenities=&sort=`
GET	`/accommodations/{id}`	—	Property detail
GET	`/accommodations/{id}/room-types`	—	`?check_in=&check_out=&adults=&children=` → available room types with nightly and total price
GET	`/search`	—	Unified full-text search across catalogue

9.3.3 API-03 Trip and Itinerary

Method	Path	Auth	Purpose
POST	`/trips`	T	Create draft trip
GET	`/trips`	T	List own trips (`?status=`)

Method	Path	Auth	Purpose
GET	`/trips/{id}`	T	Full trip with itinerary and cost breakdown
PATCH	`/trips/{id}`	T	Update dates, party, title (DRAFT/PRICED only)
DELETE	`/trips/{id}`	T	Discard a draft
PUT	`/trips/{id}/flights`	T	Upsert inbound/outbound flight details
POST	`/trips/{id}/items`	T	Add an itinerary item
PATCH	`/trips/{id}/items/{item_id}`	T	Modify an unlocked item
DELETE	`/trips/{id}/items/{item_id}`	T	Remove an unlocked item
POST	`/trips/{id}/itinerary/generate`	T	Sequence, validate and cost the itinerary
GET	`/trips/{id}/itinerary`	T	Current itinerary with per-day structure
GET	`/trips/{id}/validation`	T	Validation findings without regenerating
POST	`/trips/{id}/quote`	T	Lock prices, place inventory holds, return quote + expiry
GET	`/trips/{id}/summary`	T	Cost breakdown for the review screen
POST	`/trips/{id}/confirm`	T	Create the booking basket in PENDING (requires valid quote)
POST	`/trips/{id}/cancel`	T	Cancel entire trip subject to policy

9.3.4 API-04 Transport Quoting

Method	Path	Auth	Purpose
POST	`/transport/quotes`	T	Quote one or more transfer legs (distance, duration, price by vehicle class)
GET	`/transport/vehicle-classes`	—	Classes with capacity and indicative pricing
GET	`/transport/corridors`	—	Configured origin-destination corridors

9.3.5 API-05 Bookings

Method	Path	Auth	Purpose
GET	`/bookings`	T/P/D	List, scoped by role
GET	`/bookings/{id}`	T/P/D	Detail incl. status history and provider contact
POST	`/bookings/{id}/cancel`	T/P	Cancel with reason; returns computed refund
GET	`/bookings/{id}/cancellation-preview`	T	Refund that *would* apply, without cancelling
POST	`/bookings/{id}/voucher`	T	Generate/download PDF voucher
POST	`/bookings/{id}/accept` | `/decline`	P	Provider response where confirmation is manual

9.3.6 API-06 Driver

Method	Path	Auth	Purpose
POST	`/driver/availability`	D	Go online/offline
GET	`/driver/offers`	D	Pending offers
POST	`/driver/offers/{id}/accept` | `/decline`	D	Respond within TTL
GET	`/driver/assignments`	D	`?status=&date=`
GET	`/driver/assignments/{id}`	D	Pickup detail, flight info, tourist first name and masked phone
POST	`/driver/assignments/{id}/status`	D	`EN_ROUTE|ARRIVED|STARTED|COMPLETED`
POST	`/driver/location`	D	Batched location points
GET	`/driver/earnings`	D	`?period=` summary and line items

9.3.7 API-07 Payments

Method	Path	Auth	Purpose
POST	`/payments/intents`	T	Create payment intent for a trip; returns PSP client secret / redirect / USSD instruction
GET	`/payments/{id}`	T	Payment status
POST	`/payments/{id}/verify`	T	Client-initiated status refresh

Method	Path	Auth	Purpose
POST	`/webhooks/psp/{provider}`	— (signed)	PSP callback; signature-verified, idempotent
GET	`/payments/methods`	T	Methods available for the tourist's country and currency
POST	`/refunds`	A/F	Issue refund against a payment or booking

9.3.8 API-08 Messaging, Notifications, Reviews

Method	Path	Auth	Purpose
GET	`/conversations`	T/P/D	Booking-scoped threads
GET	`/conversations/{id}/messages`	T/P/D	Paged messages
POST	`/conversations/{id}/messages`	T/P/D	Send
POST	`/messages/{id}/report`	T/P/D	Report abuse
GET	`/notifications`	T/D/P	In-app inbox
POST	`/notifications/{id}/read`	T/D/P	Mark read
GET/PATCH	`/me/notification-preferences`	T/D/P	Channel opt-in per event class
POST	`/reviews`	T	Submit review for a completed booking
GET	`/reviews`	—	`?subject_type=&subject_id=` published reviews
POST	`/reviews/{id}/response`	P	Provider response
POST	`/reviews/{id}/flag`	T/P	Flag for moderation

9.3.9 API-09 Provider Portal

Method	Path	Auth	Purpose
POST	`/provider/register`	—	Begin onboarding
POST	`/provider/documents`	P	Upload verification documents
GET	`/provider/dashboard`	P	KPIs
CRUD	`/provider/accommodations[/{id}[/room-types[/{id}]]]`	P	Listing management
PUT	`/provider/room-types/{id}/availability`	P	Bulk availability and rate calendar upsert
CRUD	`/provider/activities[/{id}[/schedules[/{id}]]]`	P	Activity management
PUT	`/provider/activities/{id}/departures`	P	Bulk departure upsert
CRUD	`/provider/drivers`, `/provider/vehicles`	P	Fleet management
GET	`/provider/bookings`	P	`?status=&from=&to=`
GET	`/provider/earnings`, `/provider/payouts`	P	Statements

9.3.10 API-10 Administration

Method	Path	Auth	Purpose
CRUD	`/admin/countries`, `/regions`, `/destinations`, `/attractions`	A	Catalogue
GET/POST	`/admin/providers`, `/admin/providers/{id}/verify`	A	Verification workflow
GET/POST	`/admin/drivers/{id}/verify`, `/admin/vehicles/{id}/verify`	A	Verification workflow
GET	`/admin/trips`, `/admin/bookings`, `/admin/payments`	A	Operational search
POST	`/admin/bookings/{id}/force-transition`	A	Exceptional state change; always audited
CRUD	`/admin/commission-rules`, `/admin/tariffs`, `/admin/policies`	A	Commercial configuration
GET/POST	`/admin/payouts`, `/admin/payouts/{id}/approve`	A	Payout control
GET/POST	`/admin/reviews`, `/admin/reviews/{id}/moderate`	A	Moderation
GET	`/admin/reports/{report_code}`	A	Reporting (Section 40)
GET	`/admin/audit-logs`	A	Investigation
GET/PUT	`/admin/settings`	A	System configuration

9.4 Detailed Endpoint Specifications

The following endpoints carry the majority of the system's business risk and are therefore specified in full. All others follow the same conventions.

9.4.1 POST /auth/register

Auth: none. **Rate limit:** 5 per hour per IP.

Request:

```
{
  "email": "alice@example.com",
  "password": "••••••••••••",
  "first_name": "Alice",
  "last_name": "Muller",
  "nationality": "DE",
  "locale": "en",
  "preferred_currency": "USD",
  "marketing_opt_in": false
}
```

Validation: email RFC 5322 and unique among non-deleted users; password minimum 12 characters, checked against a breached-password list, not equal to email local-part; `nationality` ISO 3166-1 alpha-2; `preferred_currency` in the enabled set.

Responses: `201` with `{ "data": { "user": {...}, "verification_required": true } }`; `409 EMAIL_ALREADY_REGISTERED`; `422` on validation failure; `429` on rate limit.

Side effects: creates `user` (status `PENDING`) and `tourist_profile`; sends verification email; writes audit entry `user.register`.

9.4.2 POST /auth/login

Request `{ "email", "password", "mfa_code"? }`. Responses: `200` with `access_token` (15 min), `refresh_token` (30 days, rotating), `token_type`, `expires_in`, and `principal` (roles, provider id where applicable). `401 INVALID_CREDENTIALS` — deliberately identical for unknown email and wrong password. `403 EMAIL_NOT_VERIFIED`, `403 ACCOUNT_SUSPENDED`, `423 ACCOUNT_LOCKED` after the configured failure threshold, `401 MFA_REQUIRED` when the role mandates TOTP and no code was supplied.

9.4.3 GET /accommodations/{id}/room-types

Query: `check_in` (date, required), `check_out` (date, required, > check_in), `adults` (≥1), `children` (≥0), `rooms` (≥1).

Response `200`:

```
{
  "data": [
    {
      "id": "3f2a…",
      "name": "Deluxe Sea View",
      "max_adults": 2, "max_children": 1,
      "available_rooms": 3,
      "min_nights": 2,
      "nightly": [
        { "date": "2027-08-10", "price": { "amount": "93.00", "currency": "USD" } },
        { "date": "2027-08-11", "price": { "amount": "93.00", "currency": "USD" } }
      ],
      "total": { "amount": "465.00", "currency": "USD" },
      "cancellation_policy": {
        "code": "FLEX_48H",
        "summary": "Free cancellation up to 48 hours before check-in."
      }
    }
  ]
}
```

Business rules applied: a room type appears only if every night in the range has `rooms_open − rooms_held − rooms_sold ≥ rooms` and `is_closed = false`; `min_nights` is satisfied; and requested occupancy fits capacity. Availability shown here is **indicative** — the authoritative check occurs at `POST /trips/{id}/quote`.

Errors: `422 INVALID_DATE_RANGE` (check-out ≤ check-in, or range exceeds the 30-night maximum stay), `404 NOT_FOUND`.

9.4.4 POST /transport/quotes

Request:

```
{
  "trip_id": "9c1e…",
  "legs": [
    {
      "reference": "leg-1",
      "origin": { "destination_id": "znz-…" },
      "target": { "accommodation_id": "acc-…" },
      "depart_at": "2027-08-10T14:45:00+03:00",
      "pax": 2,
      "luggage": 2
    }
  ],
  "vehicle_class": "STANDARD"
}
```

Processing: resolve each endpoint to coordinates → look up `route_cache`, else call the routing port → apply the tariff resolution of Section 12.4 → return per-class options.

Response `200`:

```
{
  "data": [
    {
      "reference": "leg-1",
      "distance_m": 57400,
      "travel_seconds": 5100,
      "polyline": "…",
      "options": [
        { "vehicle_class": "STANDARD", "seats": 4, "luggage": 3,
          "price": { "amount": "38.00", "currency": "USD" },
          "breakdown": { "base": "12.00", "distance": "22.96",
                         "time": "0.00", "surcharges": "3.04" } },
        { "vehicle_class": "VAN", "seats": 7, "luggage": 6,
          "price": { "amount": "52.00", "currency": "USD" } }
      ]
    }
  ]
}
```

Errors: `422 NO_TARIFF_CONFIGURED` when no corridor or fallback tariff matches; `502 ROUTING_UNAVAILABLE` when the provider fails and no cache entry exists (see the degradation rule in Section 12.6).

9.4.5 POST /trips/{id}/quote

The most consequential endpoint in the system. It converts a plan into a priced, inventory-backed, time-boxed offer.

Auth: tourist, owner of the trip. **Idempotency-Key:** required.

Processing sequence, entirely within one transaction:

1. Assert `trip.status ∈ {DRAFT, PRICED}` and the itinerary passes validation (Section 10.6). Otherwise `409 TRIP_NOT_QUOTABLE`.
2. Release any prior holds belonging to this trip.
3. For each accommodation item: lock the `room_availability` rows for every night (`FOR UPDATE`, PK order); assert capacity; increment `rooms_held`; create `inventory_hold`.
4. For each activity item: lock the `activity_departure`; assert `capacity_total − held − sold ≥ pax`; increment `capacity_held`; create `inventory_hold`.
5. For each transfer item: re-quote using current tariffs; transfers hold no inventory but reserve a vehicle *class*, not a specific driver.
6. Recompute all line totals, the platform fee and taxes; write to `trip`.
7. Set `trip.status = PRICED`, `priced_at = now()`, `quote_expires_at = now() + hold_ttl` (default 20 minutes, configurable).
8. Emit `TripPriced`.

Response `200` returns the full cost breakdown plus `quote_expires_at` and a `quote_token` that must be presented at confirmation.

Errors: `409 INVENTORY_UNAVAILABLE` with a `details` array naming each unavailable item and, where the catalogue offers them, alternative departures or room types; `409 PRICE_CHANGED` when a provider changed a rate since the last generate call, returning old and new totals for explicit re-acceptance; `422 ITINERARY_INVALID` with the validation findings.

9.4.6 POST /trips/{id}/confirm

Request `{ "quote_token": "…", "payment_method": "CARD" }`. **Idempotency-Key** required.

Processing: verify `quote_token` matches the current quote and `now() < quote_expires_at` (`409 QUOTE_EXPIRED` otherwise); create one `booking` per component with `status = PENDING`; snapshot cancellation policy and commission rate onto each booking; link `itinerary_item.booking_id`; set `trip.status = PENDING_PAYMENT`; extend hold expiry to the payment window (default 30 minutes). Response `201` returns the basket and the total payable. **No inventory is committed and no provider is notified at this point** — that happens only on payment capture.

9.4.7 POST /payments/intents

Request `{ "trip_id", "method", "currency", "return_url" }`. **Idempotency-Key** required.

Processing: assert `trip.status = PENDING_PAYMENT` and holds are live; compute the charge amount from `trip.total_amount` (never from client input); create `payment` in `INITIATED`; call the PSP adapter; persist `psp_reference`; schedule `verify_payment_status` as a webhook-failure fallback.

Response `201` returns a method-specific `action` object: for cards, `{"type":"CLIENT_SECRET","client_secret":"…"}` or `{"type":"REDIRECT","url":"…"}`; for mobile money, `{"type":"USSD_PUSH","instruction":"Approve the prompt on +255…"}`.

Errors: `409 TRIP_NOT_PAYABLE`, `409 QUOTE_EXPIRED`, `422 UNSUPPORTED_METHOD_FOR_CURRENCY`, `502 PSP_UNAVAILABLE`.

9.4.8 POST /webhooks/psp/{provider}

Unauthenticated by session; authenticated by signature. Processing: verify HMAC signature and timestamp freshness (reject > 5 minutes skew); persist the raw event to `payment_webhook_event` keyed by the PSP event id (duplicate id → `200` immediately, no reprocessing); acquire an advisory lock on the payment; apply the state transition; on `CAPTURED`, run the confirmation routine of Section 20.8 (commit holds, confirm bookings, emit events). Always respond `200` once the event is durably stored, even if downstream processing is deferred, so the PSP does not retry unnecessarily; genuine processing failures are retried internally.

9.4.9 POST /driver/assignments/{id}/status

Request `{ "status": "ARRIVED", "at": "2027-08-10T14:41:22+03:00", "location": {"lat":-6.22,"lng":39.22} }`.

Validation: the transition must be legal for the current status (Section 11.7); `at` must be within ±10 minutes of server time; the driver must be the assignee. `ARRIVED` and later transitions require a location within the configured geofence radius of the expected point (default 300 m) or an explicit `override_reason`, which is flagged for review.

Responses: `200` with the new state; `409 ILLEGAL_TRANSITION`; `403 NOT_ASSIGNEE`; `422 GEOFENCE_VIOLATION`.

9.5 WebSocket Channels

Channel	Subscriber	Messages
`trip.{trip_public_id}`	Tourist (owner)	`booking.status_changed`, `driver.assigned`, `driver.location`, `driver.arrived`
`driver.{driver_public_id}`	Driver	`offer.new`, `offer.expired`, `assignment.cancelled`
`provider.{provider_public_id}`	Provider portal	`booking.new`, `booking.cancelled`

Connections authenticate with the same access token in the connect payload; subscription to a channel re-runs the ownership check. Location messages are emitted only while an assignment is in `EN_ROUTE`, `ARRIVED` or `STARTED`.

9.6 Rate Limits

Class	Limit
Unauthenticated catalogue reads	60 / minute / IP
`POST /auth/login`	10 / hour / IP and 5 / hour / email
`POST /auth/register`, password reset	5 / hour / IP
Authenticated reads	300 / minute / principal
`POST /trips/{id}/quote`	20 / hour / trip
`POST /payments/intents`	10 / hour / tourist
`POST /driver/location`	120 / minute / driver (batched)
Messaging sends	60 / hour / conversation

10 Trip Planning Engine

10.1 Responsibilities

The trip planning engine converts a set of tourist selections into a validated, sequenced, priced itinerary. It is deterministic: the same inputs, the same catalogue state and the same tariff configuration always produce the same itinerary and the same total. It performs no optimisation search and no learning; it applies ordering rules, inserts required transfers, validates constraints, and costs the result.

10.2 Planning Workflow

```
[Tourist]                        [Backend]
    |                               |
Create trip  ------------------->  POST /trips
    |                              trip.status = DRAFT
    |                              itinerary v1 created (empty)
    |                               |
Enter dates, party  ------------>  PATCH /trips/{id}
    |                              day skeleton rebuilt (day 1..N)
    |                               |
Enter flights  ------------------>  PUT /trips/{id}/flights
    |                              gateway destination bound
    |                              arrival/departure anchors created
    |                               |
Choose accommodation ------------>  POST /trips/{id}/items (STAY)
    |                              nightly items generated per night
    |                               |
Choose activities  -------------->  POST /trips/{id}/items (ACTIVITY)
    |                              departure bound; meeting point set
    |                               |
Choose attractions -------------->  POST /trips/{id}/items (ATTRACTION)
    |                               |
Generate itinerary -------------->  POST /trips/{id}/itinerary/generate
    |                              1. anchor arrival/departure
    |                              2. sort items per day
    |                              3. insert required TRANSFER items
    |                              4. call routing for each transfer
    |                              5. run validation rule set
    |                              6. compute line totals + fees
    |                               |
    |<---------------------------  itinerary + findings + totals
    |                               |
Review, adjust, repeat              (loop)
    |                               |
Request quote  ------------------>  POST /trips/{id}/quote
    |                              holds placed, prices locked, TTL set
    |                               |
Confirm  ---------------------->    POST /trips/{id}/confirm
                                    basket created, PENDING_PAYMENT
```

10.3 Structural Model

A trip owns exactly one current `itinerary`, which owns ordered `itinerary_item` rows partitioned by `day_number`. Regeneration increments `itinerary.version` and rewrites unlocked items; items whose `booking_id` refers to a confirmed booking are locked (`is_locked = true`) and are never rewritten. This allows a tourist to add day 4 to an already-confirmed trip without disturbing days 1 to 3.

```
trip (10-15 Aug, 2 adults)
 └── itinerary v3
      ├── day 1 (10 Aug)
      │    ├── seq 1  TRANSFER   ZNZ -> Ocean Breeze     14:45-16:10  [locked]
      │    └── seq 2  STAY       Ocean Breeze, Deluxe    16:10-…      [locked]
      ├── day 2 (11 Aug)
      │    ├── seq 1  TRANSFER   Hotel -> Stone Town     08:30-09:55
      │    ├── seq 2  ACTIVITY   Heritage Walking Tour   10:00-13:00
      │    └── seq 3  TRANSFER   Stone Town -> Hotel     14:00-15:25
      …
      └── day 6 (15 Aug)
           └── seq 1  TRANSFER   Hotel -> ZNZ            09:00-10:25
```

10.4 Day Sequencing Algorithm

Deterministic, non-optimising, and specified here so that two implementations produce identical output.

```
INPUT : trip, unlocked items, locked items, flights
OUTPUT: ordered items with computed times, or validation findings

 1  days := [trip.start_date .. trip.end_date]
 2  FOR each day IN days:
 3      fixed := items on day with a provider-imposed time
 4               (ACTIVITY with departure, flight anchors, check-in/out)
 5      flexible := remaining items on that day (ATTRACTION, FREE_TIME)
 6      order := sort(fixed) BY starts_at ASC, THEN item_type rank,
 7                              THEN item.id ASC          # total order
 8      FOR each adjacent pair (A, B) IN order:
 9          origin := end_location(A)      # accommodation, meeting point…
10          target := start_location(B)
11          IF origin != target:
12              t := travel_time(origin, target)          # Section 12.3
13              insert TRANSFER item with
14                  ends_at   = B.starts_at - buffer_before(B)
15                  starts_at = ends_at - t
16      FOR each item IN flexible:
17          place into the largest remaining gap that can contain
18          (travel_in + visit_minutes + travel_out); if none, emit
19          finding NO_SLOT_FOR_ATTRACTION and leave it unscheduled
20      renumber sequence_no from 1 in ascending starts_at order
21  RETURN items, findings
```

`buffer_before` is configuration, not code: `system_setting` holds `buffer.activity_minutes` (default 15), `buffer.airport_departure_minutes` (default 180 for international departure), `buffer.check_in_minutes` (default 0).

Tie-break rank for identical `starts_at`: `TRANSFER (0) < STAY check-out (1) < ACTIVITY (2) < ATTRACTION (3) < STAY check-in (4) < FREE_TIME (5)`. This guarantees a total order and satisfies principle A7.

10.5 Travel-Time Sourcing

Travel times come from the routing port (Section 12.3), never from straight-line estimates, except in the explicit degraded mode of Section 12.6. Results are cached in `route_cache` keyed by geohash of both endpoints at precision 7 (≈150 m), the travel mode, and the provider. A **destination-pair matrix** is precomputed nightly for all active destination pairs so that the planner can price and sequence a typical itinerary with zero external calls.

10.6 Validation Rules

Validation runs on every `generate` and again inside `quote`. Findings have severity `ERROR` (blocks quoting) or `WARNING` (advisory).

ID	Severity	Rule
VR-01	ERROR	Every item's `starts_at`/`ends_at` fall within the trip date range, extended by the arrival and departure flight times

ID	Severity	Rule
VR-02	ERROR	No two non-`STAY` items on the same day overlap in time
VR-03	ERROR	Consecutive items are geographically reachable: gap between `A.ends_at` and `B.starts_at` ≥ travel time + buffer
VR-04	ERROR	Every accommodation night in the trip range is covered by exactly one `STAY` item, or explicitly marked as "own arrangement"
VR-05	ERROR	Party size ≤ room occupancy for every stay; party size within `[min_pax, max_pax]` for every activity
VR-06	ERROR	Activity booked at or before its `booking_cutoff_hours` relative to departure
VR-07	ERROR	Airport arrival transfer starts no earlier than the flight arrival plus `buffer.arrival_processing_minutes` (default 45)
VR-08	ERROR	Airport departure transfer arrives at the gateway at least `buffer.airport_departure_minutes` before scheduled departure
VR-09	ERROR	Every item's provider and listing are active and not suspended
VR-10	ERROR	Currency is uniform across all items in the trip
VR-11	ERROR	Minimum-nights requirement of each selected room type is satisfied
VR-12	WARNING	An attraction is scheduled outside its opening hours for that weekday
VR-13	WARNING	A day contains more than `limit.items_per_day` (default 5) timed items
VR-14	WARNING	Total travel time in a day exceeds `limit.travel_minutes_per_day` (default 240)
VR-15	WARNING	An activity has an age requirement and the party includes children below it
VR-16	WARNING	The trip has no accommodation for one or more nights
VR-17	WARNING	Same-day arrival flight and an activity starting within 3 hours of landing

Each finding carries `{code, severity, message, item_ids[], suggested_action}` so the client can render an inline fix affordance.

10.7 Cost Computation

```
FOR each item:
    line_total := unit_price × quantity            (STAY: nightly rate × nights × rooms)
                                                   (ACTIVITY: per-person × pax, or group price)
                                                   (TRANSFER: tariff result, Section 12.4)

subtotal := Σ line_total
fee      := round(subtotal × platform_fee_rate, 2)        # system_setting
tax      := Σ applicable tax rules (Section 18.3)
total    := subtotal + fee + tax
```

Money arithmetic uses `Decimal` with `ROUND_HALF_UP` at two decimal places, applied once per line and once per aggregate. Floating point is prohibited anywhere in the pricing path.

10.8 Regeneration and Versioning

Every `generate` writes a new `itinerary.version`. The previous version's items are retained for 30 days in `itinerary_item_archive` to support "undo" and dispute investigation. A regeneration that would alter a locked item is rejected with `409 LOCKED_ITEM_CONFLICT`, naming the item.

10.9 Edge Cases

Case	Handling
Tourist changes trip dates after some bookings are confirmed	Rejected with `409`; the tourist must cancel the affected bookings first (policy applies)
Flight arrival time updated by the tourist	Arrival transfer is re-timed automatically if unassigned; if a driver is assigned, the driver is notified and may re-accept, else re-dispatched
Activity departure cancelled by provider	Item is unscheduled, tourist notified with alternative departures, trip returns to `DRAFT` if not yet paid, or refund path if paid
Accommodation split across two properties	Supported: multiple `STAY` items with contiguous, non-overlapping date ranges; VR-04 checks contiguity
Day trip with no accommodation	Supported; VR-16 warns rather than blocks

Case	Handling
Trip entirely without transfers (tourist self-drives)	Supported; transfers are optional items, and VR-03 is skipped between items marked `own_transport = true`

11 Airport Transfer and Driver Assignment

11.1 Purpose

Airport transfer is the single highest-anxiety moment of the journey and therefore receives the strongest guarantees in the system: a named driver, a known vehicle, a known time, and a documented fallback if anything fails.

11.2 Flight Data Capture

`trip_flight` stores, per direction: flight number, airline IATA code, scheduled arrival/departure instant (with offset), gateway destination, terminal, passenger count and luggage count. The Platform does **not** integrate a flight-status feed in V1; the tourist and the driver may each update actual arrival time, and the system re-times the transfer accordingly. Section 44 lists flight-status integration as a Version 2 candidate with a defined integration point (`FlightStatusPort`) already declared.

11.3 Pickup Scheduling

```
pickup_at = flight.scheduled_arrival
          + buffer.arrival_processing_minutes          (default 45)
```

The buffer accounts for disembarkation, immigration and baggage reclaim, and is a configurable per-gateway setting (`system_setting` key `buffer.arrival_processing.{gateway_code}`), because a small island terminal and a large hub differ materially. Pickup location is a named meeting point held on the destination record (`gateway_meeting_point` text plus coordinates), for example “Arrivals hall, outside the main exit, driver holding a name board”.

11.4 Information Guaranteed to the Tourist Before Departure

The following must be visible in the tourist app and in the confirmation email no later than `assignment_disclosure_hours` (default 24) before the scheduled arrival:

Field	Source
Driver first name and photograph	`driver` + `user`
Driver rating and completed trip count	`driver`
Driver spoken languages	`driver.languages`
Vehicle make, model, colour, plate	`vehicle`
Seat and luggage capacity	`vehicle`
Pickup meeting point text, coordinates and photograph	`destination`
Scheduled pickup time in destination-local time	computed
Masked in-app contact channel	`conversation`
Fallback support number	`system_setting`
Destination address of the drop-off	`accommodation`

If no driver has accepted by the disclosure deadline, the escalation of Section 11.8 applies.

11.5 Dispatch Lifecycle

```
BookingConfirmed(TRANSFER)
        |
        v
 build candidate list  (Section 11.6)
        |
        v
 +----> offer to candidate[i]          driver_offer created
 |          |  push + WebSocket, TTL = offer.ttl_seconds (default 90 for
 |          |  imminent trips, 3600 for scheduled trips > 24 h away)
 |          |
 |      accepted? --yes--> driver_assignment ASSIGNED
 |          |              notify tourist + driver, disclose details
 |          no / expired
 |          |
 +----- i := i + 1  while i < len(candidates) and deadline not reached
            |
            v
   no acceptance -> escalate to operations queue (Section 11.8)
```

For scheduled transfers the dispatcher begins offering at `pickup_at − dispatch.lead_hours` (default 72) so that assignment happens well before the disclosure deadline. Airport transfers booked less than 72 hours ahead are dispatched immediately.

11.6 Driver Candidate Scoring

Deterministic, transparent, administrator-tunable. No machine learning.

Eligibility filter (hard constraints — a driver failing any is excluded):

1. `driver.verify_status = VERIFIED` and `user.status = ACTIVE`.
2. `driver.licence_expires_on > pickup_at`; vehicle `insurance_expires_on` and `inspection_expires_on > pickup_at`.
3. Vehicle `seat_capacity ≥ pax` and `luggage_capacity ≥ luggage`.
4. Vehicle `vehicle_class` matches the booked class.
5. No overlapping `driver_assignment` in `[pickup_at − pre_buffer, expected_end + post_buffer]` (enforced by the `EXCLUDE` constraint as well).
6. Pickup point inside `driver.service_area` when a service area is defined.
7. `driver.is_online = true` **or** the transfer is scheduled more than 12 hours ahead (scheduled work is offered to offline drivers too).

Score (higher is better):

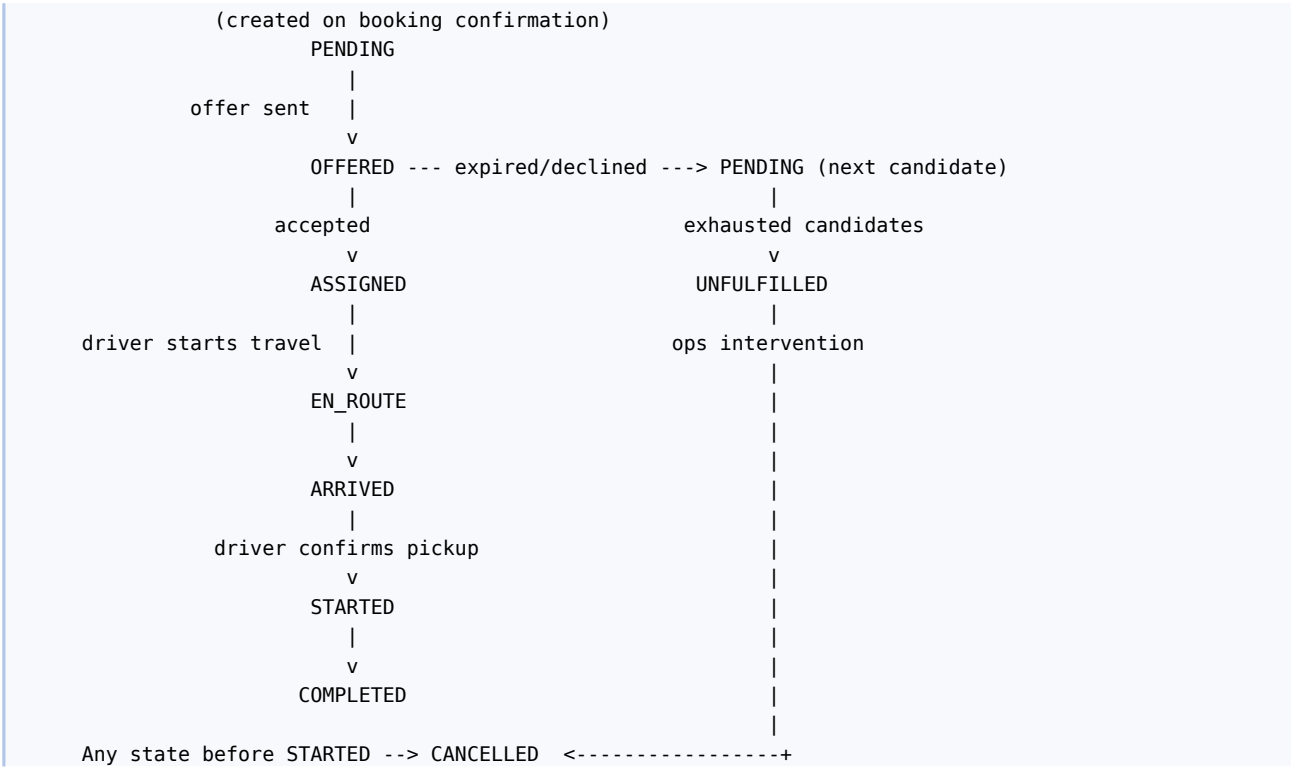
```
score =  w_prox * proximity_score
       + w_rate * (rating_avg / 5)
       + w_acc  * (acceptance_rate / 100)
       + w_exp  * min(completed_trips / 200, 1)
       + w_util * (1 - utilisation_today)

proximity_score = 1 - min(distance(driver_anchor, pickup) / max_radius_m, 1)
driver_anchor   = last_known_location if online and fresh (< 15 min),
                  else driver.home_destination.centroid

Default weights (system_setting, sum to 1.0):
  w_prox 0.40   w_rate 0.25   w_acc 0.20   w_exp 0.10   w_util 0.05
Tie-break: lower completed_trips first (favours supply development),
           then driver.id ascending (total order)
```

The weights, `max_radius_m`, and the tie-break rule are configuration. Every dispatch decision writes an audit entry containing the candidate list and each candidate's component scores, so a provider dispute can be answered with the exact computation.

11.7 Assignment State Machine



Transition	Trigger	Guard
PENDING → OFFERED	dispatcher	eligible candidate exists
OFFERED → ASSIGNED	driver accepts	offer not expired; no overlap
OFFERED → PENDING	decline or TTL expiry	candidates remain
PENDING → UNFULFILLED	candidate list exhausted or deadline passed	—
ASSIGNED → EN_ROUTE	driver action	within `pickup_at ± 4 h`
EN_ROUTE → ARRIVED	driver action	inside geofence (300 m) or override
ARRIVED → STARTED	driver action	tourist present; optional PIN confirmation
STARTED → COMPLETED	driver action	inside drop-off geofence or override
* (before STARTED) → CANCELLED	tourist, provider or admin	policy evaluated

11.8 Exception Handling

Exception	Detection	Response
No driver accepted by disclosure deadline	Scheduled check at `pickup_at − 24 h`	Ticket raised in operations queue at severity HIGH; administrator may assign manually or contact a partner transport provider; tourist notified only if manual assignment is not completed within 6 hours
Flight delayed	Tourist or driver updates actual arrival	Pickup re-timed; driver notified; if the new time conflicts with the driver's next assignment, the assignment is released and re-dispatched
Driver cancels < 4 h before pickup	Driver action	Immediate re-dispatch with an elevated candidate radius; driver's `acceptance_rate` penalised; repeated occurrences trigger compliance review
Tourist no-show	Driver marks `NO_SHOW` after a 60-minute wait at the meeting point, with geofence evidence	Booking → `NO_SHOW`; cancellation policy applies (typically non-refundable); dispute path available
Driver no-show	Tourist reports, or no `ARRIVED` within 30 minutes of pickup time	Ticket at severity CRITICAL; replacement dispatched; full refund of the transfer plus a goodwill credit per policy
Vehicle breakdown	Driver sets `INCIDENT` with a reason	Replacement dispatched; original assignment closed as

Exception	Detection	Response
mid-trip		`INCIDENT`; pro-rata settlement

11.9 Meeting Protocol

To eliminate the "which car is mine" failure, the app shows the vehicle photograph, colour and plate; the driver holds a name board with the tourist's surname; and the tourist app displays a four-digit **pickup PIN** which the driver enters (or the driver's code is shown to the tourist, configurable per market) to move `ARRIVED → STARTED`. The PIN is generated at assignment and stored hashed.

12 Transportation and Routing

12.1 Scope

Transport in V1 is **scheduled point-to-point transfer** with an assigned driver and vehicle. Supported leg patterns:

```
Gateway      -> Accommodation      (arrival transfer)
Accommodation-> Attraction/Activity meeting point
Attraction   -> Accommodation
Accommodation-> Gateway            (departure transfer)
Destination  -> Destination        (inter-town relocation)
Accommodation-> Accommodation      (property change mid-trip)
Custom point -> Custom point       (coordinates supplied by the tourist)
```

Half-day and full-day vehicle hire (driver at disposal) is modelled as a transfer with `hire_hours > 0` and a duration-based tariff; it is a SHOULD-HAVE item in Section 38.

12.2 Leg Composition Rules

A transfer leg is defined by an origin, a target, a departure instant, a party size, a luggage count and a vehicle class. Origin and target each resolve to a coordinate through one of four bindings — a destination centroid, an accommodation, an activity meeting point, or an explicit coordinate supplied by the tourist — and the binding is stored on the itinerary item so that the leg can be re-priced identically later. A leg whose origin and target resolve to the same coordinate is rejected at validation. Every leg is priced, assigned and settled independently of every other leg.

12.3 Routing Port

The routing provider is an external dependency behind an internal interface. The application never imports a vendor SDK outside `apps/location/adapters/`.

```python
class RoutingPort(Protocol):
    def route(self, origin: Coord, target: Coord, mode: TravelMode,
              depart_at: datetime | None) -> RouteResult: ...
    def matrix(self, origins: list[Coord], targets: list[Coord],
               mode: TravelMode) -> MatrixResult: ...
    def geocode(self, query: str, bias: Coord | None) -> list[PlaceCandidate]: ...
    def reverse_geocode(self, point: Coord) -> PlaceCandidate | None: ...

# RouteResult: distance_m, duration_seconds, polyline, provider, fetched_at
```

Recommended providers, in preference order, with the decision left to a commercial evaluation:

Provider	Advantages	Disadvantages
Google Maps Platform (Routes, Distance Matrix, Geocoding)	Best road coverage and address quality in Tanzania; reliable ETAs	Highest cost; restrictive caching and display terms that must be reviewed before relying on `route_cache`
Mapbox (Directions, Matrix, Search)	Strong pricing, permissive caching, good mobile SDKs	Slightly weaker minor-road coverage in Zanzibar
OpenRouteService / OSRM on	Self-hostable, no per-call cost,	Operational burden; ETA quality depends on

Provider	Advantages	Disadvantages
OpenStreetMap	unrestricted caching	local OSM data density and needs correction factors

The design assumption is that the provider is replaceable. A pragmatic V1 configuration is Mapbox or a self-hosted OSRM for the destination-pair matrix, with Google Geocoding for address resolution — but no code depends on that choice.

Provider terms on caching vary and must be honoured. `route_cache.expires_at` is set from a per-provider configured maximum cache age.

12.4 Transfer Tariff Model

Transfer prices are **not** provider-quoted per booking; they come from an administrator-managed tariff table so that the tourist sees a consistent price and the platform controls margin.

`transfer_corridor` (optional, for fixed-price routes):

Field	Meaning
origin_destination_id, target_destination_id	Corridor endpoints
vehicle_class	Class the row applies to
fixed_price, currency	Fixed corridor price, e.g. ZNZ → Nungwi, STANDARD
is_bidirectional	Applies in reverse as well
valid_from / valid_to	Effective dating

`transfer_tariff` (metered fallback):

Field	Meaning
region_id, vehicle_class	Applicability
base_fare	Flag-fall
per_km_rate, per_minute_rate	Metered components
minimum_fare	Floor
night_surcharge_pct, night_from, night_to	Time-of-day surcharge
airport_surcharge	Gateway pickup or drop-off supplement
waiting_rate_per_minute	Applied after free waiting allowance
currency, valid_from, valid_to	

Resolution order — first match wins, and the matched rule id is stored on `booking_transfer.tariff_id` so the price is reproducible forever:

```
1. Active transfer_corridor for (origin_dest, target_dest, class)
2. Active transfer_corridor for the reverse pair when is_bidirectional
3. Active transfer_tariff for (region of origin, class)
4. Active transfer_tariff for (country default, class)
5. No match -> 422 NO_TARIFF_CONFIGURED (never guess a price)
```

Metered computation:

```
price = base_fare
      + per_km_rate     × (distance_m / 1000)
      + per_minute_rate × (travel_seconds / 60)
price = max(price, minimum_fare)
IF pickup_at local time within [night_from, night_to]:
      price = price × (1 + night_surcharge_pct/100)
IF origin or target is a gateway destination:
      price = price + airport_surcharge
price = round(price, 2)
```

Worked example (ZNZ → Nungwi, STANDARD, daytime, no corridor row): base 12.00 + 0.40 × 57.4 (= 22.96) + 0.00 × 85 = 34.96; night surcharge none; airport surcharge 3.04 → **USD 38.00**.

12.5 Vehicle Classes

Class	Seats	Large bags	Typical use
STANDARD	4	3	Couples, short transfers
COMFORT	4	3	Higher-specification vehicle, air-conditioned
VAN	7	6	Families, long airport transfers
MINIBUS	14	12	Groups and multi-family parties

Class capacity is enforced at quoting time: the quote returns only classes whose `seats ≥ pax` and `luggage ≥ luggage_count`.

12.6 Degradation When Routing Is Unavailable

Context	Behaviour
Planning (`generate`)	Serve from `route_cache` if present; else from the nightly destination-pair matrix; else compute a haversine distance × road-factor (configurable, default 1.35) and a speed model (configurable, default 45 km/h), and mark the item `estimate_quality = APPROXIMATE`, which the UI renders with an explicit "approximate" label
Quoting (`quote`)	Cache and matrix permitted. Haversine fallback is **not** permitted for a priced corridor without a fixed corridor price; the endpoint returns `502 ROUTING_UNAVAILABLE` rather than commit the platform to a guessed price
Live driver navigation	The driver app uses the device's installed navigation application; the Platform does not provide turn-by-turn guidance in V1

12.7 Multi-Stop and Return Legs

A return trip is two independent `TRANSFER` items, each independently quoted, assigned and priced — not a single round trip. This keeps assignment, cancellation and settlement simple. Waiting time (driver waits at an attraction) is expressed instead through vehicle hire (`hire_hours`), which is the correct commercial model for that behaviour.

13 Location, GPS and Tracking

13.1 Coordinate Model

All coordinates are WGS 84 (`SRID 4326`) stored as PostGIS `geography(Point)`. Latitude/longitude are exchanged over the API as decimal degrees with a maximum of seven decimal places. Distance computations use `ST_Distance` on the geography type (metres, geodesic), never planar approximations.

13.2 Geocoding and Reverse Geocoding

Forward geocoding is used only in administrative tooling (when an administrator or provider creates a listing and enters an address) and in the tourist app's custom-pickup picker. Results are always confirmed by a human against a map pin before being persisted; the Platform never silently stores an unconfirmed geocode. Reverse geocoding is used to label a driver's or tourist's ad-hoc pickup point. Both are rate-limited and cached for 30 days keyed by normalised query text.

13.3 Distance and Route Calculation

Handled by the routing port (Section 12.3). Two distinct products are consumed: point-to-point `route` (for a specific leg, returning a polyline for map display) and `matrix` (for the nightly destination-pair precomputation). The system stores `distance_m` and `travel_seconds` on the itinerary item and on `booking_transfer` at confirmation time, so that later provider re-pricing or map changes cannot alter a settled booking.

13.4 Driver Location Ingest

```
Driver app                        Backend
  |  collect fix every 10 s while assignment is EN_ROUTE/ARRIVED/STARTED
  |  buffer locally
  |  POST /driver/location  (batch of up to 12 points, every ~60 s,
  |                          or every 15 s while EN_ROUTE)
  +------------------------> validate (monotonic timestamps, plausible
                              speed < 200 km/h, accuracy < 100 m)
                              write to driver_location (partitioned daily)
                              write latest to Redis key drv:{id}:last
                              publish to WebSocket channel trip.{trip_id}
```

Collection stops the moment the assignment reaches `COMPLETED`, `CANCELLED` or `INCIDENT`. The driver app must display a persistent foreground notification while location collection is active, and the driver must be able to see and understand when it is off.

13.5 Tracking Presented to the Tourist

The tourist sees the driver's position on a map **only** while their own assignment is in `EN_ROUTE`, `ARRIVED` or `STARTED`, and only for their own booking. Position updates arrive over the `trip.{trip_public_id}` WebSocket channel, with a polling fallback (`GET /bookings/{id}/driver-position`, 20-second interval) for clients unable to hold a socket. Updates are throttled server-side to one every 10 seconds and the driver's exact position is snapped to the road network for display.

13.6 Geofencing

Geofences validate driver status transitions (Section 11.7) and trigger proximity notifications.

Geofence	Radius (configurable)	Effect
Pickup point	300 m	Permits `ARRIVED`; triggers "your driver has arrived" notification
Approaching pickup	1,500 m	Triggers "your driver is 5 minutes away" notification
Drop-off point	300 m	Permits `COMPLETED`

A transition attempted outside the geofence requires an `override_reason`, is permitted, and is flagged in `audit_log` for review — because rejecting a legitimate transition would strand a tourist, which is worse than reviewing an anomaly later.

13.7 Location Privacy

Control	Rule
Purpose limitation	Driver location is collected only during an active assignment and only for dispatch, tracking and dispute resolution
Tourist location	Never continuously tracked. Captured only when the tourist explicitly sets a custom pickup point
Visibility	A tourist sees only the driver assigned to their own active booking; a driver never sees a tourist's live position
Retention	Raw points 30 days; thereafter only the aggregated route polyline attached to the completed booking
Consent	Explicit OS-level permission plus an in-app explanation screen before the first collection; the driver may revoke, which forces `is_online = false`
Access	Live location is readable by the assigned tourist, the driver's provider organisation, and administrators with the `SUPPORT_AGENT` role or above; every administrative access is audited

14 Accommodation Subsystem

14.1 Model

Accommodation is a two-level model: a **property** (`accommodation`) owned by a provider and located at a destination, containing one or more **room types** (`room_type`). The Platform sells room *types*, not individual rooms; the property allocates a physical room at check-in. This matches how small and mid-sized Zanzibar properties actually operate and avoids modelling a room-level PMS.

Supported property types: HOTEL, RESORT, LODGE, GUESTHOUSE, APARTMENT, VILLA. The type affects presentation and filtering only.

14.2 Rates

The rate for a given night is resolved as:

```
1. room_availability.rate_override for that (room_type, date)   -> use it
2. else room_type.base_rate                                     -> use it
Total stay = Σ nightly rate over [check_in .. check_out - 1 day]
```

Rates are per room per night, inclusive of the room for the stated occupancy. Extra-person charges, where a property applies them, are held in `room_type.amenities → pricing_rules` and applied deterministically at quote time. Rate plans (breakfast-inclusive, non-refundable) are modelled as separate `room_type` rows in V1 rather than as a rate-plan dimension; this is a deliberate simplification recorded in Section 44.3 as a candidate for normalisation in Version 2.

14.3 Availability

`room_availability` holds one row per room type per date. The provider publishes `rooms_open`; the system maintains `rooms_held` and `rooms_sold`.

```
sellable(room_type, date) = rooms_open - rooms_held - rooms_sold
                            (and NOT is_closed)

A stay of N nights for R rooms is bookable iff
    for every night d in the stay:  sellable(room_type, d) >= R
    and min_nights(first night) <= N
```

Rows are created by the provider's calendar upsert (`PUT /provider/room-types/{id}/availability`), which accepts a date range and applies a bulk update. A nightly job extends the calendar horizon (default 400 days) by copying the room type's defaults into any missing dates, so a provider who stops maintaining the calendar does not accidentally stop selling.

14.4 Booking Rules

Rule	Statement
BR-101	Check-out date must be strictly after check-in date; maximum stay is 30 nights
BR-102	Occupancy of the selected room type must accommodate the assigned guests; excess parties must book multiple rooms
BR-103	A booking may not be created for a past date, or within `accommodation.booking_cutoff_hours` of check-in (default 4)
BR-104	Availability is verified twice: indicatively at search, authoritatively under row lock at quote
BR-105	A confirmed accommodation booking decrements `rooms_open` availability through `rooms_sold`; it is never deleted, only cancelled
BR-106	The cancellation policy in force is the one snapshotted onto the booking at confirmation, not the property's current policy

14.5 Check-in and Check-out

`accommodation.check_in_time` and `check_out_time` drive the `STAY` itinerary item boundaries, and therefore the timing of the arrival transfer and any first-day activity. Early check-in and late check-out are not sold in V1; they are a provider-side arrangement noted in the booking's `guest_notes`.

14.6 Cancellation Policies

Policies are rows in `cancellation_policy`, referenced by properties and activities and snapshotted onto bookings:

Code	Rule
FLEX_48H	Full refund if cancelled more than 48 h before check-in; no refund thereafter
MODERATE_7D	Full refund > 7 days; 50% between 7 days and 48 h; none thereafter
STRICT_14D	50% refund > 14 days; none thereafter
NON_REFUNDABLE	No refund at any time

A policy is expressed generically as an ordered list of `{hours_before, refund_percent}` tiers, so administrators can create new policies without code changes. Evaluation is specified in Section 20.9.

14.7 Provider Operations

Providers manage properties, room types, photographs, amenities, the availability and rate calendar, arriving bookings, and cancellations, through the portal (Section 26). A provider may not modify a confirmed booking's dates or price; changes are handled as cancel-and-rebook or via a support ticket, so that the financial record stays coherent.

15 Attractions Subsystem

15.1 Model

An attraction is a place of interest, not a bookable product. It exists to help the tourist decide where to go and to give the itinerary a geographic anchor. Where an attraction can be visited commercially, a provider publishes an `activity` linked to it.

Fields: name, description, destination, coordinates, images, opening hours, entrance fee and currency, recommended visit duration, tags, accessibility notes, `feature_rank`, `is_active`.

15.2 Opening Hours

Stored as JSONB in a fixed schema, evaluated in the destination's timezone:

```
{
  "mon": [["09:00","18:00"]],
  "tue": [["09:00","18:00"]],
  "fri": [["09:00","12:00"],["14:00","18:00"]],
  "sun": [],
  "exceptions": [
    { "date": "2027-08-14", "closed": true, "reason": "Public holiday" }
  ]
}
```

Rule VR-12 evaluates a scheduled attraction visit against this structure and issues a warning when it falls outside opening hours.

15.3 Entrance Fees

`entrance_fee` and `fee_currency` are informational in V1: the fee is paid by the tourist on site and is shown in the itinerary as an "estimated on-site cost" that is **excluded from the trip total**, with that exclusion stated explicitly on the cost breakdown screen. Collecting entrance fees through the Platform requires a settlement relationship with each site operator and is listed as a Version 2 item.

15.4 Relationship to Activities

```
attraction (Stone Town)  1 ─── 0..* activity
                                  ├─ Heritage Walking Tour   (Provider A)
                                  ├─ Spice Farm & Stone Town (Provider B)
                                  └─ Sunset Dhow from Forodhani (Provider C)
```

An activity may also exist with no attraction (for example an open-water snorkelling excursion), in which case `activity.attraction_id` is null and the activity's own coordinates anchor it.

15.5 Use in the Itinerary

An attraction may be added to an itinerary as an `ATTRACTION` item, which occupies `visit_minutes` and triggers transfer insertion, but creates **no booking and no charge**. This lets a tourist plan a self-guided visit within the same timeline as their booked services.

16 Activity Subsystem

16.1 Model

An activity is a bookable, provider-supplied experience with a price, a duration, a capacity and a meeting point. Activities are entirely database-driven; no activity type, category or name appears in application code. The seed catalogue for Zanzibar includes snorkelling and diving excursions, Mnemba Atoll trips, dolphin tours, spice farm tours, Stone Town heritage and Freddie Mercury heritage walks, Prison Island trips, Jozani Forest visits, sunset dhow cruises, kitesurfing lessons, sandbank picnics, village and cooking experiences, and fishing trips — as **data**.

16.2 Schedules and Departures

Two concepts, deliberately separated:

- `activity_schedule` is a **recurring rule**: weekday mask, start time, capacity, validity window.
- `activity_departure` is a **concrete, sellable instance** at an exact instant with its own capacity counters.

A nightly job (`materialise_activity_departures`) expands schedules into departures across a rolling horizon (default 180 days). Providers may also create ad-hoc departures directly, and may cancel or close individual departures without touching the schedule.

```
activity: "Mnemba Atoll Snorkelling"
  schedule S1: Mon-Sat, 08:30, capacity 12, valid 2027-01-01..2027-12-31
       |
       | materialisation (nightly, idempotent upsert)
       v
  departure 2027-08-12T08:30+03  cap 12  held 2  sold 6   OPEN
  departure 2027-08-13T08:30+03  cap 12  held 0  sold 12  FULL
  departure 2027-08-14T08:30+03  cap 12  held 0  sold 0   CANCELLED (weather)
```

16.3 Capacity

```
sellable(departure) = capacity_total - capacity_held - capacity_sold
Bookable iff sellable >= pax
         and departure.status = 'OPEN'
         and now() <= departs_at - booking_cutoff_hours
         and min_pax <= pax <= max_pax
```

Capacity is enforced under row lock at quote time and by a database `CHECK` constraint, so oversell is impossible even under a race.

16.4 Requirements and Restrictions

`requirements JSONB` carries structured, machine-checkable restrictions that feed validation rule VR-15 and the booking guards:

```
{
  "min_age": 8,
  "max_age": null,
  "swimming_ability_required": true,
  "medical_declarations": ["pregnancy", "heart_condition"],
  "what_to_bring": ["swimwear", "towel", "sunscreen"],
  "not_suitable_for": ["reduced_mobility"]
}
```

Free-text is rendered to the tourist; structured keys are validated against the trip party where the data exists (for example `date_of_birth` on the tourist profile).

16.5 Deterministic Catalogue Ranking

When a tourist opens the activity list, results are ordered by an explicit, reproducible expression — never by a learned model:

```
ORDER BY
  (a.destination_id = :selected_destination) DESC,          -- exact destination first
  (a.tags && :interest_tags) DESC,                          -- tag overlap, boolean
  a.feature_rank ASC,                                       -- administrator curation
  agg.rating_avg DESC NULLS LAST,                           -- published reviews
  agg.rating_count DESC,
  a.price_per_person ASC,
  a.id ASC                                                  -- total order tie-break
```

The tourist may override with an explicit `sort` parameter (`price_asc`, `price_desc`, `rating`, `duration`, `distance`). The default ordering is documented in the help centre so that providers understand exactly how placement is earned — an obligation that a learned ranker could not meet.

16.6 Booking Cutoffs and Confirmation Mode

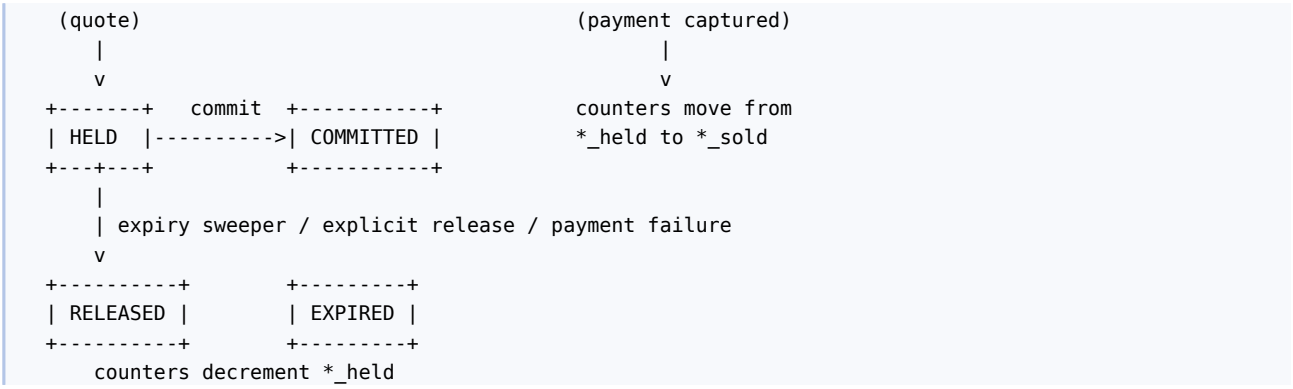
`activity.booking_cutoff_hours` prevents last-minute bookings the provider cannot staff. `activity.confirmation_mode` is either `INSTANT` (capacity is authoritative; the booking confirms on payment) or `ON_REQUEST` (the booking enters `AWAITING_PROVIDER` on payment, and the provider must accept within `provider_response_hours`, default 24, or it auto-cancels with full refund). V1 supports both; the catalogue displays the mode clearly.

17 Inventory and Availability Management

17.1 Principles

#	Principle
I1	Capacity counters live in exactly one place per resource type: `room_availability` and `activity_departure`
I2	Every capacity change happens inside a transaction that holds a row lock on the counter row
I3	Search may read cached or stale capacity; **committing** a booking may not
I4	Holds are explicit, time-boxed rows — never implicit reservations inferred from booking status
I5	Expiry is driven by a sweeper job **and** defensively re-checked at commit, so a delayed sweeper cannot cause an oversell

17.2 Hold Lifecycle



Hold TTL: 20 minutes at quote, extended to 30 minutes from the moment a payment intent is created (so a tourist completing a 3-D Secure challenge or a mobile-money prompt is not timed out). Both values are `system_setting` keys.

17.3 Concurrency Control

The critical section for placing a hold:

```
BEGIN;
SELECT id, rooms_open, rooms_held, rooms_sold, version
  FROM room_availability
 WHERE room_type_id = :rt AND stay_date BETWEEN :ci AND :co - 1
 ORDER BY id            -- ascending PK order: deadlock avoidance
   FOR UPDATE;
-- application asserts sellable >= rooms for every night
UPDATE room_availability
   SET rooms_held = rooms_held + :rooms, version = version + 1
 WHERE id = ANY(:ids);
INSERT INTO inventory_hold (...) VALUES (...);
COMMIT;
```

The database `CHECK (rooms_held + rooms_sold <= rooms_open)` is the final backstop. If it ever fires, that is a defect and the transaction aborts rather than overselling.

17.4 Oversell Prevention Summary

Layer	Control
Search	Indicative only; never authoritative
Quote	Row lock + application assertion + hold row
Confirm	Hold validity re-checked; expired hold → `409 HOLD_EXPIRED`
Capture	Hold committed inside the same transaction that confirms the bookings
Storage	`CHECK` constraints on both counter tables
Reconciliation	Nightly job compares Σ confirmed bookings against `*_sold` and alerts on drift

17.5 Expiry Sweeper

Runs every 60 seconds: selects `inventory_hold` rows with `status = 'HELD' AND expires_at < now()` in batches of 200, and for each, decrements the corresponding counter under lock and marks the hold `EXPIRED`. If the associated trip is `PENDING_PAYMENT` with a payment in a non-terminal state, the sweeper defers once and raises an alert rather than releasing under an in-flight payment.

18 Pricing Engine

18.1 Principles

Prices are computed server-side, always. The client never sends a price; it sends selections. Every computed price is reproducible from stored inputs: the applied tariff id, rate override, commission rule id and FX rate are all persisted on the booking or payment.

18.2 Price Components

Component	Source	Applied to
Accommodation line	Σ nightly rates × rooms	STAY items
Activity line	`price_per_person × pax`, or `price_per_group` when defined and cheaper for the party	ACTIVITY items
Transfer line	Corridor price or metered tariff (Section 12.4)	TRANSFER items
Platform service fee	`platform_fee_rate` × subtotal, or a flat minimum, whichever is greater	Trip
Taxes	Rule rows by country and service type	Trip and/or line
Discounts	Promotion codes (SHOULD-HAVE, Section 38)	Trip

18.3 Fees and Taxes

The platform service fee is a tourist-facing charge, distinct from the provider commission (which is deducted from the provider's share and is invisible to the tourist). Both may be non-zero simultaneously; the cost breakdown screen shows the service fee as its own line and never shows commission.

Tax rules are rows: `{country_id, service_type, tax_code, rate_percent, is_inclusive, valid_from, valid_to}`. For Tanzania, tourism VAT treatment and any infrastructure levies are configured as rows and must be confirmed with a tax adviser before launch; the engine supports both inclusive and exclusive computation:

```
exclusive: tax = round(line_total × rate, 2);      total = line_total + tax
inclusive: tax = round(line_total × rate/(1+rate), 2);  total = line_total
```

18.4 Currency and FX

Concept	Definition
Catalogue currency	The currency in which a provider publishes a price (`room_type.currency`, `activity.currency`)
Presentment currency	The currency shown to and charged to the tourist (`trip.currency`)
Settlement currency	The currency the PSP settles to the Platform
Payout currency	The currency in which a provider is paid (`provider.payout_currency`)

Conversion uses `fx_rate` rows refreshed hourly from a configured rate source, with a `markup_percent` (default 2.0) applied to protect against intra-day movement. **The rate used for a trip is frozen at `priced_at` and stored on the trip and on every derived record.** A trip's total can therefore never change because of a rate move between quoting and payment.

Rule: a trip's items must all resolve to a single presentment currency (VR-10). Mixed-currency catalogues are converted at quote time, not at display time, and the display shows the converted figure with a "converted from" note.

18.5 Rounding

All money is `Decimal`, `ROUND_HALF_UP`, at the currency's minor-unit precision (2 for USD, 2 for TZS as used by the PSP, 0 for currencies without minor units). Rounding is applied once per line total and once per aggregate; intermediate values retain full precision. The invariant `total = subtotal + fee + tax` is asserted before the trip is written, and a mismatch raises an internal error rather than persisting an inconsistent basket.

18.6 Quote Validity

A quote is valid for `quote_ttl_minutes` (default 20). After expiry the tourist must re-quote, which re-reads catalogue prices. If any price changed, the response is `409 PRICE_CHANGED` with a per-line old/new comparison, and the tourist must explicitly accept the new total. Silent re-pricing is prohibited.

19 Notification, Communication and Engagement Subsystems

19.1 Notification Events

Every row below is a real system event with a real trigger. There are no marketing broadcasts in V1 beyond the opt-in flagged channel.

Event code	Trigger	Recipients	Channels
`ACCOUNT_VERIFY`	Registration	Tourist	Email
`PASSWORD_RESET`	Reset request	Any	Email
`TRIP_QUOTE_EXPIRING`	5 min before `quote_expires_at`	Tourist	Push
`PAYMENT_SUCCEEDED`	Payment captured	Tourist	Push, Email
`PAYMENT_FAILED`	Payment failed	Tourist	Push, Email
`TRIP_CONFIRMED`	All bookings confirmed	Tourist	Push, Email (with itinerary PDF)
`BOOKING_CONFIRMED`	Component booking confirmed	Tourist, Provider	Push, Email
`BOOKING_AWAITING_PROVIDER`	On-request activity paid	Provider	Push, Email, SMS
`DRIVER_ASSIGNED`	Assignment accepted	Tourist, Driver	Push, Email
`DRIVER_OFFER`	Offer dispatched	Driver	Push (high priority), WebSocket
`DRIVER_EN_ROUTE`	Status → EN_ROUTE	Tourist	Push
`DRIVER_NEARBY`	1,500 m geofence	Tourist	Push
`DRIVER_ARRIVED`	Status → ARRIVED	Tourist	Push, SMS
`TRIP_REMINDER_72H` / `_24H` / `_3H`	Scheduler	Tourist	Push, Email (72 h only)
`ACTIVITY_REMINDER`	12 h before departure	Tourist	Push
`CHECKIN_REMINDER`	24 h before check-in	Tourist	Push
`BOOKING_CANCELLED`	Cancellation	Tourist, Provider, Driver	Push, Email
`REFUND_ISSUED`	Refund settled	Tourist	Push, Email
`MESSAGE_RECEIVED`	New message	Counterparty	Push
`REVIEW_INVITE`	4 h after booking completion	Tourist	Push, Email
`PAYOUT_RELEASED`	Payout paid	Provider	Email
`DOCUMENT_EXPIRING`	30 days before licence/insurance expiry	Driver, Provider	Push, Email
`VERIFICATION_DECISION`	Approval or rejection	Provider, Driver	Email, Push

19.2 Channels and Preferences

Push via FCM (Android and iOS via APNs bridge); email via a transactional email provider; SMS via a gateway with Tanzanian and international reach. Users control channels per **event class** (transactional, reminders, marketing) in `notification_preference`. Transactional notifications that carry safety or financial significance — `DRIVER_ARRIVED`, `PAYMENT_*`, `BOOKING_CANCELLED`, `REFUND_ISSUED` — cannot be disabled on at least one channel; this is stated in the terms of use.

Channel selection also degrades: if a push token is stale or push delivery fails, the dispatcher escalates to email, and for the three time-critical driver/arrival events, to SMS.

19.3 Templates and Localisation

`notification_template` rows are keyed by `(event_code, channel, locale)` and rendered with a sandboxed template engine over an explicit context dictionary. Templates are content, editable by administrators without deployment. Every template must exist in the default locale; a missing locale falls back to default rather than failing.

19.4 Delivery, Retry and Auditing

Each notification creates one `notification` row and one `notification_delivery` row per channel attempt, recording provider reference, status, attempt count and error detail. Retries: 5 attempts with exponential backoff (10 s, 60 s, 5 min, 30 min, 2 h), then dead-letter with an operations alert. Delivery receipts from the push and email providers update the delivery row via webhook where supported.

19.5 Booking-Scoped Messaging

A `conversation` is created automatically when a booking is confirmed, and is closed 72 hours after the booking completes. Participants are the tourist and either the assigned driver or the provider's staff. There is no free-form user-to-user messaging: every thread is anchored to a booking, which makes moderation tractable and gives every message a business context.

Property	Behaviour
Contact masking	Real phone numbers and emails are never exchanged; if voice contact is needed, the Platform surfaces a masked-number relay (SHOULD-HAVE) or the support line
Attachments	Images only, ≤ 5 MB, scanned and re-encoded, stored in object storage with signed short-lived URLs
Read state	`read_at` per message; delivery over WebSocket with REST fallback
Rate limit	60 messages per hour per conversation
Retention	24 months after trip completion, then purged

19.6 Moderation and Abuse Controls

Messages pass a deterministic filter that flags contact-detail patterns (phone numbers, email addresses, external messaging handles) and a maintained prohibited-term list. Flagged messages are still delivered but marked for review, except where they contain contact details, which are masked in transit — this is a rule-based regular-expression filter, not a classifier. Any participant may report a message; a report opens a moderation ticket, and repeated substantiated reports trigger account suspension under the compliance workflow.

19.7 Reviews

Rule	Statement
BR-121	Only a tourist whose booking reached `COMPLETED` may review that booking; one review per booking
BR-122	The review window opens 4 hours after completion and closes 30 days later
BR-123	A review carries an overall 1–5 rating plus a subject-specific breakdown (accommodation: cleanliness, location, value, staff; driver: punctuality, driving, vehicle, courtesy; activity: guide, value, safety, enjoyment)
BR-124	Reviews are published after moderation; the default is auto-publish with a deterministic pre-filter, and any review flagged by the filter or by a user is held for human moderation
BR-125	Providers may post exactly one public response per review
BR-126	Neither party may edit a published review; the tourist may withdraw theirs, which unpublishes it and recomputes aggregates
BR-127	Ratings are aggregated in `rating_aggregate` per subject with count, mean to two decimals, and a per-star histogram; a subject with fewer than 3 published reviews displays "New" rather than a mean

The moderation pre-filter checks for prohibited terms, contact details, and reviews that reference a booking the reviewer did not hold. It is rule-based. There is no sentiment model.

19.8 Anti-Abuse in Reviews

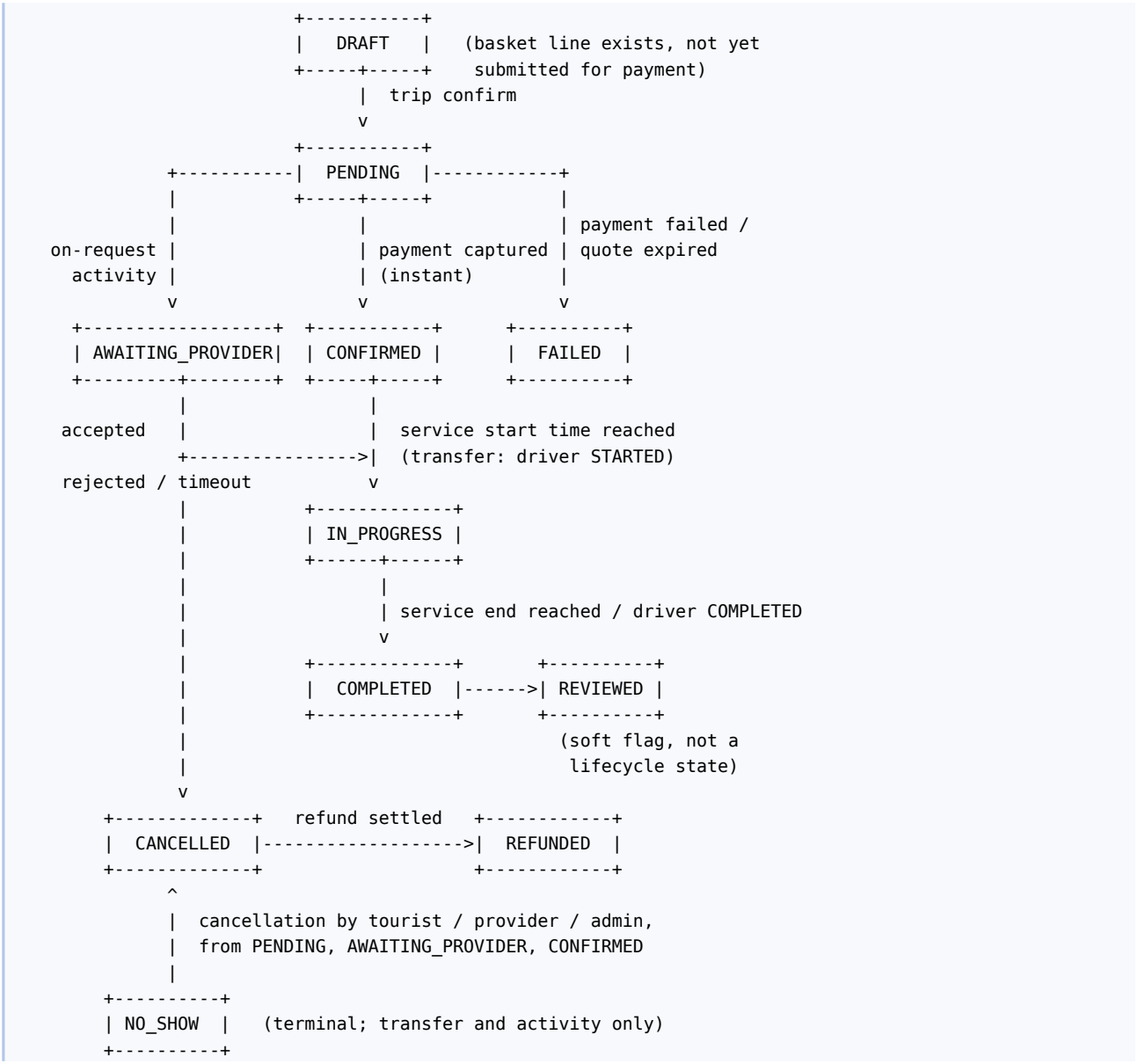
Because a review must be attached to a completed, paid booking, fake-review injection requires a real paid transaction, which is the strongest available structural deterrent. In addition: a tourist account with an unusual ratio of five-star reviews to bookings, or multiple bookings with the same provider from a shared device or payment instrument, raises a deterministic flag for human review (thresholds in `system_setting`). Providers cannot review tourists in V1; driver-side feedback on tourists is captured as a private incident report visible only to administrators.

20 Booking Engine, Policies and State Machines

20.1 Responsibilities

The booking engine is the single authority for creating, transitioning, cancelling and completing every commercial reservation, regardless of service type. Type-specific behaviour lives in the subtype tables and in small strategy classes; the lifecycle, the audit trail and the transition guards are shared. No other module may write `booking.status`.

20.2 Booking State Machine



From	To	Actor	Guard
DRAFT	PENDING	Tourist	Valid quote token; hold live
PENDING	CONFIRMED	System	Payment captured; hold committed; instant-confirmation service
PENDING	AWAITING_PROVIDER	System	Payment captured; `confirmation_mode = ON_REQUEST`
PENDING	FAILED	System	Payment failed or hold expired
AWAITING_PROVIDER	CONFIRMED	Provider	Within `provider_response_hours`
AWAITING_PROVIDER	CANCELLED	Provider / System	Rejected, or response window elapsed → automatic full refund
CONFIRMED	IN_PROGRESS	System / Driver	Service start reached, or driver sets `STARTED`
IN_PROGRESS	COMPLETED	System / Driver	Service end reached, or driver sets `COMPLETED`
CONFIRMED / IN_PROGRESS	NO_SHOW	Driver / Provider	Documented wait elapsed; evidence recorded
PENDING / AWAITING_PROVIDER / CONFIRMED	CANCELLED	Tourist / Provider / Admin	Policy evaluated; refund computed
CANCELLED	REFUNDED	System	Refund settled at PSP

Every transition writes a `booking_status_history` row with actor, reason and timestamp. Transitions are validated by a single table-driven guard function; an illegal transition raises `ConflictError` → `409 ILLEGAL_TRANSITION`. Administrators may force a transition via `POST /admin/bookings/{id}/force-transition`, which requires a reason, is restricted to `SUPER_ADMIN`, and is always audited.

20.3 Type-Specific Nuances

Aspect	ACCOMMODATION	ACTIVITY	TRANSFER
Inventory	`room_availability` nights	`activity_departure` seats	None (vehicle class, not a unit)
Confirmation mode	Always instant	Instant or on-request	Instant
Fulfilment actor	Property staff	Activity guide	Assigned driver
`IN_PROGRESS` trigger	Check-in datetime reached	Departure time reached	Driver sets `STARTED`
`COMPLETED` trigger	Check-out datetime reached	Departure + duration	Driver sets `COMPLETED`
NO_SHOW applicable	No (charged per policy)	Yes	Yes
Extra entity	`booking_accommodation`	`booking_activity`	`booking_transfer` + `driver_assignment`

20.4 Payment Status Linkage

Booking status is derived from, and must remain consistent with, the payment covering the trip.

Payment status	Permitted booking statuses in that trip
INITIATED, PENDING	PENDING only
AUTHORISED	PENDING only
CAPTURED	CONFIRMED, AWAITING_PROVIDER, IN_PROGRESS, COMPLETED, CANCELLED, NO_SHOW
FAILED, EXPIRED	FAILED or DRAFT (holds released)
REFUNDED (full)	CANCELLED or REFUNDED
PARTIALLY_REFUNDED	Any post-capture state; at least one booking CANCELLED/REFUNDED

A nightly consistency job asserts this table across all live trips and raises a P2 alert on any violation. This is the single most important invariant in the system: a confirmed booking with no captured payment means the Platform has promised a service it has not been paid for.

20.5 Trip State Machine

```
DRAFT ---generate/price---> PRICED ---confirm---> PENDING_PAYMENT
  ^                           |                        |
  |<---- quote expired --------+                        | payment captured
  |<---- payment failed --------------------------------+
                                                       v
                                                  CONFIRMED
                                                       |
                                             first booking IN_PROGRESS
                                                       v
                                                 IN_PROGRESS
                                                       |
                                            all bookings terminal (COMPLETED/
                                             CANCELLED/NO_SHOW)
                                                       v
                                                  COMPLETED

 Any state -> CANCELLED  (tourist cancels whole trip, or all component
                          bookings end cancelled)
```

20.6 Driver Availability State Machine

```
        OFFLINE  <-------------------+
           |                        |
  go online (verified, valid docs)  | go offline / documents expired /
           v                        | suspended
        ONLINE ----------------------+
           |
  offer accepted
           v
        ENGAGED  --- assignment COMPLETED/CANCELLED ---> ONLINE
```

A driver in `ENGAGED` receives no further offers whose time window overlaps the current assignment, but may accept non-overlapping future scheduled work.

20.7 Concurrency and Locking

The booking module never holds a transaction open across an external call. The confirmation routine below is structured so that the PSP interaction happens *before* the transaction, and the transaction contains only local work.

20.8 Confirmation Routine (on payment capture)

Executed inside one database transaction, triggered by the webhook handler or the verification poller. Idempotent: re-running it for an already-confirmed trip is a no-op.

```
advisory-lock(payment.id)
reload payment; IF status already CAPTURED and trip CONFIRMED -> return
set payment.status = CAPTURED, captured_at, settlement_amount, fx_rate
FOR each booking in trip (ordered by id):
    lock booking FOR UPDATE
    assert booking.status = PENDING
    FOR each inventory_hold of this booking:
        lock the counter row FOR UPDATE
        assert hold.status = HELD and not expired
        counter.*_held -= qty ; counter.*_sold += qty
        hold.status = COMMITTED
    booking.status = CONFIRMED   (or AWAITING_PROVIDER)
    booking.confirmed_at = now()
    snapshot commission_rate, commission_amount, net_amount
    write booking_status_history
trip.status = CONFIRMED ; trip.confirmed_at = now()
lock itinerary items; set is_locked = true for covered items
queue domain events (after commit): TripConfirmed, BookingConfirmed×N
COMMIT
after commit: dispatch driver offers, notifications, ledger accrual
```

Failure at step 9 (a hold expired while the payment was in flight) is the one genuinely hard case. Handling: attempt to re-acquire capacity immediately under the same lock; if capacity is available, proceed; if not, mark that single booking `FAILED`, confirm the remainder, and initiate an automatic partial refund for the failed component, notifying the tourist with alternatives. Failing the entire trip because one activity sold out would be a worse outcome for everyone.

20.9 Cancellation and Refund Policy Engine

Policy evaluation is a pure function:

```
INPUT : policy_snapshot (ordered tiers), booking.starts_at,
        cancelled_at, gross_amount, cancelling_party
OUTPUT: refund_amount, provider_compensation, platform_fee_retained

hours_before = (booking.starts_at - cancelled_at) / 3600

IF cancelling_party in (PROVIDER, DRIVER, PLATFORM):
    refund_percent = 100                       # BR-045: never penalise the
                                               # tourist for supply failure
ELSE:
    refund_percent = first tier T in policy_snapshot.tiers
                     (ordered by hours_before DESC)
                     where hours_before >= T.hours_before
                     else 0

refund_amount = round(gross_amount × refund_percent / 100, 2)

platform_fee_retained:
    0 if cancellation is by provider/platform, or > 7 days before start
    else the pro-rata service fee for that component

provider_compensation = gross_amount - refund_amount - platform_fee_retained
                        (accrued to the provider ledger as a cancellation fee)
```

Worked example: activity, gross USD 110.00, policy `MODERATE_7D` (100% > 7 days, 50% between 7 days and 48 h, 0% within 48 h), cancelled by the tourist 3 days before departure. `refund_percent = 50` → refund USD 55.00; service fee for that line (5% = 5.50) retained; provider compensation USD 49.50.

Refund preview is available before committing (`GET /bookings/{id}/cancellation-preview`), so the tourist always sees the financial consequence before acting.

Trip-level cancellation evaluates each component booking independently against its own snapshotted policy and returns an itemised total.

20.10 No-Show and Dispute Handling

`NO_SHOW` requires evidence: for a transfer, the driver must have been inside the pickup geofence for the configured wait period (default 60 minutes for airport pickups, 15 minutes otherwise), evidenced by `driver_location` rows. The tourist is notified at 15, 30 and 45 minutes. A tourist may dispute a `NO_SHOW` within 7 days, which opens a support ticket; an administrator may reverse the outcome, which triggers a compensating refund and a reversing ledger entry — never an edit to the original records.

21 Payment Subsystem

21.1 Principles and PCI Scope

#	Principle
PM1	The Platform never sees, transmits or stores a primary account number, CVV or full magnetic-stripe data. Card entry happens in a PSP-hosted field or redirect. The target compliance scope is **PCI DSS SAQ A**.
PM2	The charged amount is computed server-side from `trip.total_amount`; a client-supplied amount is never trusted.
PM3	Every money-moving request carries an idempotency key and is safe to retry.
PM4	The PSP is the authority on payment state; the Platform reconciles to it, never the reverse.
PM5	Webhooks are signature-verified, replay-protected and idempotent.
PM6	Every payment state change writes an immutable `payment_transaction` row.

21.2 Provider Selection

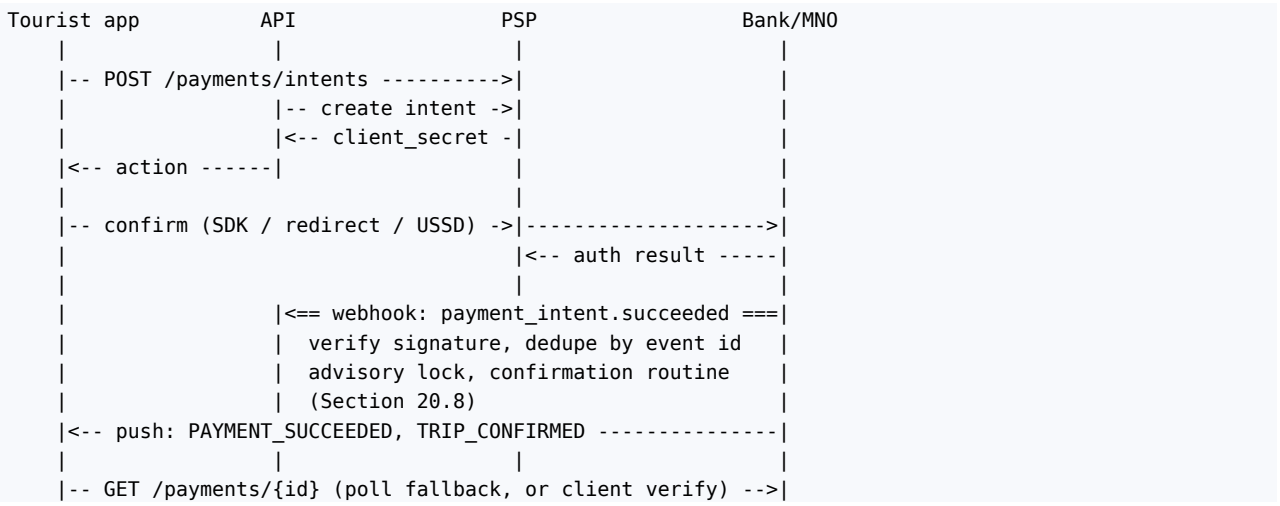
No provider is pre-selected. The architecture requires one card acquirer with international acceptance plus one mobile-money aggregator with Tanzanian coverage, both behind a common port.

```python
class PaymentGatewayPort(Protocol):
    def create_intent(self, req: PaymentIntentRequest) -> PaymentIntentResult: ...
    def capture(self, psp_ref: str, amount: Money) -> TransactionResult: ...
    def refund(self, psp_ref: str, amount: Money, reason: str) -> TransactionResult: ...
    def fetch_status(self, psp_ref: str) -> PaymentStatus: ...
    def verify_webhook(self, headers: Mapping, body: bytes) -> WebhookEvent: ...
```

Candidate	Strengths	Weaknesses
Stripe	Best-in-class international card acceptance, 3-D Secure 2, strong SDKs and webhooks, multi-currency presentment	Tanzanian entity support and settlement into TZS must be verified commercially; no local mobile money
Flutterwave	Cards plus African mobile money including Tanzanian networks; single integration	Variable settlement times; reconciliation reporting is less mature
DPO Pay	Established East African acquirer, strong local settlement	Weaker developer experience and webhook reliability
Selcom / Azam Pay (local aggregators)	Direct M-Pesa, Tigo Pesa, Airtel Money, Halopesa coverage	Domestic focus; limited international card acceptance

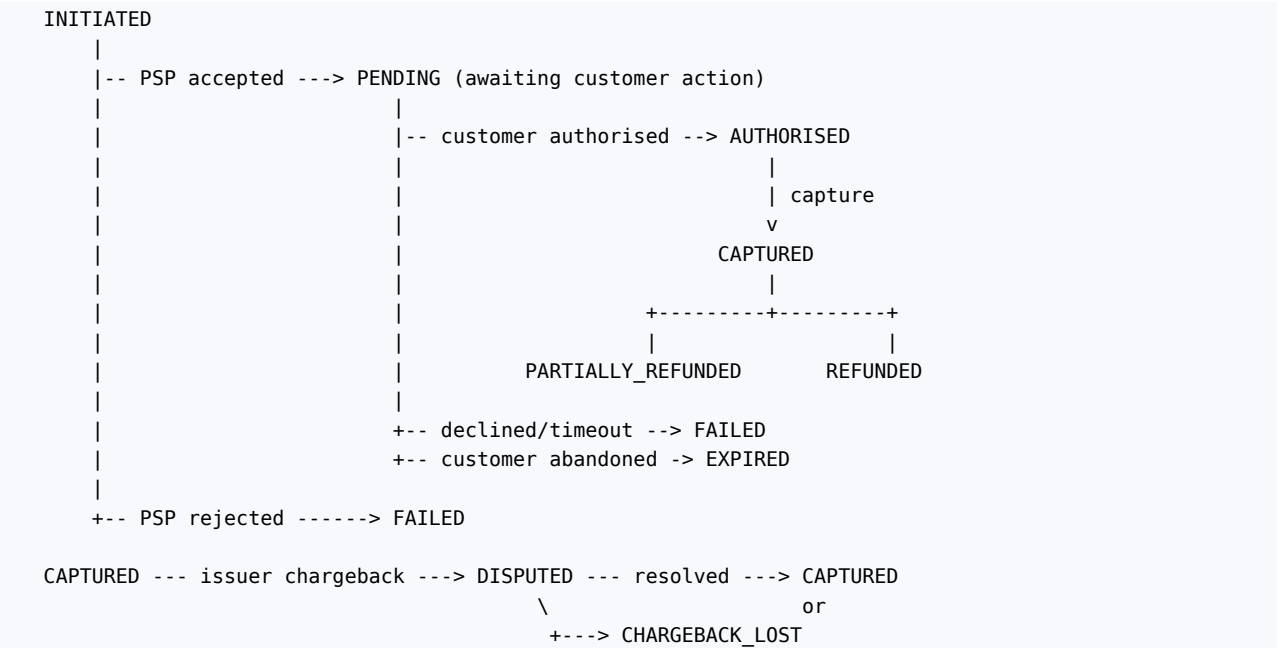
Recommended V1 configuration: an international card acquirer (Stripe or Flutterwave) as the primary rail, since the target customer is an international tourist paying before travel, plus a local mobile-money aggregator as a secondary rail for in-destination top-ups and for provider payouts. The adapter pattern means this decision can be revisited without touching business logic.

21.3 Payment Flow



Card flows use 3-D Secure 2 where the issuer requires it; the intent result carries a challenge action which the mobile SDK handles. Mobile-money flows are asynchronous by nature: the intent returns a USSD-push instruction, the app shows a waiting state with a 5-minute countdown, and the outcome arrives by webhook or by the polling fallback.

21.4 Payment State Machine



Auto-capture is used for card payments (authorise-and-capture in one step) because the service is confirmed immediately; separate authorisation is retained in the model for the future on-request activity flow where a hold before provider acceptance is preferable.

21.5 Webhooks and Idempotency

Control	Implementation
Authenticity	HMAC signature verified against the PSP's signing secret held in the secrets manager
Freshness	Reject events with a timestamp skew > 5 minutes
Deduplication	`payment_webhook_event.psp_event_id` unique; a duplicate returns `200` without reprocessing
Ordering	Events may arrive out of order; the handler applies a state transition only if it is legal and advances the state, otherwise it records and ignores
Durability	The raw payload is persisted before any processing, so a processing bug never loses an event
Fallback	`verify_payment_status` polls the PSP at 30 s, 2 min, 5 min, 10 min, 20 min and 30 min after intent

Control	Implementation
	creation if no terminal webhook has arrived

21.6 Refunds

Refunds are always initiated by the Platform (never by a provider directly) and always reference an original payment. Full and partial refunds are supported; a partial refund carries the specific `booking_id` so the ledger can reverse the correct accrual. Approval: refunds arising automatically from a cancellation policy are issued without human approval; discretionary or goodwill refunds above `refund_auto_approve_limit` require a `FINANCE_OFFICER`. Refunds return to the original instrument; mobile-money refunds return to the originating wallet number.

21.7 Multi-Currency Handling

Presentment currency is chosen by the tourist (default from `preferred_currency`, constrained to the set the PSP supports for their country). The charge is made in the presentment currency; the PSP settles in the settlement currency; the FX rate and any markup are recorded on the payment row. Provider payouts are made in the provider's payout currency, converted at the rate in force on the payout date and recorded on the payout. Every conversion in the system stores its rate, source and timestamp — there are no untraceable conversions.

21.8 Failure Taxonomy

Normalised code	Typical cause	Tourist-facing behaviour
`INSUFFICIENT_FUNDS`	Declined for funds	Retry with a different method; holds retained until the payment window closes
`CARD_DECLINED`	Issuer decline	Same
`AUTHENTICATION_FAILED`	3-D Secure abandoned or failed	Re-attempt the challenge
`EXPIRED_INSTRUMENT`	Expired card	Prompt for another method
`MOBILE_MONEY_TIMEOUT`	No USSD approval	Resend the prompt once, then offer card
`GATEWAY_UNAVAILABLE`	PSP outage	Show a service message; holds extended by the outage duration up to a cap; operations alerted
`AMOUNT_MISMATCH`	Client/server disagreement	Hard failure, audited as a potential tampering attempt

After three consecutive failures on one trip, further attempts are throttled for 30 minutes and the tourist is offered support contact.

21.9 Reconciliation

A daily job ingests the PSP settlement report and matches every line to a `payment` or `refund` by `psp_reference`. Outcomes: matched; unmatched-at-PSP (money received with no local record — always investigated as P1); unmatched-locally (local capture with no settlement line — retried, then escalated); amount mismatch (recorded as an FX or fee difference and posted to a dedicated ledger account). A reconciliation report is produced daily and is the basis of the finance month-end close.

22 Revenue, Commission and Settlement

22.1 Commercial Model

```
Tourist pays gross                        e.g. USD 834.75
     |
     v
Platform receives into PSP account
     |
     +-- Platform service fee (tourist-facing)      USD  39.75
     |
     +-- Component gross by provider
          |
          +-- Accommodation provider  465.00
          |       - commission 15%     -69.75  -> net 395.25
          +-- Activity provider        170.00
          |       - commission 18%     -30.60  -> net 139.40
          +-- Transport provider       160.00
                  - commission 20%     -32.00  -> net 128.00
     |
     v
Platform revenue = service fee 39.75 + commissions 132.35 = USD 172.10
Provider payouts  = 395.25 + 139.40 + 128.00               = USD 662.65
```

22.2 Commission Rule Resolution

Rules are rows in `commission_rule` with a `scope` and a `priority`. Resolution takes the highest-priority active rule matching the booking, evaluated in this order:

```
1. scope = LISTING   and listing_id  matches       (a negotiated rate for one hotel)
2. scope = PROVIDER  and provider_id matches
3. scope = TYPE      and booking_type matches      (e.g. all ACTIVITY = 18%)
4. scope = GLOBAL                                  (default fallback)
```

Calculation methods: `PERCENT` (rate × gross), `FLAT` (fixed amount), `TIERED` (bands over monthly provider volume, evaluated against the completed volume of the preceding calendar month so the rate is deterministic within a month). `min_fee` and `max_fee` clamp the result. The resolved rule id and the effective rate are **snapshotted onto the booking at confirmation**, so a later rule change cannot retroactively alter a settled booking.

22.3 Ledger Entry Types

The ledger is double-entry and append-only. Every booking generates a balanced set of entries.

Entry type	Direction	Account	When
`CUSTOMER_PAYMENT`	CREDIT	Platform clearing	Payment captured
`PROVIDER_ACCRUAL`	CREDIT	Provider payable	Booking completed
`COMMISSION_REVENUE`	CREDIT	Platform revenue	Booking completed
`SERVICE_FEE_REVENUE`	CREDIT	Platform revenue	Payment captured
`PSP_FEE`	DEBIT	Platform expense	Settlement reconciliation
`REFUND_TO_CUSTOMER`	DEBIT	Platform clearing	Refund settled
`PROVIDER_ACCRUAL_REVERSAL`	DEBIT	Provider payable	Cancellation after accrual
`COMMISSION_REVERSAL`	DEBIT	Platform revenue	Cancellation after accrual
`CANCELLATION_FEE_ACCRUAL`	CREDIT	Provider payable	Cancellation with provider compensation
`PAYOUT_SETTLEMENT`	DEBIT	Provider payable	Payout released
`FX_DIFFERENCE`	Either	Platform FX account	Reconciliation variance
`GOODWILL_CREDIT`	DEBIT	Platform expense	Support-issued credit
`CHARGEBACK`	DEBIT	Platform clearing	Dispute lost

Invariant asserted nightly (BR-064): for every booking and currency, `Σ CREDIT − Σ DEBIT` equals the expected residual, and `Σ PROVIDER_ACCRUAL − Σ PAYOUT_SETTLEMENT − Σ reversals` equals `provider_balance.available_amount + pending_amount`.

22.4 Settlement Timing

Accrual occurs at **booking completion**, not at payment, so that the Platform does not owe a provider for a service not yet delivered. Between capture and completion the money sits in the clearing account. A configurable hold period (`settlement_hold_days`, default 2) after completion allows disputes to surface before funds become available for payout.

```
capture ----> [clearing] ----completion----> [provider payable, pending]
                                                        |
                                          settlement_hold_days elapsed
                                                        v
                                                [provider payable, available]
                                                        |
                                                weekly payout batch
                                                        v
                                                    [paid out]
```

22.5 Payouts

A scheduled job assembles, per provider and currency, all `available` ledger balance into a `payout` with `payout_item` lines referencing each booking. Rules: minimum payout threshold (`payout_minimum`, default USD 50 equivalent) below which the balance rolls forward; payouts require `FINANCE_OFFICER` approval before release; release calls the payout rail (bank transfer or mobile money) via an adapter; the payout row records the rail reference and paid timestamp; failure marks the payout `FAILED` and returns the balance to available, with an alert.

Providers see, in the portal: current available balance, pending balance with expected availability dates, each payout with its line items, and downloadable statements per period.

22.6 Refund Impact on Settlement

Scenario	Ledger effect
Cancellation before completion (no accrual yet)	`REFUND_TO_CUSTOMER` debit only; no provider impact
Cancellation after completion but before payout	`REFUND_TO_CUSTOMER`, `PROVIDER_ACCRUAL_REVERSAL`, `COMMISSION_REVERSAL`, and `CANCELLATION_FEE_ACCRUAL` for any retained provider compensation
Cancellation after payout	Reversal entries create a negative provider balance recovered from the next payout; if no future payout is expected, an invoice is raised and the case escalates to finance
Chargeback	`CHARGEBACK` debit plus reversal of provider accrual, subject to the provider agreement's chargeback allocation clause

22.7 Financial Reporting Outputs

Daily: gross bookings, net revenue, commission by type, refunds, PSP fees, reconciliation exceptions. Monthly: provider statements, revenue by destination and service type, take rate, cancellation and refund rates, payout register. All reports are generated from the ledger, never from the booking table, so the reported figures reconcile to money movement by construction.

23 Mobile Application Architecture

23.1 Applications

App	Audience	Platforms	Distribution
Tourist App	International tourists	iOS 15+, Android 8+	App Store, Google Play
Driver App	Verified drivers	Android 8+ primary; iOS 15+ secondary	Google Play, App Store

23.2 Framework Decision

Flutter 3.x (Dart) for both applications.

Criterion	Assessment
Single codebase, two platforms	Halves client effort for a small team; the two apps additionally share a common package for the API client, models and design system
Rendering consistency	Identical UI across platforms; important for a booking flow whose correctness depends on precise state display
Background location on Android	Adequate via platform channels and `flutter_background_geolocation`-class plugins; the driver app's most demanding requirement
Ecosystem	Mature packages for maps, push, secure storage, local database
Risks	Platform-channel work for background location and for PSP SDKs; larger binary than native

Alternatives: React Native (comparable; rejected because the team's Dart/Flutter experience is stronger and background-location plugin quality is better in Flutter), fully native Kotlin + Swift (best platform fidelity; rejected on cost — it doubles client effort for a marginal gain in this product).

23.3 Client Layer Structure

```
lib/
├── main.dart
├── app/                  routing, theming, dependency injection, env config
├── core/
│   ├── api/              Dio client, interceptors, error mapping, retry
│   ├── auth/             token store, refresh, session state
│   ├── storage/          secure storage, Drift (SQLite) database, cache
│   ├── connectivity/     online state, request queue
│   ├── location/         permissions, fixes, geofence helpers
│   ├── notifications/    FCM registration, handlers, deep-link routing
│   └── errors/           failure types, user-facing message mapping
├── domain/               entities, value objects, repository interfaces
├── data/                 DTOs, mappers, repository implementations
├── features/
│   ├── auth/  onboarding/  explore/  planner/  itinerary/
│   ├── checkout/  trips/  tracking/  messages/  reviews/  profile/
│   └── (driver app) availability/ offers/ assignment/ earnings/
└── shared/               design system, widgets, formatters, validators
```

Each feature folder contains `presentation/` (screens, widgets), `application/` (state notifiers/blocs) and `domain/` bindings. Business logic never lives in a widget.

23.4 Navigation

Declarative routing with `go_router`, giving deep links and web-parity URLs for the tourist app.

```
/                         splash -> resolves auth -> /home or /onboarding
/onboarding
/auth/login  /auth/register  /auth/forgot
/home
/explore                        /explore/destinations/:id
/attractions/:id                /activities/:id
/stays?destination=&in=&out=    /stays/:id            /stays/:id/rooms
/trips                          /trips/:id
/trips/:id/planner              /trips/:id/flights
/trips/:id/transport            /trips/:id/itinerary
/trips/:id/summary              /trips/:id/costs
/trips/:id/pay                  /trips/:id/confirmed
/trips/:id/active               /bookings/:id/track
/messages   /messages/:id       /notifications
/reviews/new/:bookingId
/profile    /settings           /help
```

Deep links from push notifications map to these routes; an unauthenticated deep link stores the intended route and resumes it after login.

23.5 State Management

Riverpod 2 with immutable state classes. Three state tiers:

Tier	Contents	Lifetime
Session	Principal, tokens, locale, currency	App lifetime, persisted
Feature	Screen and flow state (e.g. the trip draft under construction)	Route lifetime, persisted for the planner so a killed app does not lose a half-built trip
Ephemeral	Form field state, animation state	Widget lifetime

The trip draft is the one piece of client state with real persistence requirements: it is mirrored to the server on every mutation (the server is authoritative) and cached locally so the planner survives a connectivity loss.

23.6 API Communication

Dio client with an interceptor chain: attach access token → attach `X-Request-Id`, `Accept-Language`, `X-Currency` → attach `Idempotency-Key` on mutating calls → on `401`, refresh once and replay → on `429/503`, exponential backoff with jitter for idempotent calls only → map error envelope to typed failures. Timeouts: 10 s connect, 20 s receive, 40 s for payment intent creation. Certificate pinning is enabled for the API and payment domains in release builds.

23.7 Local and Secure Storage

Data	Store	Notes
Access/refresh tokens, pickup PIN	Platform secure storage (Keychain / Android Keystore)	Never in shared preferences; cleared on logout
Confirmed itinerary, bookings, vouchers, driver details	Encrypted SQLite (Drift + SQLCipher)	The offline pack (Section 23.8)
Catalogue browse cache, images	Cache directory with TTL	Evictable
Draft trip	Encrypted SQLite	Mirrors server state
Driver queued status events and location batches	Encrypted SQLite, FIFO outbox	Replayed on reconnect

23.8 Offline Handling

The tourist lands in a country where their SIM may not work. Offline capability is therefore a functional requirement, not a nicety.

Capability	Offline behaviour
Confirmed itinerary, per-day plan, vouchers	Fully available from the encrypted local pack, downloaded automatically on trip confirmation and refreshed on every app open
Driver name, photo, vehicle, plate, pickup point, pickup PIN, support number	Available offline
Map of pickup point	Static map image cached at confirmation; live map requires connectivity
Browsing and planning	Read-only from cache with a clear “offline” banner; mutations are blocked rather than queued, because pricing and availability must not be computed against stale data
Payment	Blocked offline
Messaging	Composed messages queue and send on reconnect, marked “pending”
Driver status transitions	Queued in an outbox with the device timestamp, replayed in order on reconnect; the server accepts a timestamp up to 6 hours old for replayed events and marks them `offline_replay = true`

The rule that keeps this safe: **anything that spends money or consumes inventory requires connectivity; anything that merely reports or displays does not.**

23.9 Push Notifications

FCM for Android and APNs (via FCM) for iOS. The device registers its token at login and on token rotation (`POST /me/devices`), and unregisters at logout. Payloads are data-only messages carrying `event_code`, `deep_link` and a minimal display payload — never PII beyond a first name. High-priority channels are used for `DRIVER_OFFER` (driver app) and `DRIVER_ARRIVED` (tourist app); the driver app additionally uses a full-screen intent and a distinct sound for offers, because a missed offer is a lost booking.

23.10 Maps and Location

The map widget is provided by the same vendor as the routing port where licensing allows, to avoid attribution and tile-cost duplication. Location permissions are requested contextually with an explanatory screen immediately before the request, never at first launch. The driver app requires background location and declares it in both stores with the transport-service justification; the tourist app requests only foreground location and only when setting a custom pickup point.

23.11 Real-Time

A single WebSocket connection per session, authenticated at connect, resubscribed on resume, with exponential-backoff reconnection and a REST polling fallback after three failed reconnects. Messages are small JSON envelopes `{channel, event, data, sent_at}`. The client ignores unknown event types, so the server can add events without forcing an app update.

23.12 Performance, Accessibility and Compatibility

Cold start under 2.5 s on a mid-range Android device; list scrolling at 60 fps; images served in WebP with responsive variants and lazy loading. Accessibility: minimum contrast 4.5:1, minimum touch target 48 dp, full screen-reader labelling, dynamic type support up to 200%, no colour-only status encoding (booking status uses icon plus text plus colour). Compatibility floor: iOS 15, Android 8 (API 26); tested on 360 dp width upward.

23.13 Release Management

Semantic versioning; a `min_supported_version` returned by the API forces an upgrade prompt when a client falls below the floor; feature flags delivered through `GET /config` allow server-side control of client features without a store release. Crash and performance monitoring in both apps, with PII scrubbing in breadcrumbs.

24 Tourist Application — Screen Specifications

Each screen is specified with purpose, components, user actions, data displayed, validation, API calls, navigation, and the three mandatory UI states (loading, empty, error). Common error handling: network failure shows a retry affordance with cached content where available; `401` triggers silent refresh then re-login; `5xx` shows a generic message plus the `request_id` for support.

24.1 Splash

Purpose Resolve session and configuration before showing anything else. **Components** Logo, indeterminate progress. **Actions** None. **Data** None displayed. **API** `GET /config` (min version, feature flags, enabled currencies), token refresh if present. **Navigation** → Onboarding (first run) / Home (authenticated) / Login (expired). **States** Loading is the screen; error → offline notice with retry; forced-upgrade → blocking dialog with store link. Maximum display 3 s before falling through to cached config.

24.2 Onboarding

Purpose Explain the value proposition in three panels: plan before you fly, one price, met at the airport. **Components** Carousel, progress dots, Skip, Get Started. **Actions** Swipe, skip, continue. **Data** Static localised content.

API None. **Navigation** → Registration or Login. **States** No loading or empty state; shown once, controlled by a local flag.

24.3 Registration

Purpose Create a tourist account. **Components** First name, last name, email, password with strength meter, nationality picker, currency picker, terms checkbox, social sign-in buttons. **Actions** Submit, switch to login, open terms. **Validation** All names 2–80 characters; email format; password ≥ 12 characters with the server-side breach check; terms must be accepted; nationality required. **API** `POST /auth/register`. **Navigation** → Email verification notice → Home. **States** Loading disables the form and shows an inline spinner on the button; `409` maps to a field-level "this email is already registered" with a login link; `422` maps errors to fields by the `details[].field` path.

24.4 Login

Purpose Authenticate. **Components** Email, password with reveal toggle, remember-me, submit, forgot-password link, social sign-in. **Validation** Non-empty; format check on email. **API** `POST /auth/login`. **Navigation** → Home, or → intended deep link. **States** Error banner for `401` using a single non-enumerating message; `423` shows the lockout expiry time; `403 EMAIL_NOT_VERIFIED` offers to resend verification.

24.5 Forgot Password

Purpose Initiate and complete a reset. **Components** Email field; on the deep-linked second step, new password and confirmation. **Validation** As registration. **API** `POST /auth/password/forgot`, `POST /auth/password/reset`. **Navigation** → Login on success. **States** The success message is identical whether or not the address exists, to avoid account enumeration; expired-token errors offer to restart.

24.6 Home

Purpose Orient the tourist: continue an active or draft trip, or start exploring. **Components** Active-trip card (next item, countdown, driver status when applicable), draft-trip card with a Resume action, featured destinations carousel, featured activities carousel, a prominent Plan a Trip button, notification bell with unread count. **Data** Trips summary, curated catalogue. **API** `GET /trips?status=active,draft`, `GET /destinations`, `GET /activities?featured=true`. **Navigation** → Active Trip / Trip Planner / Explore / Notifications. **States** Loading uses skeleton cards; empty (no trips) shows an illustrated first-run prompt; error shows cached content with a stale badge.

24.7 Explore Zanzibar

Purpose Browse the destination catalogue. **Components** Search field, map/list toggle, category chips (beaches, heritage, water sports, nature, culture), destination cards, activity cards. **Actions** Search, filter, open a detail, add to a trip. **API** `GET /search`, `GET /destinations`, `GET /activities`, `GET /attractions`. **Navigation** → Destination / Attraction / Activity detail. **States** Skeletons while loading; a no-results state that offers to clear filters; error with retry. Search input is debounced at 350 ms and requires two characters.

24.8 Destinations

Purpose Present a destination and everything bookable within it. **Components** Hero gallery, description, map with the destination boundary, distance-from-airport chip, tabbed lists of attractions, activities and stays. **Data** Destination detail plus counts. **API** `GET /destinations/{id}`, then the tabbed lists. **Navigation** → detail screens; → Trip Planner with the destination preselected. **States** Standard; tabs load independently so one failure does not blank the screen.

24.9 Attraction Details

Purpose Inform, and allow addition to the plan as an unbooked visit. **Components** Gallery, description, opening-hours table for the coming week, entrance fee with an explicit "paid on site, not included in your trip total" note, recommended visit duration, map, related activities. **Actions** Add to trip, view on map, open a related activity. **API** `GET /attractions/{id}`. **Navigation** → Trip Planner (item added) / Activity Details. **States** Standard; the add action requires a selected trip and prompts to create one if none exists.

24.10 Activity Details

Purpose Convert interest into a bookable selection. **Components** Gallery, provider name and rating, duration, price per person, inclusions and exclusions, requirements, meeting point with map, cancellation policy summary, date picker showing the next 30 days with availability, departure list with remaining seats, pax stepper, Add to Trip. **Validation** `min_pax ≤ pax ≤ max_pax`; departure not past its cutoff; age requirements checked against the party. **API** `GET /activities/{id}`, `GET /activities/{id}/departures`. **Navigation** → Trip Planner. **States** Empty departures shows the next available date; sold-out departures render disabled with a "full" label; error shows a retry on the departures panel only.

24.11 Accommodation Search

Purpose Find a stay for the trip dates. **Components** Destination, check-in/check-out, guests and rooms selector, filters (price band, property type, star rating, amenities), sort control, map/list toggle, result cards with the total price for the stay. **Validation** Check-out after check-in; stay ≤ 30 nights; dates within the trip range when launched from a trip. **API** `GET /accommodations`. **Navigation** → Accommodation Details. **States** Skeleton cards; a no-results state offering to widen dates or relax filters; error with retry. Prices displayed are totals for the whole stay, with the nightly average shown beneath, so comparison is honest.

24.12 Accommodation Details

Purpose Evaluate a property. **Components** Gallery, description, amenity grid, map, house rules, check-in/check-out times, cancellation policy, reviews summary with the rating histogram, room-type list. **API** `GET /accommodations/{id}`, `GET /reviews?subject_type=accommodation&subject_id=…`. **Navigation** → Room Selection. **States** Standard; fewer than three reviews shows "New on the platform" rather than a misleading average.

24.13 Room Selection

Purpose Choose a room type and quantity. **Components** Room-type cards with occupancy, bed configuration, amenities, per-night rate, stay total, remaining availability indicator, quantity stepper, policy note, Add to Trip. **Validation** Party fits the occupancy; requested rooms ≤ available; minimum-nights satisfied. **API** `GET /accommodations/{id}/room-types?check_in=&check_out=&adults=&children=&rooms=`. **Navigation** → Trip Planner. **States** Unavailable room types are shown greyed with the reason ("not available for 12 Aug"), which is more useful than hiding them.

24.14 Trip Planner

Purpose The workspace where the journey is assembled. **Components** Trip header (destination, dates, party, editable), day tab strip, per-day timeline of items with times, durations, prices and drag handles, an add-item action sheet (stay, activity, attraction, transfer, free time), a validation banner summarising errors and warnings, a running total footer with a Continue action. **Actions** Add, edit, reorder, remove items; edit trip dates or party; regenerate; open validation findings; continue to summary. **Validation** Client-side date and party checks; the authoritative rule set is server-side (Section 10.6) and findings are rendered inline against the offending items. **API** `POST/PATCH/DELETE /trips/{id}/items`, `POST /trips/{id}/itinerary/generate`, `GET /trips/{id}`. **Navigation** → any detail screen, → Flight Information, → Transportation, → Trip Summary. **States** Loading uses an optimistic update with rollback on failure; empty shows a guided three-step prompt (dates → stay → activities); locked items render with a padlock and are non-draggable; error keeps the last good itinerary on screen.

24.15 Flight Information

Purpose Capture arrival and departure details so transfers can be timed. **Components** Two panels (inbound, outbound), each with airline, flight number, date, scheduled time, gateway picker, passenger count, luggage count; an explanatory note about the 45-minute arrival buffer. **Validation** Flight number pattern `^[A-Z0-9]{2}\s?\d{1,4}$`; arrival within the trip range; departure after arrival; passengers equal to the trip party unless explicitly overridden. **API** `PUT /trips/{id}/flights`. **Navigation** → Airport Pickup. **States** Straightforward form states; a warning appears if the arrival time leaves less than the configured buffer before a first-day activity.

24.16 Airport Pickup

Purpose Book the arrival transfer. **Components** Origin (locked to the gateway), destination selector defaulting to the booked accommodation, computed pickup time with the buffer explained, pax and luggage, vehicle-class cards with capacity and price, meeting-point description with photograph, Add to Trip. **Validation** Class capacity ≥ pax and luggage; pickup time after arrival plus buffer. **API** `POST /transport/quotes`, `POST /trips/{id}/items`. **Navigation** → Trip Planner. **States** `NO_TARIFF_CONFIGURED` renders as "transfers to this location are not yet available — contact support"; routing unavailability shows a retry rather than a guessed price.

24.17 Transportation

Purpose Manage all non-airport transfer legs. **Components** List of legs derived from the itinerary with origin, destination, time, distance, duration and price; per-leg vehicle-class selection; an add-custom-leg action with map pickers; a note where a leg was auto-inserted by the planner. **Validation** Origin ≠ destination; time consistent with adjacent items; class capacity. **API** `POST /transport/quotes`, item mutations. **Navigation** → Trip Planner. **States** Legs with approximate estimates carry an explicit "approximate" badge (Section 12.6).

24.18 Driver Assignment

Purpose Show who is coming, once known. **Components** Per-transfer card with assignment status, driver photo, first name, rating, languages, vehicle make/model/colour/plate, seat and luggage capacity, pickup time and place, pickup PIN, message and support actions. **Data** Available only after `ASSIGNED`; before that the card shows "we will confirm your driver by 〈date〉". **API** `GET /bookings/{id}`. **Navigation** → Messages, → Driver Tracking on the day. **States** Pre-assignment state is explicit rather than blank; `UNFULFILLED` shows the support escalation with a direct contact action.

24.19 Itinerary

Purpose The day-by-day journey view, and the primary offline artefact. **Components** Day tabs, chronological cards with icons per item type, times in destination-local time, travel legs shown as connectors with distance and duration, per-item status chips, download-PDF and share actions, an offline badge when served from cache. **API** `GET /trips/{id}/itinerary`. **Navigation** → item details, → Active Trip. **States** Fully available offline once the trip is confirmed; while a trip is a draft, the itinerary reflects the last generate.

24.20 Trip Summary

Purpose The final review before payment — the screen the entire product exists to produce. **Components** Trip header; arrival block; per-day summary; a per-service-type cost table; the totals block; the validation state; policy summaries for every component; a prominent Continue to Payment action; a quote-expiry countdown once quoted. **Data** Everything in Section 3.3's example artefact. **API** `GET /trips/{id}/summary`, `POST /trips/{id}/quote`. **Navigation** → Cost Breakdown, → Payment, back → Trip Planner. **States** Blocking errors disable Continue and link to the offending item; `409 INVENTORY_UNAVAILABLE` renders per-item with alternatives; `409 PRICE_CHANGED` renders an old/new comparison requiring explicit acceptance; the countdown warns at 5 minutes and, on expiry, offers to re-quote.

24.21 Cost Breakdown

Purpose Complete transparency on what is being charged. **Components** Itemised lines grouped by service type with quantity, unit price and line total; subtotal; platform service fee with a plain-language explanation; taxes; total; a separate clearly-labelled block for estimated on-site costs not collected by the Platform (attraction entrance fees); the currency and, where converted, the source currency and rate. **API** Data from the trip payload; no separate call. **States** Static; every figure traceable to a line item. Commission is never shown here.

24.22 Payment

Purpose Take payment. **Components** Method selector (card, mobile money) filtered by the tourist's country and currency; PSP-hosted card field or redirect; mobile-money number entry with network detection; amount

confirmation; terms acknowledgement; Pay action; a hold-expiry countdown. **Validation** Method supported for the currency; mobile number in E.164 with a supported network prefix; the amount is displayed, never entered. **API** `POST /payments/intents`, `GET /payments/{id}`. **Navigation** → Booking Confirmation on success; back is blocked while a payment is in flight. **States** In-flight state is explicit and non-dismissible with a spinner and a "do not close the app" message; failure maps the normalised code (Section 21.8) to a specific, actionable message with a retry; mobile-money waiting shows a countdown with a resend option; expiry of the payment window returns the tourist to the summary with holds released and a clear explanation.

24.23 Booking Confirmation

Purpose Close the loop with certainty. **Components** Success state, trip reference, per-component confirmation list with references, what-happens-next block (driver confirmation timing, reminder schedule), add-to-calendar, download-itinerary, share, Go to My Trips. **API** `GET /trips/{id}`. **States** Partial success (one component `AWAITING_PROVIDER`) is shown honestly with the expected response deadline; the automatic download of the offline pack happens here.

24.24 My Trips

Purpose All trips in one place. **Components** Segmented control (Upcoming, Active, Past, Drafts); trip cards with destination image, dates, status chip, total and next-item preview; pull-to-refresh. **API** `GET /trips?status=`. **Navigation** → Trip detail / Active Trip / Trip Planner (drafts). **States** Per-segment empty states with an appropriate call to action; drafts show their expiry.

24.25 Active Trip

Purpose The in-destination companion. **Components** "Now and next" card, today's timeline, live driver status when a transfer is in progress, quick actions (message provider, call support, view voucher, get directions), a day switcher, an emergency-contact block. **API** `GET /trips/{id}`, WebSocket `trip.{public_id}`. **Navigation** → Driver Tracking, Messages, Itinerary. **States** Fully functional offline for display; live elements degrade to their last known values with a timestamp.

24.26 Driver Tracking

Purpose Remove arrival anxiety. **Components** Map with the driver marker, pickup marker and route; status banner (assigned / en route / arrived); ETA; driver and vehicle card; pickup PIN; message and call-support actions; a share-my-status action that sends a read-only link to a chosen contact. **API** WebSocket `trip.{public_id}`, fallback `GET /bookings/{id}/driver-position`. **Navigation** ← Active Trip. **States** Tracking is available only while the assignment is `EN_ROUTE`, `ARRIVED` or `STARTED`; before that the screen states when tracking will begin; a stale position (> 2 minutes) is labelled with its age rather than silently shown as current.

24.27 Messages

Purpose Booking-scoped communication. **Components** Conversation list with counterparty, booking context and unread badges; thread view with bubbles, timestamps, read receipts, image attachment, report action. **Validation** Message ≤ 2,000 characters; images ≤ 5 MB; contact details are masked with an explanatory note. **API** `GET /conversations`, `GET/POST /conversations/{id}/messages`. **States** Queued messages when offline are marked pending; closed conversations are read-only with an explanation and a support link.

24.28 Notifications

Purpose In-app inbox. **Components** Grouped list (today, earlier), read/unread styling, deep links, mark-all-read, a link to preferences. **API** `GET /notifications`, `POST /notifications/{id}/read`. **States** Empty state; infinite scroll with cursor pagination.

24.29 Reviews

Purpose Collect and display feedback. **Components** Pending-review prompts for completed bookings; a review form with an overall star rating, category ratings, a comment field and a photograph upload; the tourist's own review history with provider responses. **Validation** Overall rating required; comment ≤ 2,000 characters; the review window must be open. **API** `POST /reviews`, `GET /reviews?author=me`. **States** Submitted reviews awaiting moderation are labelled as such; the window-closed state explains why the form is unavailable.

24.30 Profile

Purpose Manage identity and traveller details. **Components** Avatar, name, email with verification state, phone with verification state, nationality, date of birth, emergency contact, travel-document field shown only when a booked service requires it with an explanation of why. **Validation** Standard formats; changing email or phone triggers re-verification. **API** `GET/PATCH /me`. **States** Verification-pending banners; a save-failure state that preserves the entered values.

24.31 Settings

Purpose Control the experience. **Components** Language, presentment currency, notification preferences per event class with the non-disableable items shown as locked with an explanation, location-permission status with a link to system settings, appearance, privacy controls (download my data, delete my account), legal documents, app version and build. **API** `GET/PATCH /me/notification-preferences`, `DELETE /me`. **States** Account deletion requires typed confirmation and explains the retention consequences from Section 31.7.

24.32 Help and Support

Purpose Resolve problems without an AI assistant. **Components** Searchable help-article categories; contextual articles based on the current trip state; a structured contact form with category, related booking selector and description; ticket list with status; an emergency contact block with the local support number, shown prominently and available offline. **API** `GET /help/articles`, `POST /support/tickets`, `GET /support/tickets`. **States** The emergency contact block is rendered from cached data and never depends on connectivity, because it is needed exactly when connectivity has failed.

25 Driver Application — Screen Specifications

The driver app is optimised for one-handed use, glanceability while stationary, and reliability on poor connectivity. It uses larger type, higher contrast and fewer options per screen than the tourist app.

25.1 Splash

Resolves session, verification status and document validity. **API** `GET /config`, `GET /me`. **Navigation** → Login / Verification / Dashboard. A driver whose documents have expired is routed to Verification with a blocking explanation rather than to the Dashboard.

25.2 Login

Phone number (E.164 with country picker) plus password, with an OTP step. **Validation** Phone format; OTP six digits, 5-minute validity, three attempts. **API** `POST /auth/login`, OTP verify. **States** Lockout is communicated with the exact unlock time; a suspended account shows the compliance contact.

25.3 Registration

Captures name, phone, email, language selection and the transport provider affiliation (or self-registration as an independent operator, which creates a single-driver provider record). **Validation** Unique phone and email; provider invitation code where applicable. **API** `POST /auth/register` (driver variant). **Navigation** → Verification.

25.4 Verification

Purpose Collect and track the documents required to work. **Components** Checklist with per-item status (national identity document, driving licence with expiry, driver photograph, vehicle registration, insurance certificate with expiry, inspection certificate with expiry); camera or file upload per item; rejection reasons displayed inline; an overall status banner. **Validation** Image quality floor, file ≤ 10 MB, expiry dates in the future. **API** `POST /provider/documents`, `GET /me`. **States** `SUBMITTED` shows the expected review turnaround; `REJECTED` shows the specific reason and a re-upload action; an approaching expiry (30 days) shows a persistent warning because it will block dispatch.

25.5 Dashboard

Purpose The driver's home. **Components** Online/offline toggle with an explicit blocked state when documents are invalid; today's earnings; today's assignment list with times and pickup points; next assignment countdown; acceptance-rate and rating tiles; announcements. **API** `GET /driver/assignments?date=today`, `GET /driver/earnings?period=today`. **States** Offline state is unmistakable; the blocked state names the exact reason (for example "insurance expired on 3 Aug").

25.6 Online/Offline Status

A single prominent control with confirmation, plus an optional service-area selector. Going online requires verification, valid documents, at least one verified active vehicle and location permission. **API** `POST /driver/availability`. **States** Each precondition failure is reported individually with a fix action.

25.7 Incoming Booking Offer

Purpose Present an offer and capture a decision within the TTL. **Components** Full-screen, sound and vibration; countdown ring; pickup point and distance from the driver; drop-off; scheduled pickup time; passenger and luggage count; flight number where it is an airport transfer; the driver's net earning for the job; Accept and Decline. **Validation** Server re-checks eligibility on accept; a lost race returns `409 OFFER_TAKEN`. **API** `POST /driver/offers/{id}/accept|decline`. **States** Expiry closes the screen with a neutral message; declining asks for an optional reason; the earning shown is net of commission, because a gross figure would be misleading.

25.8 Booking Details

Full detail for an accepted assignment: pickup and drop-off with map, scheduled time, tourist first name and masked contact, flight information, luggage, special notes, the cancellation policy that applies to the driver, and the net fare. **API** `GET /driver/assignments/{id}`.

25.9 Navigation

Hands off to the device's installed navigation application with the destination pre-filled; the Platform provides no turn-by-turn guidance in V1. The screen retains a compact status bar so the driver can progress the trip without returning to the app's root.

25.10 Pickup

Purpose Manage the arrival and meeting. **Components** Arrived action (geofence-checked), tourist first name, meeting-point description and photo, PIN entry, wait timer with the policy threshold, message and call-support actions, and a No-Show action that unlocks only after the configured wait. **API** `POST /driver/assignments/{id}/status`. **States** A geofence violation prompts for an override reason rather than blocking; the no-show action shows exactly how much longer the driver must wait.

25.11 Active Trip

Shows the route, elapsed time and distance, drop-off details, and a single primary action to complete. Location collection runs with a persistent foreground notification. An incident action allows the driver to report a breakdown or safety issue, which alerts operations immediately.

25.12 Trip Completion

Confirms completion inside the drop-off geofence, shows the fare breakdown (gross, commission, net), offers an optional note, and returns to the Dashboard. **API** `POST /driver/assignments/{id}/status` with `COMPLETED`.

25.13 Earnings

Daily, weekly and monthly summaries; a line item per completed assignment with gross, commission and net; pending versus available balance with expected availability dates; payout history. **API** `GET /driver/earnings`, `GET /provider/payouts`.

25.14 Trip History

Completed and cancelled assignments with filters by date and status, each opening the read-only detail and the tourist's rating where published.

25.15 Ratings

Current average, total count, a per-star histogram, and recent published comments. Ratings older than 12 months are excluded from the displayed average, matching the aggregation rule, and this is stated on the screen.

25.16 Profile

Personal details, languages spoken, photograph, and document status with expiry dates.

25.17 Vehicle Management

List of the driver's vehicles with class, capacity, plate, document expiry and active flag; add and edit flows that route new or changed vehicles back through verification.

25.18 Notifications

Inbox mirroring the tourist app, with offers, assignment changes, document expiry warnings and payout confirmations.

25.19 Support

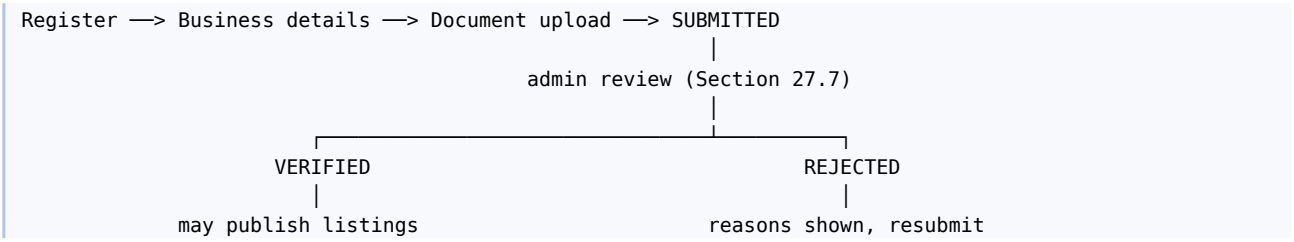
Help articles targeted at drivers, a ticket form, and a prominent emergency contact available offline.

26 Service Provider Portal

26.1 Purpose and Platform

A responsive React web application serving accommodation providers, activity providers and transport operators. It is deliberately web-based: providers work from a desk or a phone browser, and requiring an app install would suppress supply onboarding.

26.2 Onboarding and Verification



Documents required by provider type: business registration certificate and tax identification for all; tourism operating licence for activity providers; property licence for accommodation; vehicle and driver documentation for transport (handled per driver and vehicle). Every document carries an expiry where applicable, and an expiring document generates a `DOCUMENT_EXPIRING` notification at 30 days and blocks new bookings on expiry.

26.3 Dashboard

Today's arrivals and departures or departures-by-activity; pending action items (on-request bookings awaiting response, unanswered reviews, expiring documents); occupancy or capacity utilisation for the next 30 days; revenue for the current and previous period; recent bookings; payout status.

26.4 Listing Management

Provider type	Capabilities
Accommodation	Create and edit properties, room types, photographs (ordered, with a primary), amenities, house rules, check-in/check-out times, cancellation policy selection, activate/deactivate
Activity	Create and edit activities, descriptions, inclusions/exclusions, requirements, meeting point with map pin, duration, price per person and per group, min/max party, booking cutoff, confirmation mode, cancellation policy, photographs
Transport	Manage drivers and vehicles, documents, vehicle classes and service areas; transfer pricing is platform-managed (Section 12.4) and is displayed read-only with the applicable tariff

All listing edits by a verified provider publish immediately except changes to price, cancellation policy or capacity, which take effect only for new bookings — never for existing ones.

26.5 Availability and Rate Calendar

A month-grid calendar per room type or activity showing, per date, availability, held, sold and rate. Bulk editing supports a date range, a weekday mask, and set-availability / set-rate / open / close operations in one submission. A conflict (attempting to reduce availability below what is already sold or held) is rejected with the specific dates named. **API** `PUT /provider/room-types/{id}/availability`, `PUT /provider/activities/{id}/departures`.

26.6 Booking Management

List and detail views with filters by status, date range and listing; guest name (first name and last initial only until 24 hours before service, then full name), party size, special requests, and the booking's cancellation policy. Actions: accept or decline on-request bookings within the response window; cancel with a reason (which triggers a full tourist refund under BR-045 and is recorded against the provider's reliability metrics); mark no-show with evidence; message the tourist.

26.7 Earnings, Statements and Payouts

Balance summary (pending and available with dates), a transaction list per booking showing gross, commission rate, commission amount and net, payout history with line items, and downloadable CSV/PDF statements per period. The commission rate applied to each booking is displayed, because a provider is entitled to see exactly what was deducted and why.

26.8 Reviews

Published reviews with ratings and comments, the rating histogram and trend, and a single-response affordance per review. Flagging a review for moderation with a reason opens a moderation ticket.

26.9 Reports

Bookings by period, revenue by listing, occupancy or utilisation, cancellation rate, lead time distribution, and rating trend — all exportable. These are database aggregations, not predictive analytics.

26.10 Staff and Access

The provider owner may invite staff with the `PROVIDER_STAFF` role, which excludes payout and banking access. All staff actions are attributed in the audit log.

26.11 Support

Help centre for providers, a ticket form, and the platform partnership contact.

27 Administration Console

27.1 Purpose and Access Control

A React web application, reachable only from allow-listed networks, requiring mandatory TOTP, with every action written to `audit_log`. Roles are as defined in Section 5.2; the interface hides functions the role cannot perform and the API enforces the same rules independently.

27.2 Dashboard

Live operational KPIs: trips created, quoted and confirmed today; payment success rate; bookings by status; transfers awaiting driver assignment with time-to-deadline; unfulfilled assignments; on-request bookings nearing their response deadline; open P1/P2 tickets; payment reconciliation exceptions; system health indicators.

27.3 User Management

Search across tourists, drivers and provider staff by name, email, phone or reference. Per user: profile, trips, bookings, payments, devices, login history, notification history. Actions: suspend, reactivate, force password reset, resend verification, initiate account closure, and export the user's data for a privacy request. Every action requires a reason.

27.4 Tourist Management

Trip and booking history, lifetime value, cancellation and no-show counts, support ticket history, and the ability to issue a goodwill credit within the role's cap.

27.5 Driver Management

Verification queue and decisions, document review with expiry tracking, online-status history, assignment history with acceptance and completion rates, ratings, incident reports, suspension and reinstatement.

27.6 Vehicle Management

Registry with class, capacity, document expiry and verification state; approve, reject or deactivate; a report of vehicles blocked from dispatch and the reason.

27.7 Provider Management and Verification

The verification workflow queue with document viewer, a decision panel (approve, reject with coded reason, request more information), commission-rule assignment, payout-account status, listing counts, performance metrics (cancellation rate, response time, rating), and suspension controls.

27.8 Catalogue Management

Full CRUD over countries, regions, destinations, attractions and — for administrative correction — activities and accommodation. Destination management includes the centroid map pin, gateway flags, timezone, currency, launch date and activation. **This screen is the mechanism by which the Platform expands to a new market**; the acceptance test in Section 41.12 is executed here.

27.9 Trip and Booking Management

Operational search across all trips and bookings by reference, tourist, provider, date or status; a full timeline view of a trip including status history, payments, assignments, notifications sent and messages exchanged; and exceptional controls — force a status transition, reassign a driver, extend a hold, or re-issue a voucher — each requiring a reason and each audited.

27.10 Payment and Refund Management

Payment search, transaction detail with PSP references and raw webhook payloads, manual verification against the PSP, refund initiation with policy-computed and override amounts, refund approval queue, chargeback register, and the daily reconciliation report with an exception worklist.

27.11 Commission and Tariff Management

CRUD over commission rules with scope, method, values, clamps, priority and effective dates; a preview tool that shows the rule that would resolve for a given provider, listing and booking type; CRUD over transfer corridors and tariffs, with a quote-preview tool for any origin-destination-class combination; platform service fee and tax rules.

27.12 Cancellation Policy Management

CRUD over policies expressed as ordered `{hours_before, refund_percent}` tiers, with a preview calculator, and a listing of which properties and activities use each policy.

27.13 Review and Complaint Moderation

Moderation queue with the flagged reason, publish/reject/redact decisions, provider response moderation, and abuse-pattern reports based on the deterministic flags of Section 19.8.

27.14 Notification Management

Template editing per event, channel and locale with a live preview and a send-test action; delivery statistics by event and channel; failed-delivery worklist; and the ability to disable a non-critical event class globally during an incident.

27.15 Reports and Analytics

The catalogue in Section 40, with parameterised date ranges, on-screen rendering and CSV/XLSX export. Long-running reports are generated asynchronously and delivered as a download link.

27.16 System Configuration

Editable `system_setting` keys grouped by domain (buffers, TTLs, dispatch weights, thresholds, fee rates, limits), each with its type, current value, default, description and last-changed audit entry. Feature flags with environment scoping. Changing a setting takes effect within the cache TTL (60 seconds) without a deployment.

27.17 Audit Log

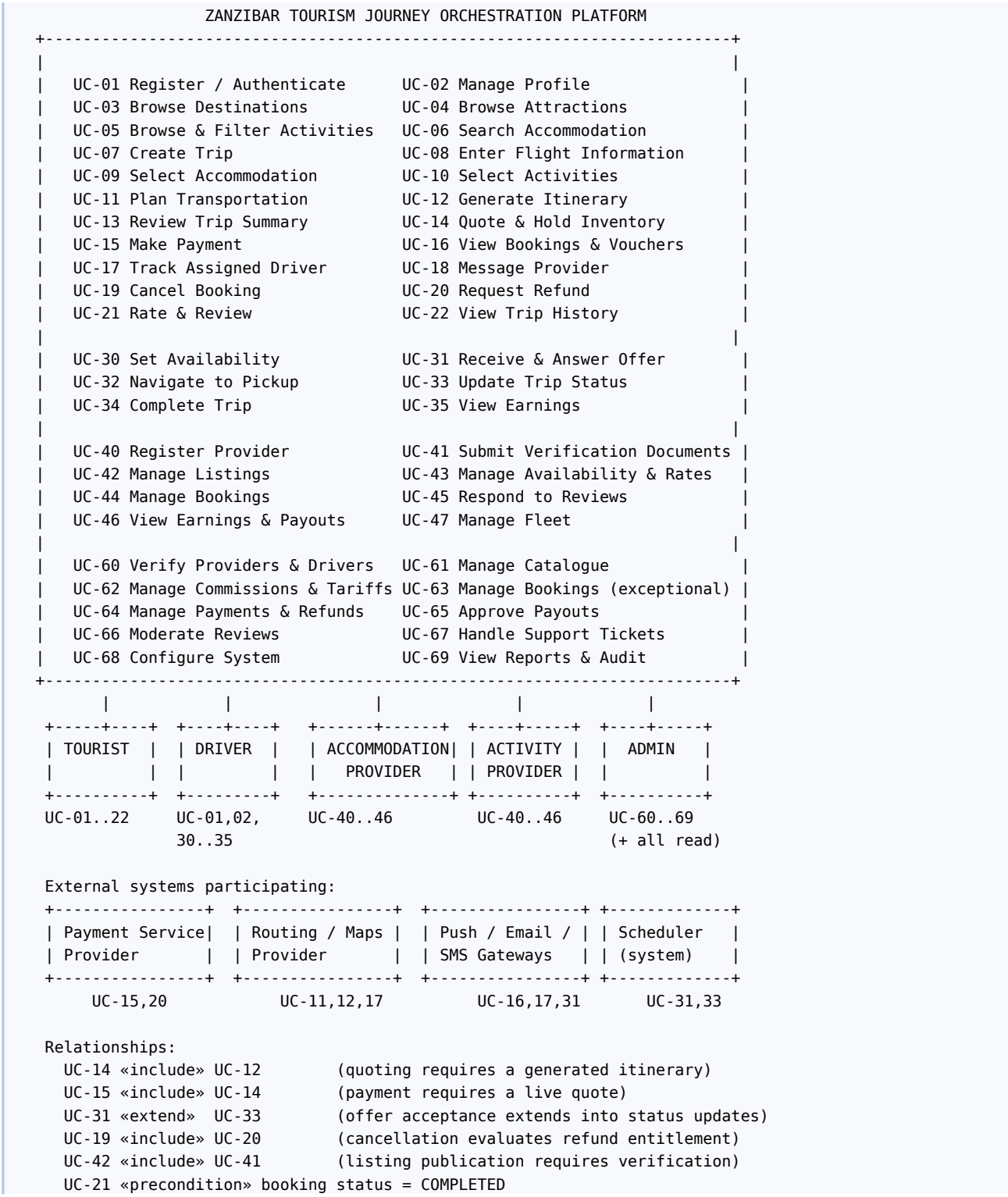
Searchable by actor, action, entity type, entity id, date range and IP; a before/after diff viewer; and export for investigations. The audit log is read-only in the interface and at the database role level for every account including administrators.

27.18 Support Ticketing

Queue with category, priority, SLA timers, assignment, internal notes, linked booking or trip, and resolution codes. Tickets are created by users through the apps and portal, and automatically by the system for operational exceptions (unfulfilled assignment, reconciliation mismatch, driver no-show).

28 System Models

28.1 Use Case Diagram



28.2 Sequence Diagrams

Notation: → synchronous call, --> response, ⇢ asynchronous/event, [ ] guard.

28.2.1 SD-01 Tourist Registration

```
Tourist   App        API          DB         EmailSvc
  |        |          |            |             |
  |--fill->|          |            |             |
  |        |--POST /auth/register->|             |
  |        |          |--check email unique----->|
  |        |          |<--not found--------------|
  |        |          |--INSERT user(PENDING)--->|
  |        |          |--INSERT tourist_profile->|
  |        |          |--INSERT audit_log------->|
  |        |          |⇢ queue verification email|
  |        |<--201 {user, verification_required}-|
  |<-shown-|          |                          |
  |        |          |                    (worker)--send-->|
  |<================= verification email ================  |
  |--tap link------------------------------------>|          |
  |        |--POST /auth/verify-email->|         |          |
  |        |          |--UPDATE user SET status=ACTIVE------>|
  |        |<--200--------------------|          |          |
```

28.2.2 SD-02 Login

```
Tourist   App          API              DB          Redis
  |--creds->|           |               |            |
  |         |--POST /auth/login--------->|            |
  |         |           |--SELECT user by email------>|
  |         |           |<--user---------|            |
  |         |           |--verify Argon2 hash (local) |
  |         |           |[fail] increment failed_login_count, maybe lock
  |         |           |[ok]  reset counter, set last_login_at
  |         |           |--store refresh jti--------------------->|
  |         |<--200 {access, refresh, principal}-----|            |
  |<--home--|           |               |            |
```

28.2.3 SD-03 Trip Creation

```
Tourist  App        API                DB
  |--new trip->|     |                  |
  |        |--POST /trips {destination, dates, party}->|
  |        |      |--validate: destination active, dates future,
  |        |      |            duration <= max_trip_days, adults >= 1
  |        |      |--INSERT trip(DRAFT), reference generated--->|
  |        |      |--INSERT itinerary(v1, empty)--------------->|
  |        |      |--build day skeleton (day 1..N)------------->|
  |        |<--201 {trip}-------------|
  |<-planner-|    |                   |
```

28.2.4 SD-04 Trip Planning and Itinerary Generation

```
Tourist  App       API           Catalogue   RouteCache   RoutingProvider
  |--add stay->|    |                |           |              |
  |        |--POST /trips/{id}/items->|           |              |
  |        |    |--validate occupancy, dates------|              |
  |        |    |--INSERT itinerary_item(STAY) x nights          |
  |        |<--201-|                |           |              |
  |--add activity->|                |           |              |
  |        |--POST /trips/{id}/items->|           |              |
  |        |    |--check departure open, cutoff, pax bounds------|
  |        |    |--INSERT itinerary_item(ACTIVITY)               |
  |        |<--201-|                |           |              |
  |--generate->|   |                |           |              |
  |        |--POST /trips/{id}/itinerary/generate->|              |
  |        |    |--load items, sort per day (Section 10.4)       |
  |        |    |--for each gap: need transfer?                  |
  |        |    |--------- lookup route ---------->|             |
  |        |    |<-------- hit / miss -------------|             |
  |        |    |[miss]--------- route() ------------------------>|
  |        |    |<-------------- distance, duration, polyline ----|
  |        |    |--------- store in cache -------->|             |
  |        |    |--INSERT TRANSFER items with computed times     |
  |        |    |--run validation rule set VR-01..VR-17          |
  |        |    |--compute line totals, fee, tax, total          |
  |        |    |--UPDATE trip, itinerary.version += 1           |
  |        |<--200 {itinerary, findings, totals}--|              |
  |<-render-|   |                |           |              |
```

28.2.5 SD-05 Accommodation Booking (search to hold)

```
Tourist  App      API            DB(room_availability)
  |--search->|     |                    |
  |       |--GET /accommodations?…------>|
  |       |     |--filter destination, dates, occupancy
  |       |     |--indicative sellable per night
  |       |<--200 [results]--------------|
  |--select room->|                      |
  |       |--GET /accommodations/{id}/room-types?…-->|
  |       |<--200 [room types, nightly, total]-------|
  |--add to trip->|                      |
  |       |--POST /trips/{id}/items(STAY)|
  |       |<--201-----------------------|
  |--quote->|     |                      |
  |       |--POST /trips/{id}/quote---->|
  |       |     |BEGIN                   |
  |       |     |--SELECT … FOR UPDATE (nights, PK order)-->|
  |       |     |--assert sellable >= rooms for each night  |
  |       |     |--UPDATE rooms_held += rooms               |
  |       |     |--INSERT inventory_hold(HELD, TTL 20 min)  |
  |       |     |--UPDATE trip(PRICED, quote_expires_at)    |
  |       |     |COMMIT                  |
  |       |<--200 {totals, quote_token, expires_at}--|
```

28.2.6 SD-06 Activity Booking

```
Tourist  App      API          DB(activity_departure)   Provider
  |--pick date->|  |                    |                   |
  |       |--GET /activities/{id}/departures?from&to&pax-->|
  |       |<--200 [departures, remaining]------------------|
  |--add->|       |                    |                   |
  |       |--POST /trips/{id}/items(ACTIVITY)------------->|
  |       |     |--assert status OPEN, now <= departs_at - cutoff,
  |       |     |         min_pax <= pax <= max_pax, age rules
  |       |<--201-|                    |                   |
  |--quote-> (as SD-05, locking activity_departure, capacity_held += pax)
  |
  |--pay--> (SD-09)
  |                    [confirmation_mode = ON_REQUEST]
  |       |     |--booking -> AWAITING_PROVIDER ------------>| notify
  |       |     |<-- provider accepts within N hours --------|
  |       |     |--booking -> CONFIRMED                      |
  |       |     |[timeout]--booking -> CANCELLED + full refund
```

28.2.7 SD-07 Transportation Booking

```
Tourist  App      API        Transport      Location      DB
  |--legs->|       |            |             |           |
  |       |--POST /transport/quotes---------->|            |
  |       |     |--resolve endpoints to coordinates-------->|
  |       |     |--------- route/matrix ------>|             |
  |       |     |<-------- distance, duration -|             |
  |       |     |--resolve tariff (corridor -> tariff -> fail)
  |       |     |--price per eligible vehicle class          |
  |       |<--200 {legs[].options[]}---------|             |
  |--select class->|                          |             |
  |       |--POST /trips/{id}/items(TRANSFER)-------------->|
  |       |<--201-|                          |             |
```

28.2.8 SD-08 Driver Assignment

```
Booking  Transport   DB          Driver App(s)      Tourist App
  |↠ BookingConfirmed(TRANSFER)   |                    |
  |         |--build candidate list (eligibility + score)|
  |         |--INSERT driver_offer(candidate[0], TTL)--->|
  |         |------------- push + WS: offer.new -------->|
  |         |                     |   [driver accepts]   |
  |         |<-- POST /driver/offers/{id}/accept --------|
  |         |--SELECT assignment FOR UPDATE              |
  |         |--assert offer live, no overlap (EXCLUDE)   |
  |         |--INSERT/UPDATE driver_assignment(ASSIGNED) |
  |         |--UPDATE booking (driver bound)             |
  |         |↠ DriverAssigned                            |
  |         |------------- push: assignment confirmed -->|
  |         |------------------------ push: DRIVER_ASSIGNED ------->|
  |         |   [decline or TTL expiry]                  |
  |         |--offer candidate[i+1] … until exhausted    |
  |         |[exhausted]--create ops ticket (HIGH)       |
```

28.2.9 SD-09 Payment

```
Tourist  App        API          PSP        DB        Booking
  |--confirm->|      |            |          |           |
  |       |--POST /trips/{id}/confirm-------->|          |
  |       |     |--verify quote_token, not expired       |
  |       |     |--INSERT booking x N (PENDING), snapshot policy+commission
  |       |     |--UPDATE trip(PENDING_PAYMENT), extend holds
  |       |<--201 {basket, total}--|          |          |
  |--pay->|     |                  |          |          |
  |       |--POST /payments/intents (Idempotency-Key)---->|
  |       |     |--amount := trip.total (server-side)    |
  |       |     |--INSERT payment(INITIATED)             |
  |       |     |------ create_intent ------->|          |
  |       |     |<----- client_secret --------|          |
  |       |<--201 {action}--------|          |          |
  |--SDK confirm / USSD approve --|---------->|          |
  |       |                       |<-- issuer/MNO result -|
  |       |                       |          |          |
  |       |     |<== webhook payment.succeeded ==========|
  |       |     |--verify signature, dedupe by event id  |
  |       |     |--advisory lock payment                 |
  |       |     |--confirmation routine (Section 20.8)---->| commit holds
  |       |     |                                         | bookings CONFIRMED
  |       |     |-» PaymentCaptured, TripConfirmed, BookingConfirmed×N
  |       |<--- push: PAYMENT_SUCCEEDED, TRIP_CONFIRMED ---|
  |       |--GET /payments/{id} (fallback poll)---------->|
```

28.2.10 SD-10 Airport Pickup (day of travel)

```
Tourist App   API        Driver App     Location   Notify
  |            |             |             |         |
  |            |<-- POST /driver/assignments/{id}/status EN_ROUTE
  |            |--assert legal transition, time window   |
  |            |-» DriverEnRoute --------------------------->| push
  |<-- push: your driver is on the way --------------------|
  |            |<-- POST /driver/location (batch) ---------|
  |            |--validate, store, publish to trip channel |
  |<== WS driver.location (throttled 10 s) ================|
  |            |--geofence 1500 m crossed --------------->| push DRIVER_NEARBY
  |            |<-- status ARRIVED (geofence 300 m) -------|
  |<-- push + SMS: your driver has arrived ---------------|
  |--show PIN->|             |             |         |
  |            |<-- status STARTED (PIN verified) --------|
  |            |--booking -> IN_PROGRESS                  |
```

28.2.11 SD-11 Active Trip and Completion

```
Driver App     API            DB           Finance      Tourist App
   |--status COMPLETED (geofence)-->|         |             |
   |            |--assignment COMPLETED       |             |
   |            |--booking -> COMPLETED       |             |
   |            |-» BookingCompleted          |             |
   |            |------------- accrue --------->|            |
   |            |          ledger: PROVIDER_ACCRUAL, COMMISSION_REVENUE
   |            |          provider_balance.pending += net   |
   |            |-» review invite (after 4 h) --------------->| push
   |<--200 {fare breakdown}---------|         |             |
```

28.2.12 SD-12 Trip Completion (whole trip)

```
Scheduler     API              DB              Notify
   |--hourly sweep---------->|                 |
   |          |--SELECT bookings WHERE ends_at < now() AND status=IN_PROGRESS
   |          |--transition to COMPLETED (per type rules)
   |          |--if all bookings of a trip terminal:
   |          |      trip -> COMPLETED
   |          |→ TripCompleted ---------------->| review invites,
   |          |                                 | trip summary email
```

28.2.13 SD-13 Cancellation

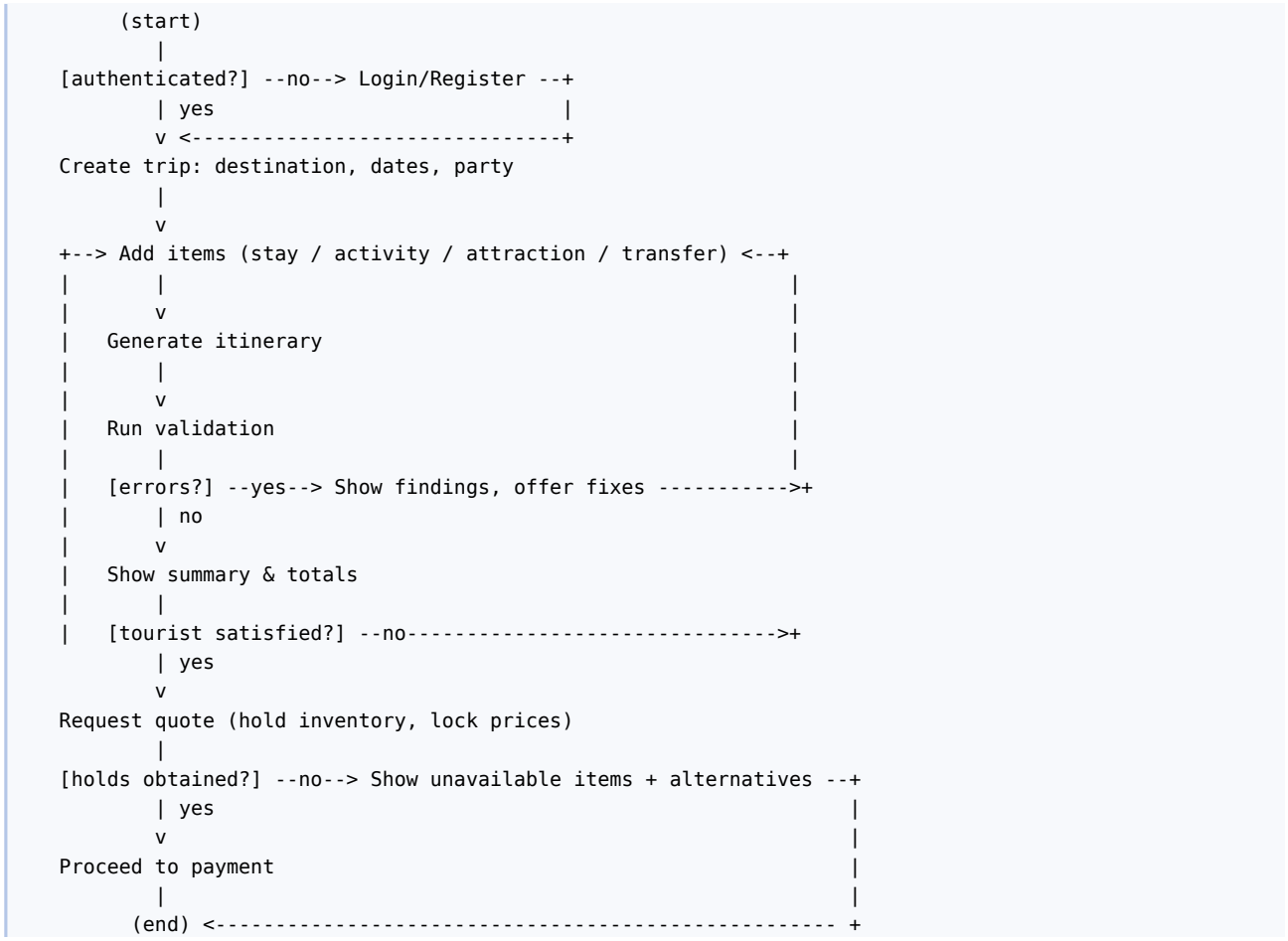
```
Tourist  App       API            Policy       Inventory     Payment
  |--cancel->|      |              |            |             |
  |       |--GET /bookings/{id}/cancellation-preview---------->|
  |       |     |--evaluate policy snapshot ---->|            |
  |       |<--200 {refund_amount, fee_retained}--|            |
  |--confirm->|    |               |            |            |
  |       |--POST /bookings/{id}/cancel--------->|            |
  |       |     |BEGIN                          |            |
  |       |     |--assert status cancellable    |            |
  |       |     |--evaluate policy -> refund_amount          |
  |       |     |--booking -> CANCELLED, history row         |
  |       |     |--release inventory ---------->|            |
  |       |     |--unassign driver if assigned  |            |
  |       |     |COMMIT                         |            |
  |       |     |→ BookingCancelled                          |
  |       |     |[refund_amount > 0] -------------------------> refund flow
  |       |<--200 {status, refund}-|            |            |
```

28.2.14 SD-14 Refund

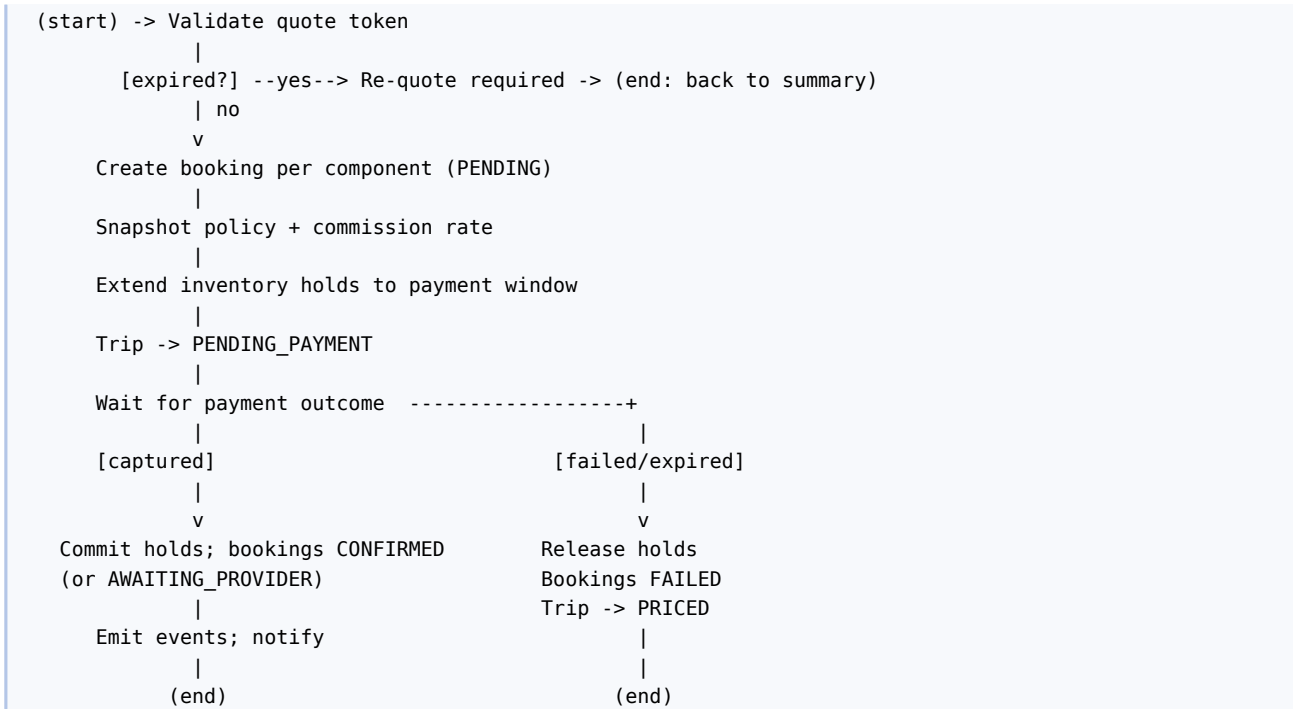
```
System/Admin   API          PSP            DB(ledger)      Tourist
     |          |           |                 |             |
     |--POST /refunds {payment, amount, reason}->|           |
     |          |--assert amount <= capturable remainder     |
     |          |--INSERT refund(PENDING)                    |
     |          |--[discretionary > limit] require FINANCE approval
     |          |------ refund() ------>|                    |
     |          |<----- accepted -------|                    |
     |          |<== webhook refund.succeeded ================|
     |          |--refund -> SETTLED                         |
     |          |--INSERT ledger: REFUND_TO_CUSTOMER,        |
     |          |   PROVIDER_ACCRUAL_REVERSAL, COMMISSION_REVERSAL,
     |          |   CANCELLATION_FEE_ACCRUAL (if any)        |
     |          |→ RefundSettled ---------------------------->| push+email
```

28.3 Activity Diagrams

28.3.1 AD-01 Trip Planning



28.3.2 AD-02 Booking and Confirmation



28.3.3 AD-03 Payment

```
(start) -> Select method
             |
       [method supported for currency?] --no--> Show alternatives
             | yes
             v
     Create payment intent (server-computed amount)
             |
     +-------+--------+
     |                |
  [card]         [mobile money]
     |                |
SDK/3-DS flow    USSD push, wait
     |                |
     +-------+--------+
             |
     Await webhook (with polling fallback)
             |
  [succeeded] --> Confirmation routine --> Notify --> (end)
  [failed]    --> Map failure code --> Offer retry
                       |
                 [3 failures?] --yes--> Throttle 30 min, offer support
                       | no
                       +--> back to method selection
```

28.3.4 AD-04 Driver Assignment

```
(start: transfer booking confirmed)
     |
  Build eligibility-filtered candidate list
     |
  [candidates empty?] --yes--> Ops ticket (HIGH) --> (end)
     | no
     v
  Score and sort candidates
     |
  +--> Offer to next candidate (TTL) <-------------+
  |        |                                       |
  |   [accepted?] --yes--> Create assignment       |
  |        | no                    |               |
  |        v                       v               |
  |   [more candidates AND       Notify tourist    |
  |    before deadline?] --yes--------------------->+
  |        | no                    |
  |        v                      (end)
  |   Mark UNFULFILLED
  |   Ops ticket (HIGH)
  |        |
  |      (end)
```

28.3.5 AD-05 Trip Completion and Settlement

```
(start: service end reached or driver completes)
     |
  Booking -> COMPLETED
     |
  Accrue ledger entries (provider net, platform commission)
     |
  provider_balance.pending += net
     |
  Wait settlement_hold_days
     |
  [dispute raised?] --yes--> Hold balance, open ticket --> (end)
     | no
     v
  Move pending -> available
     |
  Weekly payout batch assembled
     |
  [balance >= minimum?] --no--> Roll forward --> (end)
     | yes
     v
  Finance approval --> Release via payout rail
     |
  Ledger: PAYOUT_SETTLEMENT; notify provider
     |
   (end)
```

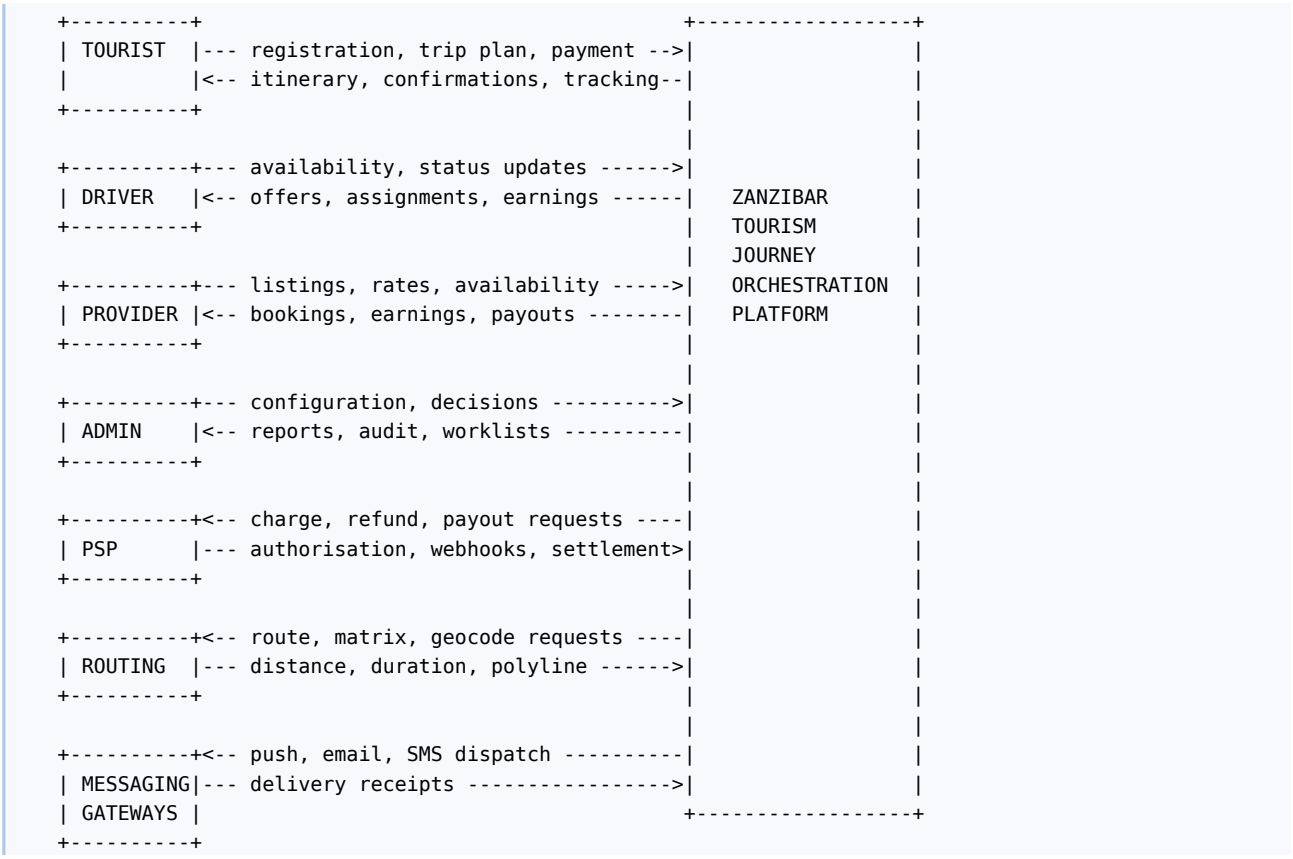
28.4 State Diagrams

The four state machines are specified in full elsewhere and are cross-referenced here so that a reader looking for “the state diagrams” finds them:

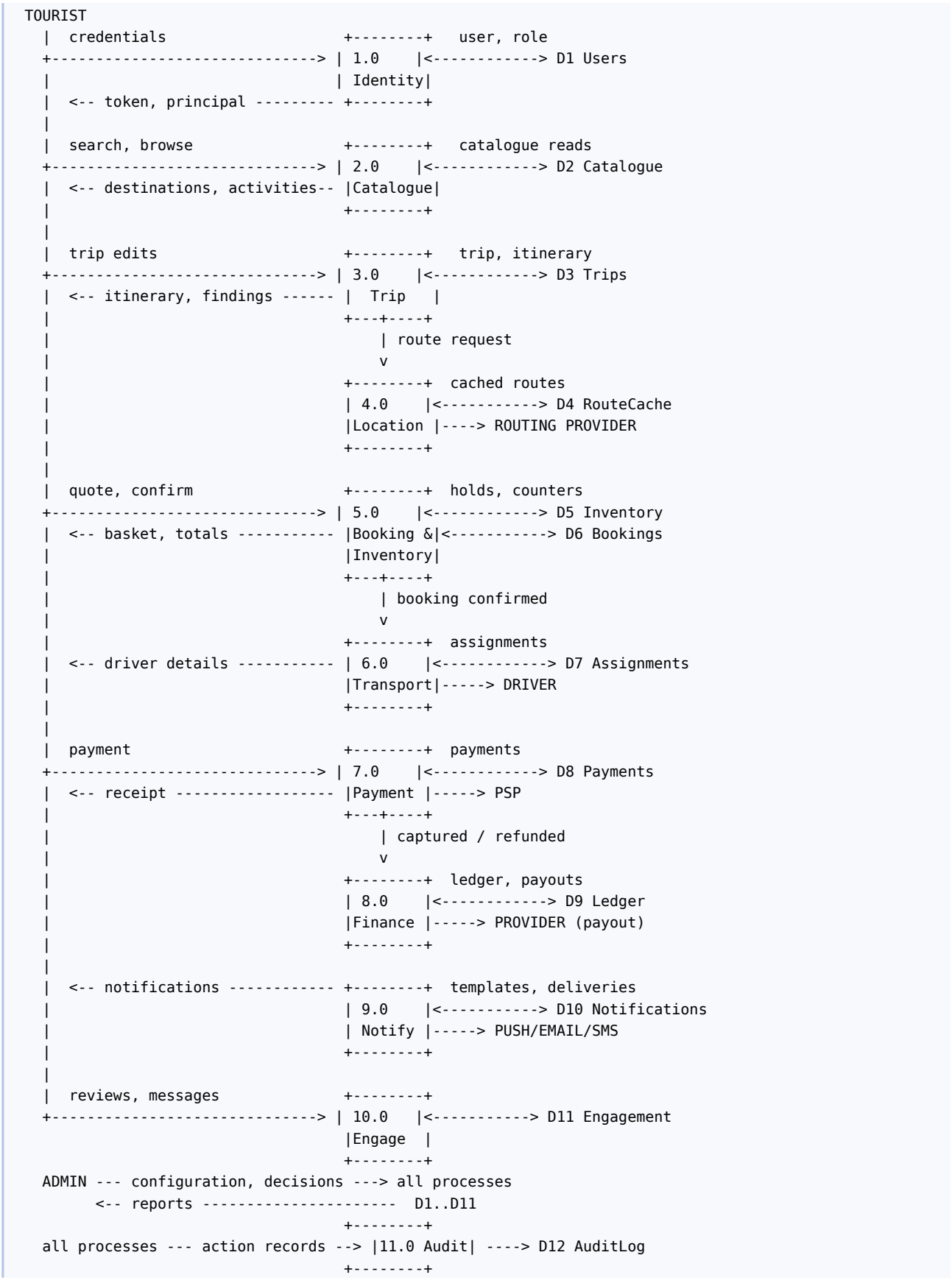
Machine	Location	States
Booking	Section 20.2	DRAFT, PENDING, AWAITING_PROVIDER, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, REFUNDED, NO_SHOW, FAILED
Payment	Section 21.4	INITIATED, PENDING, AUTHORISED, CAPTURED, PARTIALLY_REFUNDED, REFUNDED, FAILED, EXPIRED, DISPUTED, CHARGEBACK_LOST
Trip	Section 20.5	DRAFT, PRICED, PENDING_PAYMENT, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED
Driver assignment	Section 11.7	PENDING, OFFERED, ASSIGNED, EN_ROUTE, ARRIVED, STARTED, COMPLETED, CANCELLED, UNFULFILLED, INCIDENT
Driver availability	Section 20.6	OFFLINE, ONLINE, ENGAGED
Provider/driver verification	Section 26.2	DRAFT, SUBMITTED, UNDER_REVIEW, VERIFIED, REJECTED, SUSPENDED
Inventory hold	Section 17.2	HELD, COMMITTED, RELEASED, EXPIRED

28.5 Data Flow Diagrams

28.5.1 DFD Level 0 — Context



28.5.2 DFD Level 1



28.5.3 DFD Level 2 — Process 5.0, Booking and Inventory

```
   trip items, quote request
        |
        v
  +-------------+  read prices        +----------------+
  | 5.1 Price   |<------------------> | D2 Catalogue   |
  |     lines   |                     +----------------+
  +------+------+
         | priced lines
         v
  +-------------+  lock & decrement   +----------------+
  | 5.2 Place   |<------------------> | D5 Inventory   |
  |     holds   |  hold rows          | (availability, |
  +------+------+                     |  departures,   |
         |                            |  holds)        |
         | quote + TTL                +----------------+
         v
  +-------------+
  | 5.3 Create  |  bookings PENDING   +----------------+
  |     basket  |<------------------> | D6 Bookings    |
  +------+------+                     +----------------+
         |
         | awaiting payment outcome
         v
  +-------------+
  | 5.4 Confirm |  commit holds, status CONFIRMED, history rows
  |  or release |  ----> D5, D6
  +------+------+
         |
         +--> events: BookingConfirmed -> 6.0 Transport, 9.0 Notify
         +--> events: BookingCompleted -> 8.0 Finance
```

28.5.4 DFD Level 2 — Process 7.0, Payment

```
   confirm basket
        |
        v
  +---------------+   amount from D6 (never from client)
  | 7.1 Create    |---------------------------------> D8 Payments
  |     intent    |----> PSP: create_intent
  +-------+-------+
          | client action object
          v
  +---------------+
  | 7.2 Await     |<==== PSP webhook (signed) ====
  |     outcome   |----> D8 payment_webhook_event (raw, deduped)
  +-------+-------+
          |
   +------+-------+
   |              |
[captured]     [failed]
   |              |
   v              v
+---------+   +-----------+
|7.3      |   |7.4 Release|
|Confirm  |   |    holds  |----> D5
|basket   |   +-----------+
+----+----+
     |
     +----> D9 Ledger (SERVICE_FEE_REVENUE, CUSTOMER_PAYMENT)
     +----> 9.0 Notify
```

29 Non-Functional Requirements

Every requirement below is stated so that it can be verified. Unverifiable adjectives (“fast”, “secure”, “user-friendly”) are not requirements and do not appear.

29.1 Performance

ID	Requirement	Verification
NFR-P01	Catalogue read endpoints respond within 400 ms at p95 and 900 ms at p99, measured server-side, at 100 concurrent users	Load test LT-01
NFR-P02	`POST /trips/{id}/itinerary/generate` completes within 2.5 s at p95 for an itinerary of up to 20 items, with all routes served from cache or matrix	Load test LT-02
NFR-P03	`POST /trips/{id}/quote` completes within 1.5 s at p95, including all row locks	Load test LT-03
NFR-P04	`POST /payments/intents` completes within 4 s at p95, including the PSP round trip	Load test LT-04
NFR-P05	Driver offer push is delivered within 5 s of booking confirmation at p95	Integration test
NFR-P06	Driver location updates reach the subscribed tourist client within 3 s at p95	Integration test
NFR-P07	Tourist app cold start ≤ 2.5 s on a reference mid-range Android device (4 GB RAM, Android 11)	Device test
NFR-P08	Any list screen renders its first meaningful content within 1 s on a 3G profile using cached data	Device test
NFR-P09	No API response payload exceeds 250 KB uncompressed; all responses are gzip/brotli compressed	Contract test
NFR-P10	Database: no query in the request path exceeds 200 ms; any such query is either indexed or moved to a background job	`pg_stat_statements` review, CI query budget

29.2 Scalability

ID	Requirement
NFR-S01	The API tier is stateless and scales horizontally; adding an instance requires no configuration change
NFR-S02	The system supports 10,000 registered tourists, 500 providers, 500 drivers and 1,000 concurrent sessions in Year 1 without architectural change
NFR-S03	The system supports a 10× growth on those figures by vertical database scaling plus read replicas, with no schema redesign
NFR-S04	Read-heavy catalogue traffic is served from a read replica; all write paths use the primary
NFR-S05	Background queues are isolated by criticality so that a burst on one queue cannot delay another (Section 8.8)
NFR-S06	Time-series tables (`driver_location`, `audit_log`) are partitioned so that growth does not degrade query performance
NFR-S07	Peak-season load is assumed at 4× the annual mean; capacity plans and autoscaling thresholds are set against that figure

29.3 Availability

ID	Requirement
NFR-A01	Monthly availability target 99.5% for the tourist-facing API, measured by external synthetic checks against a scripted booking journey
NFR-A02	Availability target 99.9% for payment webhook ingestion, which must remain available even during a partial degradation
NFR-A03	Planned maintenance occurs in the 00:00–04:00 EAT window with 72 hours’ notice; zero-downtime deployment is the default
NFR-A04	No single point of failure in the compute tier: at least two API instances across two availability zones
NFR-A05	Database has a standby replica with automated failover; RPO ≤ 5 minutes, RTO ≤ 60 minutes

29.4 Reliability

ID	Requirement
NFR-A06	Failure of the routing provider degrades planning to cached/matrix data and blocks only new priced corridors, never confirmed trips
NFR-A07	Failure of the push provider escalates notifications to email and SMS for the critical event set
NFR-A08	Failure of the PSP prevents new payments but does not affect existing confirmed trips, tracking or messaging
NFR-A09	Every external call has a timeout, a bounded retry policy and a circuit breaker; no unbounded retry exists anywhere
NFR-A10	All mutating operations are idempotent under retry

29.5 Security

Detailed in Section 30. The headline requirements: TLS 1.2+ everywhere; Argon2id password hashing; RBAC plus ownership checks on every resource; mandatory TOTP for staff and providers; PCI DSS SAQ A scope; encryption at rest for the database, object storage and backups; field-level encryption for travel-document and licence data; complete audit logging; annual penetration test and quarterly dependency review.

29.6 Maintainability

ID	Requirement
NFR-M01	Backend line coverage ≥ 80%, and ≥ 95% for the domain layer (pricing, policy, state machines, validation)
NFR-M02	Module dependency contracts enforced in CI; a forbidden import fails the build
NFR-M03	Cyclomatic complexity ≤ 10 per function, enforced by lint
NFR-M04	All code formatted and linted automatically; no style debate in review
NFR-M05	OpenAPI specification generated from code on every build and published; a breaking change fails the contract test
NFR-M06	Every migration is reversible or ships with a documented forward-fix; migrations are applied separately from application deployment
NFR-M07	No business constant is hard-coded; all thresholds, rates, weights and TTLs live in `system_setting`
NFR-M08	A new engineer can run the full stack locally with one command and seeded data

29.7 Usability

ID	Requirement
NFR-U01	A first-time tourist completes a representative trip (stay + 2 activities + airport transfer) in ≤ 12 minutes in moderated testing, with a task success rate ≥ 90%
NFR-U02	Every screen has defined loading, empty and error states (specified in Sections 24 and 25)
NFR-U03	Every error message states what happened and what the user can do next; no raw technical text reaches a user
NFR-U04	The full trip cost is visible before any payment step; no charge is ever introduced after the summary
NFR-U05	Any destructive action (cancel, delete account) requires explicit confirmation and shows its financial consequence first
NFR-U06	A tourist can reach the confirmed itinerary, driver details and support number in at most two taps from app launch

29.8 Accessibility

Conformance target **WCAG 2.1 Level AA** for the web portals and the equivalent platform accessibility guidelines for the mobile apps: contrast ≥ 4.5:1 for text, touch targets ≥ 48 dp, complete screen-reader labelling and reading order, full keyboard operability on web, dynamic type to 200% without loss of function, no information conveyed by colour alone, captions on any instructional media, and visible focus indicators. Accessibility is verified by automated scanning (axe) in CI plus a manual screen-reader pass per release on the booking journey.

29.9 Compatibility

iOS 15+, Android 8+ (API 26). Web portals: the two most recent major versions of Chrome, Edge, Firefox and Safari, at viewport widths from 360 px. No dependency on any browser plugin. API clients are supported for 12 months from release, enforced by `min_supported_version`.

29.10 Localisation and Internationalisation

All user-facing strings externalised from day one; no concatenated sentences; ICU message format for plurals and gender. Dates, times, numbers and currencies formatted by locale, with all times rendered in the destination's timezone and labelled as such. Content entities (destination, attraction, activity, accommodation descriptions, notification templates, help articles) support per-locale rows with fallback to the default locale. V1 ships English only; the framework must make adding French, German or Italian a content exercise, not an engineering one. Right-to-left layout support is deferred but no layout may hard-code left alignment.

29.11 Data Privacy

Detailed in Section 31.

29.12 Disaster Recovery

ID	Requirement
NFR-DR01	RPO ≤ 5 minutes (continuous WAL archiving), RTO ≤ 60 minutes for a full regional restore
NFR-DR02	A documented, versioned runbook exists for database restore, region failover and PSP outage
NFR-DR03	Restore is rehearsed at least quarterly into an isolated environment, and the rehearsal result is recorded
NFR-DR04	Infrastructure is reproducible from version-controlled Terraform; no manually created production resource is permitted

29.13 Backup

Automated daily full database snapshots retained 30 days; continuous WAL archiving retained 7 days; weekly snapshots retained 12 months; monthly retained 7 years for financial records. Object storage versioned with cross-region replication. All backups encrypted; restore integrity verified by automated monthly test restore. Backup access is separately privileged from production access.

29.14 Monitoring and Alerting

Alert	Condition	Severity
API error rate	5xx > 1% over 5 min	P1
Payment success rate	< 90% over 15 min	P1
Webhook processing lag	> 2 min	P1
Reconciliation exception	any unmatched settlement line	P1
Booking/payment consistency violation	any row failing the Section 20.4 table	P2
Unfulfilled driver assignment	any assignment within 24 h of pickup unassigned	P2
Queue depth	`payments` or `finance` > 100 for 5 min	P2
Database connections	> 80% of pool for 10 min	P2
Hold expiry rate	> 30% of holds expiring over 1 h	P3
Routing provider errors	> 5% over 15 min	P3
Certificate expiry	< 21 days	P3

30 Security Architecture

30.1 Threat Model Summary

Asset	Principal threats	Primary controls
Tourist accounts and PII	Credential stuffing, account takeover, enumeration	Argon2id, rate limits, lockout, breach-list checks, non-enumerating responses, optional MFA
Payment flow	Amount tampering, replay, card data exposure	Server-computed amounts, idempotency keys, PSP-hosted fields, signed webhooks, SAQ A scope
Inventory and pricing	Oversell via race, price manipulation	Row locks, DB constraints, server-side pricing, quote tokens
Provider payouts	Fraudulent payout redirection	Tokenised payout accounts, change requires re-verification and a cooling period, finance approval on release
Driver and tourist location	Stalking, surveillance	Purpose limitation, assignment-scoped visibility, short retention, audited administrative access
Administrative functions	Insider misuse, privilege escalation	Least privilege RBAC, mandatory TOTP, IP allow-list, complete audit log, no delete rights
API surface	Broken object-level authorisation (OWASP API #1)	Ownership predicate on every object access, UUID external identifiers, automated authorisation tests

30.2 Authentication

Passwords hashed with Argon2id (memory 64 MiB, time cost 3, parallelism 4) and never logged. Minimum 12 characters, checked against a breached-credential list; no forced periodic rotation. Account lockout after 10 failed attempts in 15 minutes, with exponentially increasing lockout up to 1 hour, and a notification to the account owner. Email verification required before booking. TOTP MFA is mandatory for `PROVIDER_*` and all administrative roles and optional for tourists and drivers. Social sign-in (Google, Apple) is supported for tourists via OIDC with server-side token verification. Sessions are JWT-based: access token 15 minutes, refresh token 30 days with rotation and reuse detection — a replayed refresh token revokes the whole family and alerts the user.

30.3 Authorisation

Two independent checks on every request, both required:

```
1. ROLE CHECK      does this principal's role permit this operation class?
                   (DRF permission class, declarative per view)
2. OWNERSHIP CHECK does this principal own, or have a granted relationship to,
                   this specific row?
                   (queryset filtered by principal at the repository layer —
                    never "fetch then compare", which leaks existence)
```

Object identifiers exposed to clients are UUIDs, so identifier guessing is impractical; nevertheless the ownership check is the control, and the UUID is defence in depth. Every endpoint has at least one negative authorisation test (TC-2xx series) proving that another principal receives `404`, not `403`, for rows they do not own — because `403` confirms existence.

30.4 Transport and Storage Encryption

TLS 1.2 minimum (1.3 preferred) with modern cipher suites, HSTS with a one-year max-age and preload, and certificate pinning in the mobile release builds for the API and payment domains. Database, object storage and backups encrypted at rest with managed keys. Field-level encryption using envelope encryption for `tourist_profile.passport_reference`, `driver.licence_number` and `driver.national_id_ref`; keys held in the secrets manager, rotated annually, with decryption permitted only in the two service methods that legitimately need it, each of which writes an audit entry.

30.5 Input Validation and Injection Defence

All input validated by serializers against explicit schemas; unknown fields rejected rather than ignored. SQL injection is structurally prevented by exclusive use of the ORM and parameterised queries; raw SQL is prohibited outside reviewed reporting queries which take only bound parameters. Output encoding by default in React and in the template engine; the notification template engine is sandboxed with no attribute access. Content Security Policy on both web applications with no `unsafe-inline`. Uploads: allow-listed MIME types verified by content sniffing rather than extension, size caps, image re-encoding to strip metadata and any embedded payload, malware scanning, storage outside the web root with short-lived signed URLs, and randomised stored filenames.

30.6 CSRF, CORS and Headers

The mobile apps use bearer tokens and are not CSRF-exposed. The web portals use bearer tokens held in memory with refresh tokens in `HttpOnly`, `Secure`, `SameSite=Strict` cookies, and therefore carry CSRF double-submit tokens on state-changing requests. CORS allows only the known portal origins with credentials; wildcards are prohibited. Standard headers applied globally: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` denying unused features.

30.7 Rate Limiting and Abuse Control

The limits in Section 9.6, implemented as Redis token buckets keyed by principal where authenticated and by IP otherwise, with a WAF layer above for volumetric protection. Additional deterministic abuse rules: registration velocity per IP, payment attempt velocity per account and per instrument, and booking velocity per device fingerprint.

30.8 API Security

Every endpoint declares its authentication and authorisation requirements in code, and a test asserts that no endpoint is unintentionally public. Mass assignment is prevented by explicit serializer field lists. Excessive data exposure is prevented by role-specific serializers — a driver's serialisation of a booking omits the tourist's surname and full contact details until the disclosure window. Pagination limits are capped server-side. The OpenAPI document is published for partners but reveals no internal identifiers or infrastructure detail.

30.9 Secrets Management

No secret in source control, in an image, or in an environment file committed anywhere. Secrets live in a managed secrets store, injected at runtime, rotated on a schedule and on any suspected exposure. CI holds deployment credentials as masked, environment-scoped variables with no print access. A pre-commit secret scanner and a CI scan block accidental commits.

30.10 Payment Security

Reiterating the controls in Section 21.1: no card data touches the Platform; amounts are server-computed; idempotency keys prevent double charges; webhooks are signature-verified, timestamp-checked and deduplicated; refunds require an original payment reference and, above a threshold, human approval; every payment state change is immutably recorded. Payout account changes require re-verification, a 48-hour cooling period, and notification to the provider's registered contact — the single most common payout-fraud vector.

30.11 Location Privacy Controls

As specified in Section 13.7, enforced technically: the tracking endpoint and WebSocket subscription both re-evaluate the ownership predicate on every message, driver location collection is bound to assignment state in the API (a location POST outside an active assignment is rejected), and administrative access to location history requires the support role and writes an audit entry naming the booking under investigation.

30.12 Audit Logging

Every authentication event, authorisation failure, administrative action, financial action, verification decision, configuration change, data export and privacy request is recorded in `audit_log` with actor, role, action, entity, before/after state, IP, user agent, request id and timestamp. The table is append-only at the database role level. Logs are shipped to a separate retention store within 24 hours so that a compromise of the application database cannot erase the trail.

30.13 Vulnerability Management

Dependency scanning on every build with a policy that blocks known-critical vulnerabilities; static analysis (Bandit, Semgrep) and secret scanning in CI; container image scanning; quarterly dependency upgrade sprint; annual third-party penetration test with remediation tracked to closure; a published security contact and a coordinated disclosure policy.

30.14 Deterministic Fraud and Anomaly Rules

Explicitly rule-based, with all thresholds in `system_setting` and every flag reviewed by a human before any account action:

Rule	Condition	Action
FR-01	≥ 4 failed payments on one account in 1 hour	Throttle payments, flag for review
FR-02	≥ 3 distinct accounts sharing one payment instrument in 24 hours	Flag all involved accounts
FR-03	Trip total ≥ configured high-value threshold from an account created < 24 hours ago	Manual review before driver disclosure
FR-04	Cancellation rate of an account > 60% over ≥ 5 bookings	Flag; restrict to non-refundable inventory
FR-05	Driver marks `ARRIVED` outside geofence > 3 times in 30 days	Compliance review
FR-06	Provider cancellation rate > 15% over ≥ 20 bookings	Performance review, possible suspension
FR-07	Review submitted from the same device as the provider's account	Hold review, flag
FR-08	Payout account changed within 7 days of a large pending balance	Hold payout, require re-verification

30.15 Incident Response

Severity definitions, an on-call rotation, a documented escalation path, and a communications template set. Any suspected personal-data breach follows a defined process: contain, assess, record in the breach register, notify the data protection authority and affected individuals within the statutory period where the assessment requires it, and conduct a post-incident review with tracked actions. Payment incidents additionally notify the PSP and, where required, the acquirer.

31 Data Privacy

31.1 Personal Data Inventory

Category	Data	Subject	Lawful basis	Retention
Identity	Name, email, phone, date of birth, nationality	Tourist	Contract	Account life + 30 days
Travel document	Passport reference	Tourist	Contract / legal obligation, collected only where a booked service requires it	Encrypted; purged 90 days after trip completion
Payment	PSP token, card brand, last four digits, mobile-money number	Tourist	Contract	7 years (financial record)
Booking	Trips, itineraries, bookings, messages	Tourist, Provider	Contract	7 years financial, 24 months messages

Category	Data	Subject	Lawful basis	Retention
Location	Driver position during assignment	Driver	Contract, legitimate interest	30 days raw
Identity documents	Licence, national identity, vehicle documents	Driver, Provider	Contract, legal obligation	Duration of engagement + 12 months
Behavioural	Login history, device identifiers	All	Legitimate interest (security)	12 months
Emergency contact	Name, phone	Tourist	Vital interest	Account life

31.2 Principles Applied

Minimisation — the passport field is not collected at registration and is requested only when a booked service requires it, with an explanation of which service and why. **Purpose limitation** — data collected for fulfilment is never repurposed for marketing without separate opt-in. **Accuracy** — users may correct their own data; corrections propagate to open bookings. **Storage limitation** — the retention schedule above is enforced by scheduled purge jobs, not by policy alone. **Integrity and confidentiality** — Section 30. **Accountability** — a record of processing activities, a data protection impact assessment covering location tracking and payment processing, and processor agreements with every sub-processor.

31.3 Access Control over Personal Data

Providers see only what fulfilment requires: a guest's first name and last initial until 24 hours before service, then the full name; never the email, never the payment data, and a masked phone route rather than the number. Drivers see the tourist's first name, party size, luggage count, flight number and pickup point — nothing more. Support agents see contact details only after opening a ticket that references the record, and each such access is audited. Analytics and reporting operate on aggregated or pseudonymised data; no report exports raw personal data without a `FINANCE_OFFICER` or `SUPER_ADMIN` role and an audit entry.

31.4 Cross-Border Transfer

Tourists are predominantly EU, UK and North American residents, so processing is designed to a GDPR-equivalent standard regardless of where the infrastructure sits, and the Tanzania Personal Data Protection Act 2022 applies to processing in-country. Data residency, the primary hosting region and the transfer mechanism (standard contractual clauses with sub-processors) must be confirmed with counsel before launch; the architecture does not depend on a particular region and can be deployed to an EU or an African region without change.

31.5 Location Data Specifically

Collected only while an assignment is active; visible only to the counterpart tourist and to authorised staff; raw points purged at 30 days leaving only the route polyline attached to the booking; the driver is shown a persistent indicator whenever collection is running and can disable it, which takes them offline. Location is never used to infer anything about a driver outside working assignments, and never sold or shared.

31.6 User Rights

Right	Implementation
Access	`Settings → Download my data` generates a machine-readable export asynchronously and delivers a signed, expiring link
Rectification	Self-service profile editing; support-assisted for verified fields
Erasure	`Settings → Delete my account`; see 31.7
Restriction	Support-initiated flag that suspends processing beyond legal retention
Portability	The same export, in JSON, covering profile, trips, bookings and reviews
Objection	Marketing opt-out is immediate and self-service
Withdrawal of consent	Per-channel notification preferences, subject to the non-disableable critical set

31.7 Erasure and Retention Conflict

A deletion request cannot erase records the Platform is legally required to retain. The implemented behaviour, disclosed to the user before they confirm:

```
On erasure request:
  - account set to CLOSED, login disabled immediately
  - after a 30-day grace period (recoverable on request):
      * profile fields overwritten with pseudonymous placeholders
      * email/phone replaced with a hashed, non-reversible token
      * messages, reviews and support tickets anonymised
      * device tokens, sessions and location data deleted
  - retained, keyed to a pseudonymous subject id:
      * bookings, payments, refunds, ledger entries, payouts, invoices
        (financial and tax obligation, 7 years)
      * audit log entries (integrity obligation)
  - an open or future booking blocks erasure until the trip completes;
    the user is told this and offered cancellation instead
```

31.8 Consent and Transparency

A layered privacy notice: a short in-app summary at the point of collection and a full notice available offline. Separate, unbundled consent for marketing. No dark patterns: the decline option is as prominent as the accept option, and no pre-ticked boxes exist anywhere in the product.

31.9 Third-Party Processors

Every processor — PSP, routing provider, push/email/SMS gateways, cloud host, crash reporting — is listed in the privacy notice with its purpose and region, is covered by a data processing agreement, and receives the minimum data needed. Crash reporting is configured to scrub PII from breadcrumbs and payloads. No analytics SDK that performs cross-app tracking is permitted in either mobile application.

32 Error Handling

32.1 Principles

Errors are part of the product, not an afterthought. Four rules govern all of them: the user is told what happened and what to do next; the system never exposes stack traces, SQL, internal identifiers or infrastructure detail; every error is correlated to a `request_id` the user can quote to support; and every error is either retryable-safe or explicitly marked non-retryable.

32.2 Standard Codes

HTTP	When used	Client behaviour
400	Malformed request that fails before validation (bad JSON, missing header)	Treat as a bug; log and show a generic message
401	Missing, expired or invalid token	Refresh once, then route to login preserving the intended destination
403	Authenticated but the role forbids the operation	Show a clear permission message; never for ownership failures
404	Resource does not exist, or exists but is not owned by the principal	Show a “not found” state; never disclose existence
409	State conflict: illegal transition, version conflict, expired quote or hold, inventory unavailable, offer already taken	Specific, actionable message per `code`; often offers alternatives
422	Semantic validation failure	Map `details[].field` to form fields inline
423	Account locked	Show the unlock time
429	Rate limited	Back off using `Retry-After`; disable the action

HTTP	When used	Client behaviour
		until it elapses
500	Unhandled internal error	Generic message plus `request_id`; alert raised server-side
502 / 503	External dependency failure or maintenance	Explain which capability is affected; offer retry; preserve user input

32.3 Error Code Catalogue (selected)

Code	HTTP	Meaning	Retryable
`EMAIL_ALREADY_REGISTERED`	409	Registration conflict	No
`INVALID_CREDENTIALS`	401	Login failure, non-enumerating	Yes
`MFA_REQUIRED`	401	TOTP code needed	Yes
`ITINERARY_INVALID`	422	One or more `ERROR` findings	No, until fixed
`INVENTORY_UNAVAILABLE`	409	Capacity gone; alternatives supplied	No
`PRICE_CHANGED`	409	Catalogue price moved since generate	No, requires acceptance
`QUOTE_EXPIRED`	409	Quote TTL elapsed	No, re-quote
`HOLD_EXPIRED`	409	Hold released before capture	No
`TRIP_NOT_QUOTABLE` / `TRIP_NOT_PAYABLE`	409	Wrong trip state	No
`ILLEGAL_TRANSITION`	409	State machine guard failed	No
`LOCKED_ITEM_CONFLICT`	409	Regeneration would alter a confirmed item	No
`NO_TARIFF_CONFIGURED`	422	No corridor or tariff matches	No
`ROUTING_UNAVAILABLE`	502	Routing provider down with no cache	Yes
`PSP_UNAVAILABLE`	502	Payment provider unreachable	Yes
`PAYMENT_DECLINED`	402	Issuer or wallet declined	Yes, other method
`OFFER_TAKEN` / `OFFER_EXPIRED`	409	Driver lost the race or the TTL	No
`GEOFENCE_VIOLATION`	422	Status transition outside the fence without an override reason	Yes with reason
`CANCELLATION_NOT_PERMITTED`	409	Booking state or policy forbids it	No
`REFUND_EXCEEDS_CAPTURED`	422	Refund larger than the remaining capturable amount	No
`VERSION_CONFLICT`	409	Optimistic lock failure	Yes after refetch

32.4 Server-Side Handling

A single DRF exception handler converts the exception hierarchy of Section 8.7 into the error envelope, attaches the `request_id`, selects the localised message, and logs at the appropriate level: `INFO` for expected client errors, `WARNING` for conflicts and external-dependency failures, `ERROR` for unhandled exceptions with a full stack trace to the log store only. Unhandled exceptions inside a transaction always roll back; there is no partial write path. Celery task failures retry per the policy in Section 8.8 and dead-letter with an alert rather than failing silently.

32.5 Client-Side Handling

Mobile and web clients map `code` to a localised, specific message with an appropriate action. Never a raw message from the server for a `500`. Form errors are rendered inline against the field named by `details[].field`. Conflict errors on the booking path receive bespoke treatment because they are the ones that cost money: `INVENTORY_UNAVAILABLE` renders per-item with alternatives, `PRICE_CHANGED` renders an old/new comparison, `QUOTE_EXPIRED` offers a one-tap re-quote. Payment-in-flight states are never dismissible, and back navigation is blocked while a payment is pending resolution.

32.6 Logging and Correlation

Every request receives an `X-Request-Id` (generated if absent) which flows into the log context, into every downstream call, into Celery task headers, and into the error response. Support can therefore take a `request_id` from a user's screenshot and reconstruct the entire causal chain including background work. Logs are structured JSON, PII-redacted by a filter, retained 30 days hot and 12 months cold.

33 Testing Strategy

33.1 Test Pyramid and Ownership

Level	Share of suite	Tooling	Runs on
Unit (domain layer, pure functions)	~60%	pytest	Every commit
Integration (service + database + fakes for external ports)	~25%	pytest + `pytest-django` + testcontainers	Every commit
API contract	~7%	schemathesis against the generated OpenAPI	Every commit
Client widget/component	~5%	Flutter test, React Testing Library	Every commit
End-to-end	~3%	Playwright (web), Patrol/integration_test (mobile)	Every merge to main, nightly full run

External ports always have three implementations: real, fake (deterministic, in-memory, used in tests) and recording (used to refresh fixtures). No test in CI calls a real external service.

33.2 Unit Testing Focus

The domain layer carries the highest coverage requirement (95%) because it holds the logic whose failure costs money: transfer pricing, accommodation rate resolution, commission resolution, refund calculation, itinerary validation rules VR-01 to VR-17, day-sequencing determinism, state-machine guards, and dispatch scoring. Each is a pure function and each has a table-driven test set including boundary values, tie-breaks and currency-rounding edges.

33.3 Integration Testing Focus

Availability and hold concurrency (including deliberate parallel-transaction tests that assert no oversell), the confirmation routine's idempotency, webhook deduplication and out-of-order arrival, refund-to-ledger correctness, driver-assignment overlap prevention against the `EXCLUDE` constraint, and cache-vs-authoritative divergence.

33.4 API Testing

Schemathesis generates requests from the OpenAPI schema and asserts that no input produces a 500 and that every response conforms. Layered on top: an explicit authorisation matrix test that, for every endpoint, exercises each role and asserts the expected outcome, and an ownership test that asserts a foreign principal receives `404`.

33.5 Database Testing

Migration up/down on a production-shaped dataset; constraint tests that assert each `CHECK`, unique index and exclusion constraint actually rejects the invalid case; index-usage assertions on the ten highest-traffic queries via `EXPLAIN`; and a data-integrity job test that seeds a deliberate inconsistency and asserts the nightly consistency checker detects it.

33.6 Security Testing

Automated: dependency and container scanning, static analysis, secret scanning, TLS configuration checks, and the authorisation matrix above. Manual: annual penetration test covering authentication, authorisation, payment

tampering, injection, file upload, and business-logic abuse (price manipulation, hold hoarding, refund abuse). A specific test asserts that a modified client cannot alter a charged amount.

33.7 Performance and Load Testing

Test	Scenario	Pass criterion
LT-01	100 concurrent catalogue browsers for 10 minutes	NFR-P01 met, error rate < 0.1%
LT-02	50 concurrent itinerary generations, 20 items each	NFR-P02 met
LT-03	200 concurrent quote attempts against 20 units of inventory	NFR-P03 met; exactly 20 succeed; zero oversell
LT-04	30 payments per minute for 15 minutes	NFR-P04 met; zero double charges
LT-05	500 drivers posting location batches	NFR-P06 met; queue depth stable
LT-06	Soak: 4 hours at 40% of peak	No memory growth, no connection leak

33.8 Location and GPS Testing

Simulated GPS tracks replay real Zanzibar routes (ZNZ → Nungwi, Stone Town → Paje) to verify geofence entry and exit, status transition guards, ETA plausibility, and the tunnel/coverage-loss case where the driver app must buffer and replay. Accuracy filtering, implausible-speed rejection and duplicate-timestamp handling each have a dedicated test.

33.9 Payment Testing

Executed entirely in PSP sandbox: successful card, declined card, insufficient funds, 3-D Secure challenge passed and failed, expired card, mobile-money success, mobile-money timeout, duplicate webhook, out-of-order webhook, webhook with an invalid signature, full refund, partial refund, refund exceeding capture, and chargeback notification. Each asserts the resulting booking, trip, ledger and notification state.

33.10 Booking and Business-Rule Testing

Every business rule in Section 39 has at least one positive and one negative test. The critical scenarios: concurrent booking of the last room, concurrent booking of the last seat, quote expiry mid-payment, hold expiry mid-payment, provider cancellation of a confirmed booking, cancellation at each policy tier boundary (one second either side), no-show with and without geofence evidence, and trip-level cancellation with mixed policies.

33.11 User Acceptance Testing

Conducted with real participants: five international tourists unfamiliar with the product completing the representative journey unaided; five drivers completing onboarding, offer acceptance and a full transfer; and three of each provider type completing onboarding, listing creation, calendar management and a booking response. Acceptance requires the NFR-U01 task success rate and zero severity-1 defects.

33.12 Representative Test Cases

ID	Title	Preconditions	Steps	Expected
TC-001	Register with a valid email	No account for the address	Submit valid registration	201; user PENDING; verification email sent; audit row written
TC-002	Register with a duplicate email	Account exists	Submit same email	409 `EMAIL_ALREADY_REGISTERED`; no second user row
TC-003	Register with a weak password	—	Password “password1234”	422; breach-list rejection; no user created
TC-010	Login success	Verified account	Correct credentials	200; access and refresh tokens; `last_login_at` set
TC-011	Login with wrong password	Verified account	Wrong password ×1	401 `INVALID_CREDENTIALS`; counter incremented
TC-012	Lockout	10 failures in 15 min	11th attempt	423 with unlock time; owner notified

ID	Title	Preconditions	Steps	Expected
TC-013	Non-enumeration	Unknown email	Login attempt	Response identical to TC-011
TC-020	Search accommodation	Property with availability	Search valid dates	200; only room types available every night
TC-021	Search with check-out before check-in	—	Invalid range	422 `INVALID_DATE_RANGE`
TC-030	Create trip	Authenticated	Valid destination and dates	201; trip DRAFT; empty itinerary v1
TC-031	Create trip in the past	—	start_date yesterday	422
TC-040	Generate itinerary inserts transfers	Stay + activity at different locations	Generate	TRANSFER item inserted with distance and duration; times consistent
TC-041	Validation catches unreachable schedule	Two activities 90 min apart, 2 h travel	Generate	Finding VR-03 severity ERROR; quoting blocked
TC-042	Validation warns on opening hours	Attraction visit on a closed day	Generate	Finding VR-12 severity WARNING; quoting permitted
TC-043	Deterministic ordering	Two items with identical start times	Generate twice	Identical `sequence_no` both times
TC-050	Quote places holds	Valid itinerary	Quote	200; `rooms_held` and `capacity_held` incremented; `quote_expires_at` set
TC-051	Quote when sold out	Availability exhausted	Quote	409 `INVENTORY_UNAVAILABLE` with alternatives; no counters changed
TC-052	Hold expires	Quote, wait past TTL	Sweeper runs	Hold EXPIRED; counters decremented; trip returns to DRAFT-equivalent
TC-053	Concurrent quote for the last room	1 room; 2 simultaneous quotes	Both submit	Exactly one 200, one 409; `rooms_held` = 1
TC-060	Confirm creates the basket	Valid unexpired quote	Confirm	201; bookings PENDING; policy and commission snapshotted; trip PENDING_PAYMENT
TC-061	Confirm with an expired quote	Quote expired	Confirm	409 `QUOTE_EXPIRED`; no bookings created
TC-070	Payment success confirms everything	Basket exists	Sandbox success + webhook	Payment CAPTURED; holds COMMITTED; bookings CONFIRMED; trip CONFIRMED; notifications sent; ledger entries written
TC-071	Duplicate webhook	TC-070 completed	Replay the same event id	200; no state change; no duplicate ledger entry
TC-072	Payment failure releases holds	Basket exists	Sandbox decline	Payment FAILED; holds RELEASED; bookings FAILED; trip PRICED
TC-073	Amount tampering	Basket total 834.75	Client submits 8.34	Charge is 834.75; attempt logged as `AMOUNT_MISMATCH`
TC-074	Idempotent payment intent	—	Same Idempotency-Key twice	One payment row; identical response replayed
TC-080	Driver offer and acceptance	Confirmed transfer, eligible driver	Dispatch, accept	Assignment ASSIGNED; both parties notified; details disclosed to tourist
TC-081	Offer race	Two drivers, one	Both accept	One 200, one 409 `OFFER_TAKEN`

ID	Title	Preconditions	Steps	Expected
		offer		
TC-082	Overlapping assignment rejected	Driver has an overlapping job	Accept	409; exclusion constraint upheld
TC-083	No eligible driver	All drivers ineligible	Dispatch	Assignment UNFULFILLED; ops ticket HIGH raised
TC-084	Expired documents block dispatch	Driver insurance expired	Dispatch	Driver excluded from candidates
TC-090	Geofenced arrival	Driver within 300 m	Status ARRIVED	200; tourist notified by push and SMS
TC-091	Arrival outside geofence	Driver 2 km away	Status ARRIVED	422 `GEOFENCE_VIOLATION`; permitted with override reason; flagged in audit
TC-092	Location visibility scoping	Tourist B not on the booking	Subscribe to the channel	Subscription refused; no location data returned
TC-100	Cancellation refund at tier boundary	MODERATE_7D, 7 days + 1 s before	Cancel	100% refund
TC-101	Cancellation just inside the tier	MODERATE_7D, 7 days − 1 s before	Cancel	50% refund; service fee retained; provider compensation accrued
TC-102	Provider cancellation	Provider cancels a confirmed booking	Cancel	100% tourist refund regardless of tier (BR-045); provider metric incremented
TC-103	Refund exceeds capture	Refund 200 of a 110 booking	Submit	422 `REFUND_EXCEEDS_CAPTURED`
TC-110	Commission snapshot immutability	Booking confirmed at 15%	Change the rule to 20%; complete	Ledger accrues at 15%
TC-111	Ledger balance invariant	Mixed bookings, refunds, payouts	Nightly checker	Debits equal credits per booking and currency
TC-120	Review requires completion	Booking CONFIRMED, not completed	Submit review	409; not permitted
TC-121	One review per booking	Review exists	Submit second	409
TC-200–TC-260	Authorisation matrix	Each endpoint × each role	Call	Permitted roles 200; foreign ownership 404; forbidden roles 403
TC-300	Offline itinerary	Confirmed trip, airplane mode	Open the app	Itinerary, driver details, PIN and support number all render from cache
TC-301	Offline mutation blocked	Airplane mode	Attempt to quote	Blocked with a clear offline message; no local state corruption
TC-302	Driver offline replay	Airplane mode during a transfer	Set statuses, reconnect	Events replay in order with original timestamps, flagged `offline_replay`
TC-901	No-AI constraint	Full source tree and dependency manifest	Automated scan for ML/AI libraries and model artefacts; manual review of any recommendation, ranking, matching or pricing code path	No AI/ML dependency present; every ranking, matching and pricing path traceable to an explicit rule or SQL ordering
TC-902	Determinism	Same trip, same catalogue state	Generate and quote twice	Byte-identical itinerary structure and totals

34 Technology Stack

For each layer: the recommendation, why it suits this system, the trade-off accepted, and the alternative that would be chosen if the constraint changed.

34.1 Mobile Applications

Flutter 3.x / Dart. Suitable because one team ships two platforms for two apps, and because the booking flow's correctness benefits from identical rendering everywhere. Advantages: development speed, a single design system, strong plugin coverage for maps, push, secure storage and background location. Disadvantages: larger binaries, platform-channel work for PSP SDKs and background location, a smaller local hiring pool than Android-native. Alternatives: React Native (choose if the team's strength is JavaScript), native Kotlin/Swift (choose if background location or platform fidelity proves limiting).

34.2 Backend

Python 3.12 / Django 5 / Django REST Framework, with Django Channels for WebSocket and Celery for asynchronous work. Suitable because the system is data-and-rules heavy rather than throughput heavy, and Django's ORM, migrations and permission scaffolding remove a large amount of undifferentiated work. Advantages: velocity, ecosystem, geospatial and financial library maturity, straightforward hiring in East Africa. Disadvantages: lower raw throughput than a compiled runtime, GIL constraints on CPU-bound work (mitigated by moving such work to Celery). Alternatives: NestJS/TypeScript (choose for a shared language with the web front end), Go (choose if throughput becomes the binding constraint), Spring Boot (choose in a Java-standardised enterprise).

34.3 Database

PostgreSQL 16 + PostGIS. Justified in Section 7.1. Advantages: transactional integrity, exclusion constraints, geospatial support, JSONB, full-text search, mature managed offerings. Disadvantages: single-writer scaling ceiling, requiring read replicas and eventual partitioning. Alternatives: MySQL 8 (weaker geospatial and range-constraint support — a material loss here), MongoDB (rejected outright: this domain is relational and money requires transactions).

34.4 Caching, Queueing and Real-Time Fan-Out

Redis 7 for cache, distributed locks, rate limiting, Celery brokering and WebSocket pub/sub. Advantage: one component serving five needs. Disadvantage: a shared failure domain, mitigated by managed clustering and by ensuring nothing durable lives only in Redis. Alternative: RabbitMQ for queueing at higher volumes.

34.5 Web Applications

React 18 + TypeScript + Vite, TanStack Query for server state, Tailwind CSS with a small component library, shared between the provider portal and the admin console as one code base with two role-scoped route trees. Advantages: shared components, one build pipeline, strong typing against the generated OpenAPI client. Disadvantages: a single bundle boundary requiring careful code-splitting and route guarding. Alternative: Next.js if server rendering or SEO becomes a requirement — it is not, since both applications are behind authentication.

34.6 Maps, Routing and Geocoding

Section 12.3. Recommendation: Mapbox or a self-hosted OSRM for routing and matrix work, with a high-quality geocoder for address resolution, all behind `RoutingPort`. The decision is commercial and reversible.

34.7 Payments

Section 21.2. Recommendation: an international card acquirer plus a Tanzanian mobile-money aggregator, both behind `PaymentGatewayPort`.

34.8 Notifications

Firebase Cloud Messaging for push on both platforms; a transactional email provider with strong deliverability and template APIs; an SMS gateway with reliable Tanzanian termination and international reach. All three behind ports, because SMS providers in particular are frequently changed for cost and deliverability reasons.

34.9 Cloud Infrastructure

A single major cloud provider with a managed PostgreSQL service, managed Redis, object storage, a container runtime, a secrets manager and a CDN. Region selection follows the data-residency decision in Section 31.4 with a preference for the lowest-latency region serving both European tourists and Tanzanian operations. Advantages of managed services: backup, failover and patching are handled, which matters greatly for a small team. Disadvantage: cost and a degree of lock-in, mitigated by containerised workloads and Terraform-defined infrastructure.

34.10 Version Control, CI/CD and Tooling

Git on a hosted platform with trunk-based development and short-lived feature branches; protected main requiring review and green CI. GitHub Actions (or equivalent) for build, test, scan, image publish and deploy. Terraform for infrastructure, Docker for packaging. Observability via Prometheus/Grafana or a managed equivalent, Sentry for error tracking, OpenTelemetry for tracing. Project tracking with linked requirement identifiers so that Section 42's traceability is maintained in the tracker as well as in this document.

34.11 Summary Table

Layer	Selection	Primary alternative
Tourist & driver apps	Flutter 3 / Dart	React Native
Provider portal & admin	React 18 + TypeScript + Vite	Next.js
API	Django 5 + DRF (Python 3.12)	NestJS
Real-time	Django Channels (ASGI)	Dedicated Node WebSocket service
Async work	Celery + Redis	RQ, Dramatiq
Database	PostgreSQL 16 + PostGIS	MySQL 8
Cache/locks/pub-sub	Redis 7	Memcached + RabbitMQ
Object storage	S3-compatible	Any managed equivalent
Search	PostgreSQL FTS	OpenSearch (when catalogue scale demands)
Maps/routing	Mapbox or self-hosted OSRM	Google Maps Platform
Payments	International acquirer + local mobile money	Regional aggregator
Push/email/SMS	FCM + transactional email + SMS gateway	Any, behind the port
IaC / CI-CD	Terraform + GitHub Actions	Pulumi, GitLab CI
Monitoring	Prometheus + Grafana + Sentry + OTel	Managed APM

35 Deployment Architecture and DevOps

35.1 Environments

Environment	Purpose	Data	Access
Local	Development	Seeded synthetic	Developer
CI	Automated testing	Ephemeral per run	Pipeline only
Development	Shared integration	Synthetic, reset weekly	Engineering
Staging	Pre-production verification, UAT	Anonymised production-shaped; sandbox PSP	Engineering, QA, product
Production	Live service	Live	Restricted, audited, break-glass procedure

Staging is configured identically to production apart from scale, and every release passes through it. Production data is never copied to a lower environment without anonymisation.

35.2 Deployment Topology

Section 6.6. Production runs at minimum two API instances and two WebSocket instances across two availability zones, dedicated Celery workers per queue class, one Celery Beat scheduler (singleton, with a lock so that a duplicated scheduler cannot double-fire jobs), a managed PostgreSQL primary with a standby and a read replica, managed Redis, object storage behind a CDN, and a NAT gateway with a stable egress address for provider allow-listing.

35.3 CI/CD Pipeline

```
push / PR
  |
  ├─ lint + format check
  ├─ type check
  ├─ import-linter (module boundary contracts)
  ├─ unit tests (coverage gate: 80% overall, 95% domain)
  ├─ integration tests (testcontainers: Postgres + Redis)
  ├─ API contract tests against the generated OpenAPI
  ├─ security: dependency scan, SAST, secret scan
  └─ build container image, scan image
       |
  merge to main
       |
  ├─ deploy to staging (automatic)
  ├─ run migrations (separate, gated step)
  ├─ smoke suite + E2E suite on staging
  └─ manual approval
       |
  ├─ canary to 10% of production traffic, 15 minutes
  ├─ automated health gate (error rate, latency, payment success)
  └─ full rollout, or automatic rollback on gate failure
```

35.4 Database Migration Policy

Migrations run as a distinct pipeline step before the application rollout, and every migration must be backward compatible with the currently running version — the expand/contract pattern. Column drops and renames are split across two releases. No migration takes a long-held exclusive lock on a large table; index creation uses `CONCURRENTLY`. Every migration is reviewed for lock behaviour before merge, and destructive migrations require a second approver.

35.5 Configuration and Secrets

Twelve-factor configuration through environment variables sourced from the secrets manager at boot. No secret in an image or a repository. Business configuration lives in `system_setting` and is changeable at runtime by administrators; only infrastructure configuration requires a deployment.

35.6 Release, Rollback and Feature Flags

Semantic versioning, tagged releases, generated change logs. Rollback is a redeployment of the previous image, which is always safe because migrations are backward compatible. Feature flags decouple deployment from release and allow a partially built capability to ship dark; the flag set is served to clients through `GET /config`.

35.7 Domains, TLS and CDN

Separate hostnames for the API, WebSocket, provider portal, admin console, media and the marketing site. Certificates issued and renewed automatically with expiry alerting at 21 days. HSTS with preload. Admin console additionally restricted by network allow-list. Media served through the CDN with signed URLs for private objects, long cache lifetimes and content-hashed filenames.

35.8 Backup, Restore and Disaster Recovery

Per Sections 29.13 and 29.14: continuous WAL archiving, daily snapshots retained 30 days, weekly retained 12 months, monthly retained 7 years for financial data, cross-region replication of object storage, quarterly rehearsed restore, and a versioned runbook covering database restore, region failover, PSP outage, routing-provider outage and a compromised-credential response.

35.9 Operations Runbooks

Written and version-controlled for: unfulfilled driver assignment escalation, payment reconciliation exception, stuck payment resolution, oversell investigation, mass notification failure, PSP outage customer communication, database failover, and the break-glass production access procedure. Each runbook names the alert that triggers it, the diagnostic steps, the remediation, and the follow-up record required.

36 Project Structure

36.1 Repository Strategy

A monorepo with four deployable units and one shared contract package. The monorepo is chosen because the API contract, the mobile clients and the web clients change together in this product, and coordinating four repositories for a single feature would dominate the team’s time.

```
zanzibar-platform/
├── backend/
│   ├── config/                    settings (base, dev, staging, prod), asgi, wsgi, celery
│   ├── apps/
│   │   ├── identity/              user, roles, auth, sessions, devices
│   │   ├── catalogue/             countries, regions, destinations, attractions,
│   │   │                          activities, accommodation, room types, media
│   │   ├── provider/              providers, documents, staff, drivers, vehicles
│   │   ├── inventory/             availability, departures, holds
│   │   ├── trip/                  trips, itineraries, items, flights, validation
│   │   ├── booking/               bookings, subtypes, status machine, policies
│   │   ├── transport/             corridors, tariffs, dispatch, assignments
│   │   ├── location/              routing port, cache, driver location, geofences
│   │   ├── payment/               payments, transactions, refunds, webhooks
│   │   ├── finance/               commissions, ledger, balances, payouts
│   │   ├── notify/                templates, dispatch, deliveries, preferences
│   │   ├── messaging/             conversations, messages, moderation
│   │   ├── review/                reviews, responses, aggregates, moderation
│   │   └── administration/        audit, settings, feature flags, support tickets
│   │       └── (each app: models.py repositories.py selectors.py services.py
│   │           domain/ serializers.py views.py permissions.py urls.py
│   │           tasks.py adapters/ tests/)
│   ├── common/                    base models, money, errors, pagination,
│   │                              middleware, events, state machine core
│   ├── migrations_ops/            data migrations and backfills
│   ├── scripts/                   seed, fixtures, maintenance commands
│   └── tests/                     cross-module integration and E2E API tests
│
├── mobile/
│   ├── tourist_app/               (structure per Section 23.3)
│   ├── driver_app/
│   └── shared_package/            API client, models, design system, formatters
│
├── web/
│   ├── provider-portal/
│   ├── admin-console/
│   └── shared-ui/                 components, hooks, generated API client, theme
│
├── contracts/
│   ├── openapi/                   generated specification, versioned
│   └── events/                    domain event schemas
│
├── infrastructure/
│   ├── terraform/                 modules/ and environments/{dev,staging,prod}
│   ├── docker/                    Dockerfiles, compose for local development
│   └── k8s/ or ecs/               deployment manifests
│
├── database/
│   ├── erd/                       diagram sources and exports
│   ├── seeds/                     Zanzibar catalogue seed data
│   └── docs/                      schema notes, index rationale
│
├── documentation/
│   ├── srs/                       this document
│   ├── adr/                       architecture decision records
│   ├── runbooks/                  operational procedures
│   ├── api/                       published API guide
│   └── onboarding/                developer setup
│
├── tests/
│   ├── e2e/                       Playwright suites
│   ├── load/                      k6 or Locust scenarios
│   └── fixtures/                  shared datasets
│
└── .github/workflows/             CI/CD pipelines
```

36.2 Notes on Key Directories

`backend/apps/*/domain/` holds the pure business logic and carries the strictest coverage requirement; it must contain no imports from Django's ORM. `backend/apps/*/adapters/` is the only place a third-party SDK may be imported. `common/state_machine.py` holds the single table-driven transition validator used by every machine, so that no module implements its own. `contracts/openapi/` is generated, committed and diffed in review, which makes any breaking API change visible in the pull request. `database/seeds/` contains the Zanzibar catalogue as data — the file that proves the destination-independence claim, because replacing it configures a different market.

37 Development Phases

Fourteen phases, estimated for a team of six (two backend, two mobile, one web, one QA) with part-time design and DevOps support. Durations are engineering weeks and assume the dependencies shown are respected.

37.1 Phase 1 — Architecture and Foundation (3 weeks)

Features Repository and monorepo tooling; Django project skeleton with the module layout and import-linter contracts; base models, money type, error envelope, request-id middleware, event bus, state-machine core; PostgreSQL with PostGIS; Redis; Celery with queue classes; Docker Compose for local development; CI pipeline with lint, type check, tests and scans; Terraform for the development environment; OpenAPI generation. **Dependencies** None. **Deliverables** Running skeleton in the development environment; a single health endpoint passing the full pipeline; developer onboarding document. **Acceptance** A new engineer runs the stack locally with one command; a forbidden cross-module import fails CI; the generated OpenAPI document publishes automatically.

37.2 Phase 2 — Authentication and User Management (3 weeks)

Features User, role and profile models; registration, email verification, login, refresh with rotation and reuse detection, logout, password reset; TOTP enrolment; RBAC permission classes and the ownership predicate pattern; device registration; audit logging of all authentication events; the authorisation matrix test harness. **Dependencies** Phase 1. **Deliverables** API-01 complete; tourist app auth screens; admin console login. **Acceptance** TC-001 to TC-013 pass; the authorisation matrix test runs green across every endpoint then existing; no endpoint is unintentionally public.

37.3 Phase 3 — Zanzibar Tourism Catalogue (4 weeks)

Features Geography hierarchy; destination, attraction, activity, accommodation and room-type models; media handling with CDN delivery; full-text search; the deterministic ranking expression; catalogue APIs; admin console catalogue CRUD; the Zanzibar seed dataset. **Dependencies** Phase 2. **Deliverables** API-02 complete; Explore, Destination, Attraction, Activity, Accommodation and Room Selection screens; admin catalogue management. **Acceptance** All ten seed destinations manageable entirely through the console with no code change; ranking is byte-identical across repeated identical requests; TC-020 and TC-021 pass.

37.4 Phase 4 — Trip Planner and Itinerary Engine (5 weeks)

Features Trip, itinerary, item and flight models; item CRUD; the day-sequencing algorithm; the routing port with a fake and one real adapter; route cache and the nightly destination-pair matrix; validation rules VR-01 to VR-17; cost computation; itinerary versioning and locking. **Dependencies** Phases 3 and, for transfer insertion, the routing port from Phase 6 — the port interface is delivered here and the tariff logic in Phase 6. **Deliverables** API-03 and API-04 (quoting stub); Trip Planner, Flight Information, Itinerary, Trip Summary and Cost Breakdown screens. **Acceptance** TC-030, TC-031, TC-040 to TC-043 and TC-902 pass; generation of a 20-item itinerary meets NFR-P02.

37.5 Phase 5 — Accommodation and Inventory (3 weeks)

Features Availability calendar, rate resolution, the hold lifecycle, the concurrency-safe hold and commit routines, the expiry sweeper, and the reconciliation checker; provider availability APIs. **Dependencies** Phases 3 and 4.

Deliverables Accommodation search with authoritative availability; hold mechanics. **Acceptance** TC-050 to TC-053 pass, including the concurrency test with zero oversell under LT-03.

37.6 Phase 6 — Transportation and Tariffs (3 weeks)

Features Corridors and tariffs with the resolution order; the metered pricing function; vehicle classes; the transport quoting API; the degraded-mode rules; admin tariff management with the quote preview tool. **Dependencies** Phases 4 and 5. **Deliverables** API-04 complete; Airport Pickup and Transportation screens. **Acceptance** A quote for every seeded corridor resolves deterministically; an unconfigured corridor returns `NO_TARIFF_CONFIGURED` rather than a guessed price; the worked example in Section 12.4 reproduces exactly.

37.7 Phase 7 — Booking Engine (4 weeks)

Features Booking and subtype models; the state machine and its guards; status history; basket creation; the confirmation and release routines; cancellation policy evaluation with preview; provider accept/decline for on-request activities; voucher generation. **Dependencies** Phases 5 and 6. **Deliverables** API-05; Booking Confirmation and My Trips screens; provider booking management. **Acceptance** TC-060, TC-061, TC-100 to TC-103 pass; every transition in Section 20.2 has a passing positive and negative test.

37.8 Phase 8 — Payments and Finance (4 weeks)

Features Payment gateway port with two adapters; intent creation, capture, webhook ingestion with signature verification and deduplication, the polling fallback; refunds; the payment state machine; commission resolution; the append-only ledger with all entry types; provider balances; payout batch assembly and approval; the daily reconciliation job. **Dependencies** Phase 7. **Deliverables** API-07; Payment screen; provider earnings and payouts; admin payment, refund and payout consoles. **Acceptance** TC-070 to TC-074, TC-110 and TC-111 pass; the full PSP sandbox matrix in Section 33.9 passes; the ledger invariant holds under the mixed-scenario test.

37.9 Phase 9 — Driver System (4 weeks)

Features Driver and vehicle models with verification; the driver app end to end (auth, verification, dashboard, availability, offers, assignment, pickup, active trip, completion, earnings); the dispatch scoring function; offer dispatch with TTL and candidate progression; the assignment state machine with geofence guards; the exclusion constraint; the escalation path. **Dependencies** Phases 7 and 8. **Deliverables** API-06; the complete driver application; admin driver and vehicle management. **Acceptance** TC-080 to TC-084 and TC-090 to TC-091 pass; a full transfer completes from offer to settlement in staging.

37.10 Phase 10 — Real-Time Tracking and Notifications (3 weeks)

Features WebSocket gateway with channel authorisation; driver location ingest, validation, storage and fan-out; geofence-triggered events; the notification subsystem with all events, channels, templates, preferences, retries and delivery records; push registration in both apps. **Dependencies** Phase 9. **Deliverables** Driver Tracking and Active Trip screens; Notifications inbox; admin notification management. **Acceptance** TC-092 passes; NFR-P05 and NFR-P06 met under LT-05; every event in Section 19.1 fires in an end-to-end test.

37.11 Phase 11 — Provider Portal (3 weeks)

Features Provider registration, document upload and verification tracking; dashboard; listing management for all three provider types; the availability and rate calendar with bulk editing; booking management; earnings, statements and payouts; review responses; staff management. **Dependencies** Phases 5, 7 and 8. **Deliverables** The complete provider portal. **Acceptance** Each provider type completes onboarding through to a live, bookable listing without engineering assistance in UAT.

37.12 Phase 12 — Administration Console (3 weeks)

Features All screens in Section 27, including verification workflows, exceptional booking controls, commission and tariff management, moderation, reporting, system settings and the audit log viewer. **Dependencies** All prior phases.

Deliverables The complete admin console. **Acceptance** Every administrative action writes an audit entry with before/after state; the destination-independence acceptance test (Section 41.12) passes.

37.13 Phase 13 — Hardening, Testing and UAT (4 weeks)

Features Full regression, load and soak testing; penetration test and remediation; accessibility audit and remediation; performance tuning against the NFR-P targets; offline behaviour verification; UAT with real tourists, drivers and providers; documentation completion; runbook authoring; support team training. **Dependencies** Phases 1 to 12. **Deliverables** Test reports, remediation log, signed UAT record, runbooks. **Acceptance** Zero severity-1 defects open; all NFR targets met or formally accepted with a mitigation; TC-901 passes.

37.14 Phase 14 — Production Deployment and Launch (2 weeks)

Features Production infrastructure provisioning; production PSP credentials and live-transaction verification; store submissions and review; DNS, TLS, CDN and WAF; monitoring, alerting and on-call activation; provider onboarding and catalogue population; canary launch to a limited cohort, then general availability. **Dependencies** Phase 13. **Deliverables** Live service; launch report; a 30-day hypercare plan. **Acceptance** A real end-to-end paid booking completes in production; alerting verified by a deliberate test alert; rollback rehearsed successfully.

37.15 Timeline Summary

```
Week:  1   4   7  10  13  16  19  22  25  28  31  34  37
P1  [===]
P2      [===]
P3          [=====]
P4              [=======]
P5                    [===]
P6                        [===]
P7                            [=====]
P8                                [=====]
P9                                    [=====]
P10                                       [===]
P11                                           [===]
P12                                               [===]
P13                                                   [=====]
P14                                                         [==]
       Total: approximately 32-38 weeks including overlap and buffer
```

Phases 5 and 6 may run in parallel with different engineers, as may 11 and 12, which is how the calendar total compresses below the sum of durations.

38 MVP Definition

38.1 MUST HAVE — Version 1 Release Scope

Area	Included
Accounts	Tourist registration, verification, login, profile, password reset
Catalogue	Zanzibar destinations, attractions, activities, accommodation; search, filter, deterministic ranking
Planning	Trip creation, dates, party, flights, item selection, itinerary generation, validation, cost breakdown
Transport	Airport transfers and inter-location transfers with corridor and metered tariffs, vehicle classes
Accommodation	Search, availability, rates, room selection, booking
Activities	Departures, capacity, requirements, booking, instant and on-request confirmation
Inventory	Holds with TTL, concurrency-safe commit, expiry sweeper
Checkout	Single-basket quote, confirm, payment by international card and mobile money, USD and TZS
Bookings	Full lifecycle, vouchers, cancellation with policy-based refunds
Drivers	Registration, verification, availability, offers, assignment, status progression, earnings
Tracking	Live driver location during an active transfer, geofence notifications

Area	Included
Notifications	The full event set across push, email and SMS with preferences
Messaging	Booking-scoped tourist–provider and tourist–driver messaging
Reviews	Post-completion ratings and moderated reviews with provider responses
Provider portal	Onboarding, verification, listings, calendar, bookings, earnings, payouts
Admin console	All of Section 27
Finance	Commission, ledger, settlement, payouts, reconciliation
Offline	Confirmed itinerary, driver details, PIN and support number available offline
Quality	The NFR set in Section 29

38.2 SHOULD HAVE — Immediately After MVP

Promotion and discount codes; multi-currency presentment beyond USD and TZS; vehicle hire by the hour (driver at disposal); masked-number voice relay; saved payment methods; trip sharing with a travelling companion; provider mobile applications; additional interface languages (French, German, Italian); a public web booking front end; group and family party management with per-traveller details; waitlists for full activity departures; provider-side bulk import of listings and calendars.

38.3 FUTURE — Deliberately Deferred

Flight status integration and automatic transfer re-timing; on-demand (unscheduled) rides; multi-country and multi-destination trips; mainland Tanzania and East African expansion; loyalty programme; affiliate and agency channels; attraction entrance-fee collection; travel insurance; a partner API for inbound tour operators; dynamic pricing of any kind; a public API marketplace.

38.4 Explicitly Excluded Permanently or Until Reconsidered

Every AI and machine-learning capability listed in Section 3.5. This is not a phasing decision; it is a product constraint that must be re-authorised at the product-owner level before any such work is contemplated.

38.5 MVP Success Criteria

Metric	Target within 90 days of launch
Verified providers live	≥ 60 accommodation, ≥ 40 activity, ≥ 100 drivers
Confirmed trips	≥ 250
Payment success rate	≥ 92%
Trips with all bookings confirmed > 72 h before arrival	≥ 80%
Airport transfers with a driver disclosed ≥ 24 h ahead	≥ 98%
Driver no-show rate	≤ 1%
Oversell incidents	0
Reconciliation exceptions unresolved beyond 48 h	0
Tourist rating of the arrival experience	≥ 4.5 / 5

39 Business Rules

Rules are numbered for traceability and are implemented in the domain layer, enforced by database constraints where possible, and covered by tests.

39.1 Account and Identity

ID	Rule
BR-001	An email address may identify at most one active account
BR-002	A tourist must have a verified email before creating a booking

ID	Rule
BR-003	Provider and administrative accounts must have TOTP enrolled before accessing their consoles
BR-004	An account locked by failed attempts unlocks automatically at the stated time; the owner is notified of the lockout
BR-005	A suspended account may authenticate only to view existing bookings and contact support
BR-006	Changing a verified email or phone reverts that field to unverified and triggers re-verification

39.2 Trip and Itinerary

ID	Rule
BR-010	A trip has exactly one destination and one currency in Version 1
BR-011	Trip end date must be on or after the start date; maximum trip length is 30 days
BR-012	A trip must have at least one adult traveller
BR-013	A trip may be edited only while `DRAFT` or `PRICED`; editing a `CONFIRMED` trip requires cancelling the affected bookings first
BR-014	An itinerary item covered by a confirmed booking is immutable
BR-015	Quoting is blocked while any `ERROR`-severity validation finding exists
BR-016	A trip's currency is locked at first pricing and cannot change thereafter
BR-017	Trip totals must satisfy `total = subtotal + fee + tax` at all times

39.3 Availability and Capacity

ID	Rule
BR-020	A room type is bookable only if every night of the stay has sufficient sellable inventory and is not closed
BR-021	Published availability may never exceed the room type's physical room count
BR-022	An activity may never be booked beyond its departure capacity
BR-023	A provider may not reduce availability below what is already held or sold
BR-024	Capacity is decremented only under a row lock inside a transaction
BR-025	A hold expires at its TTL and its capacity returns automatically
BR-026	An expired hold may not be committed; the booking fails and the tourist is offered alternatives

39.4 Booking

ID	Rule
BR-030	A booking cannot become `CONFIRMED` until its covering payment is `CAPTURED` and its inventory hold is committed
BR-031	Only the transitions declared in Section 20.2 are legal; all others are rejected
BR-032	Every status change writes a history row naming the actor and reason
BR-033	A booking may not start in the past
BR-034	An activity booking must be created before the activity's booking cutoff
BR-035	An accommodation booking must be created before the property's booking cutoff
BR-036	Party size must satisfy the service's capacity and party-size bounds
BR-037	A booking's provider must be `VERIFIED` and unsuspended at the moment of confirmation
BR-038	An administrator may force a transition only with a recorded reason, and only with the `SUPER_ADMIN` role

39.5 Cancellation and Refund

ID	Rule
BR-040	Refund entitlement is computed from the policy snapshot on the booking, never from the provider's current policy
BR-041	The cancellation policy is snapshotted onto the booking at confirmation
BR-042	A tourist may cancel any booking not yet `IN_PROGRESS`; entitlement follows the policy
BR-043	The refund preview shown before cancellation must equal the refund actually issued
BR-044	A refund may never exceed the amount captured and not already refunded
BR-045	Where the provider, driver or Platform causes the cancellation, the tourist receives a full refund regardless of policy tier, and the platform service fee is also refunded

ID	Rule
BR-046	Cancelling a whole trip evaluates each component independently against its own policy
BR-047	A refund above the auto-approval limit requires `FINANCE_OFFICER` approval
BR-048	Cancellation releases inventory immediately, before the refund settles

39.6 Driver and Transport

ID	Rule
BR-050	Only a `VERIFIED` driver with unexpired licence, insurance and inspection, and at least one verified active vehicle, may receive an offer
BR-051	A driver may not hold two assignments whose time windows overlap
BR-052	The assigned vehicle must have seat and luggage capacity at least equal to the booking's requirement
BR-053	Driver identity, vehicle and pickup details must be disclosed to the tourist at least 24 hours before a scheduled arrival transfer, or immediately on assignment if booked later
BR-054	A transfer's price is fixed at confirmation and is unaffected by later tariff changes or actual route deviation
BR-055	`ARRIVED` and `COMPLETED` require the driver to be inside the relevant geofence, or to supply an override reason that is flagged for review
BR-056	A no-show may be declared only after the configured wait period, evidenced by location history
BR-057	Driver location is collected only while an assignment is active

39.7 Payment

ID	Rule
BR-060	The charged amount is always computed server-side from the trip total
BR-061	A payment may be captured at most once; duplicate webhooks have no additional effect
BR-062	A trip may hold at most one non-terminal payment at a time
BR-063	The Platform never stores card numbers, CVV or expiry data
BR-064	For every booking and currency, ledger debits and credits must reconcile; a breach raises a P2 alert
BR-065	Every currency conversion stores its rate, source and timestamp
BR-066	The FX rate applied to a trip is frozen at pricing time

39.8 Commission, Settlement and Payout

ID	Rule
BR-070	The commission rule and rate are snapshotted onto the booking at confirmation and are immune to later rule changes
BR-071	Commission accrues at booking completion, not at payment
BR-072	Personal data that is not required for fulfilment — including travel-document references — is collected only when a booked service requires it, and is purged on the schedule in Section 31.1
BR-073	A provider balance becomes available only after the settlement hold period and only in the absence of an open dispute
BR-074	A payout below the minimum threshold rolls forward to the next cycle
BR-075	Payouts require finance approval before release
BR-076	A payout account change requires re-verification, a 48-hour cooling period, and notification to the registered contact
BR-077	Ledger entries are never updated or deleted; corrections are new reversing entries

39.9 Catalogue and Provider

ID	Rule
BR-080	Only a `VERIFIED` provider may publish a bookable listing
BR-081	A provider may only own listings consistent with its provider type
BR-082	Changes to price, capacity or cancellation policy affect new bookings only
BR-083	Deactivating a listing prevents new bookings but leaves existing bookings intact
BR-084	A destination with active listings or future bookings cannot be deactivated without an explicit administrative

ID	Rule
	override and a recorded reason
BR-085	An expired provider or driver document blocks new bookings and dispatch until renewed

39.10 Reviews and Content

ID	Rule
BR-090	Only the tourist who held a `COMPLETED` booking may review it, once, within the review window
BR-091	Ratings display as “New” until at least three published reviews exist
BR-092	A provider may post one response per review
BR-093	Published reviews cannot be edited; withdrawal unpublishes and recomputes aggregates
BR-094	Contact details in messages and reviews are masked automatically

40 Reporting and Analytics

All reports are conventional database aggregations with parameterised date ranges, on-screen rendering and CSV/XLSX export. Financial reports are generated from the ledger so they reconcile to money movement by construction. There is no predictive, inferential or machine-learning analytics anywhere in this section.

40.1 Report Catalogue

Code	Report	Key measures	Dimensions	Audience
RPT-01	User growth	Registrations, verified accounts, active accounts, churn	Day/week/month, nationality, channel	Admin
RPT-02	Active tourists	Accounts with a trip in period, repeat rate	Period, destination	Admin
RPT-03	Trip funnel	Trips created, priced, quoted, confirmed; drop-off at each stage	Period, destination	Admin, product
RPT-04	Bookings	Count and value by status and service type	Period, provider, destination	Admin, finance
RPT-05	Revenue	Gross bookings, service fees, commission, net revenue, take rate	Period, service type, destination	Finance
RPT-06	Commission detail	Commission by rule, provider and booking type	Period	Finance
RPT-07	Provider earnings	Gross, commission, net, pending, available, paid	Period, provider	Finance, provider
RPT-08	Driver earnings	Completed transfers, gross, commission, net, average per transfer	Period, driver, provider	Finance, provider
RPT-09	Payout register	Batches, amounts, status, rail references	Period, provider	Finance
RPT-10	Popular destinations	Trips, bookings, revenue, average party size	Period	Admin, commercial
RPT-11	Popular activities	Bookings, seats sold, revenue, fill rate against capacity	Period, provider	Admin, commercial
RPT-12	Accommodation performance	Room nights sold, occupancy against published availability, ADR, revenue	Period, property, room type	Admin, provider
RPT-13	Cancellations	Count and value, by initiating party, by policy tier, refund totals	Period, service type, provider	Admin, finance
RPT-14	Payment performance	Attempts, success rate, failure reasons, average time to capture	Period, method, currency	Finance, engineering
RPT-15	Refunds	Count, value, reason codes, time to settle	Period	Finance
RPT-16	Ratings and reviews	Average rating, count, distribution, response rate	Period, provider, subject type	Admin, provider
RPT-17	Transfer operations	Offers sent, acceptance rate, time to assignment, unfulfilled count, no-shows	Period, corridor, driver	Operations
RPT-18	Lead time	Distribution of days between booking and	Period, service type	Commercial

Code	Report	Key measures	Dimensions	Audience
		service date		
RPT-19	Inventory health	Hold-to-conversion rate, expiry rate, stop-sell days	Period, provider	Operations
RPT-20	Reconciliation	Matched, unmatched, variance by cause	Day	Finance
RPT-21	Support	Tickets by category, priority, SLA attainment, resolution time	Period	Operations
RPT-22	Audit extract	Filtered audit entries for an investigation	Any	Compliance

40.2 Implementation Notes

Reports run against the read replica. Any report whose parameters can span more than 90 days executes asynchronously and delivers a signed download link. Aggregations that are expensive and frequently requested (daily revenue, daily booking counts, provider daily earnings) are materialised nightly into summary tables and served from there, with the raw query available for verification. Every exported file records who exported it, when, and with what parameters, in the audit log — because a bookings export contains personal data.

41 Acceptance Criteria

Each module is complete when every criterion below is demonstrably met in the staging environment against production-shaped data, with the named test cases passing.

41.1 Authentication and Accounts

A tourist can register, receive and consume a verification email, log in, refresh a session, reset a forgotten password and update their profile. A provider or administrator cannot access their console without TOTP. Ten failed attempts lock the account and notify the owner. Every endpoint enforces both the role check and the ownership predicate, and a foreign principal receives `404`. **Tests** TC-001 to TC-013, TC-200 series.

41.2 Catalogue

All ten seeded Zanzibar destinations, their attractions, activities and accommodation are browsable, searchable and filterable. Ordering is deterministic and reproducible. An administrator can create, edit, activate and deactivate every catalogue entity without a deployment. Media renders through the CDN with responsive variants. **Tests** TC-020, TC-021, TC-902.

41.3 Trip Planner and Itinerary

A tourist can create a trip, set dates and party, record inbound and outbound flights, add stays, activities and attractions, and generate an itinerary in which required transfers are inserted automatically with real distances and durations. All seventeen validation rules fire correctly, blocking on errors and advising on warnings. Regeneration is deterministic and never alters a locked item. The cost breakdown reconciles line by line to the total. **Tests** TC-030, TC-031, TC-040 to TC-043.

41.4 Accommodation

Search returns only room types available for every night of the requested stay at the requested occupancy. Nightly rates and stay totals are correct including overrides. Quoting places holds under lock; concurrent attempts on the last room produce exactly one success. Providers can publish availability and rates in bulk and cannot reduce availability below what is sold or held. **Tests** TC-020, TC-050 to TC-053, LT-03.

41.5 Activities

Departures materialise from schedules across the horizon. Capacity, party-size bounds, age requirements and booking cutoffs are all enforced. On-request activities enter `AWAITING_PROVIDER` and auto-cancel with a full refund if the

provider does not respond in time. Capacity can never be exceeded. **Tests** TC-051, TC-053, plus the on-request timeout scenario.

41.6 Transportation

Every seeded corridor quotes deterministically with the correct distance, duration and price, and the worked example in Section 12.4 reproduces exactly. Vehicle classes are filtered by capacity. An unconfigured corridor returns `NO_TARIFF_CONFIGURED` and never a guessed price. Routing outages degrade planning to cached data with an explicit "approximate" label and block priced quoting rather than inventing a price.

41.7 Airport Pickup

The tourist can select the gateway, enter arrival information, choose an eligible vehicle class, and see the computed pickup time with the buffer explained. The system records the booking, dispatches to eligible drivers, and assigns one. The tourist sees driver name, photo, rating, languages, vehicle make, model, colour, plate, capacity, pickup time, meeting point and PIN at least 24 hours before arrival. The driver receives the booking and can progress its status. Both parties are notified at every transition. If no driver accepts, an operations ticket is raised before the disclosure deadline. **Tests** TC-080 to TC-084, TC-090, TC-091.

41.8 Booking Engine

Every transition in Section 20.2 succeeds when legal and is rejected with `409` when not. Status history captures actor and reason for every change. Basket creation snapshots the cancellation policy and commission rate. Vouchers generate for every confirmed booking. **Tests** TC-060, TC-061, and the per-transition matrix.

41.9 Payment

A tourist can pay by international card and by mobile money in USD and TZS. The charged amount always equals the server-computed trip total. Successful payment confirms every component booking, commits every hold, writes the ledger entries and dispatches every notification, atomically. Duplicate and out-of-order webhooks have no additional effect. Failure releases holds and returns the trip to a re-payable state. Refunds settle and reverse the correct accruals. **Tests** TC-070 to TC-074, TC-100 to TC-103, TC-110, TC-111.

41.10 Itinerary Delivery and Offline

On confirmation the app downloads the offline pack. With the device offline, the tourist can open the app and see the full day-by-day itinerary, every booking voucher, the driver's identity and vehicle, the pickup point and PIN, and the support number. Payment and planning mutations are blocked with a clear message. **Tests** TC-300, TC-301.

41.11 Reviews

A review can be submitted only for a completed booking, once, within the window. Ratings aggregate correctly and display as "New" below three reviews. Providers can respond once. Moderation holds flagged reviews. **Tests** TC-120, TC-121.

41.12 Destination Independence

This criterion validates OBJ-6 and is executed as a formal acceptance test. An administrator, using only the admin console and the seed loader, creates a new country, region and destination (for example Arusha), adds one attraction, one activity, one accommodation with a room type and calendar, and one transfer corridor with a tariff. A tourist can then plan, quote, pay for and complete a trip to that destination. **Pass condition: no application code change, no deployment, and no database migration is required at any point.**

41.13 Administration

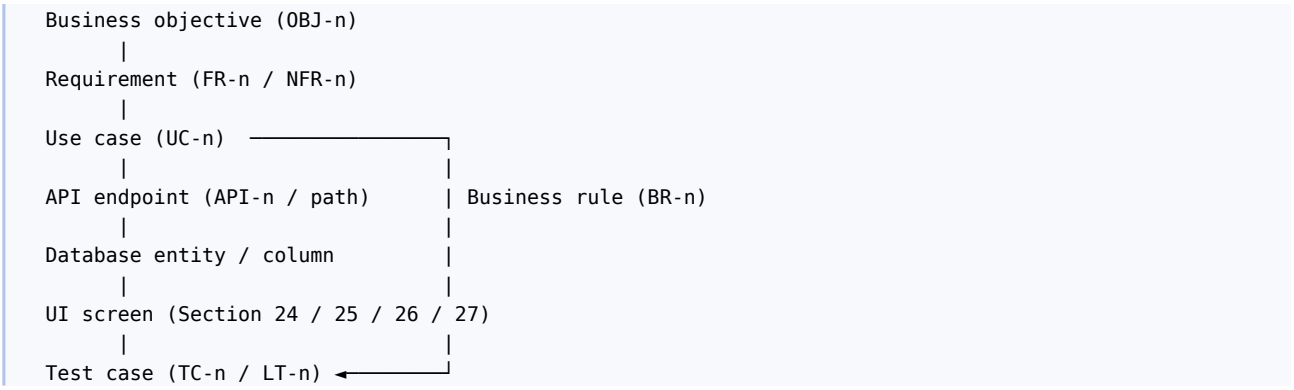
Every entity listed in Section 27 is manageable. Every administrative action writes an audit entry with before and after state, actor, role, IP and request id. Exceptional controls require a reason. Reports render and export, and every export is audited.

41.14 Non-Functional

Every NFR in Section 29 is either measured and met, or formally accepted by the product owner with a recorded mitigation. Load tests LT-01 to LT-06 pass. The accessibility audit shows no Level AA violation on the booking journey. The penetration test has no unremediated high or critical finding. **Tests** LT-01 to LT-06, TC-901.

42 Requirement Traceability

42.1 Traceability Model



Traceability is maintained in two places and they must agree: this matrix, and the issue tracker, where every work item carries its requirement identifier and every pull request references the work item. A requirement with no test is not complete; a test with no requirement is a candidate for deletion or for a missing requirement.

42.2 Matrix (representative extract)

Req	Description	Use case	API	Data	UI	Rules	Tests
FR-001	Tourist registers and verifies an account	UC-01	POST /auth/register, /auth/verify-email	user, tourist_profile	24.3	BR-001, BR-002	TC-001–003
FR-002	Tourist authenticates	UC-01	POST /auth/login, /auth/refresh	user, session	24.4	BR-004, BR-005	TC-010–013
FR-010	Tourist browses destinations	UC-03	GET /destinations	country, region, destination	24.7, 24.8	BR-084	TC-020
FR-011	Tourist browses attractions	UC-04	GET /attractions	attraction	24.9	—	TC-020
FR-012	Tourist searches activities	UC-05	GET /activities, /activities/{id}/departures	activity, activity_departure	24.10	BR-022, BR-034	TC-051
FR-013	Tourist searches accommodation	UC-06	GET /accommodations, /room-types	accommodation, room_type, room_availability	24.11–24.13	BR-020, BR-021	TC-020, TC-021
FR-020	Tourist creates a trip	UC-07	POST /trips	trip, itinerary	24.14	BR-010–012	TC-030, TC-031

Req	Description	Use case	API	Data	UI	Rules	Tests
FR-021	Tourist records flight details	UC-08	PUT /trips/{id}/flights	trip_flight	24.15	VR-07, VR-08	TC-040
FR-022	System generates an itinerary	UC-12	POST /trips/{id}/itinerary/generate	itinerary_item, route_cache	24.19	BR-014, BR-015	TC-040–043, TC-902
FR-023	System computes distance and travel time	UC-11	POST /transport/quotes	route_cache, transfer_corridor	24.17	BR-054	TC-040
FR-024	System prices the trip	UC-13	GET /trips/{id}/summary	trip, itinerary_item	24.20, 24.21	BR-017	TC-050
FR-030	System holds inventory at quote	UC-14	POST /trips/{id}/quote	inventory_hold, room_availability, activity_departure	24.20	BR-024–026	TC-050–053
FR-031	Tourist confirms the basket	UC-14	POST /trips/{id}/confirm	booking, booking_*	24.20	BR-030, BR-041	TC-060, TC-061
FR-040	Tourist pays	UC-15	POST /payments/intents, /webhooks/psp/{p}	payment, payment_transaction	24.22	BR-060–063	TC-070–074
FR-041	System confirms bookings on capture	UC-15	(internal)	booking, inventory_hold, ledger_entry	24.23	BR-030, BR-071	TC-070, TC-071
FR-050	System assigns a driver	UC-31	POST /driver/offers/{id}/accept	driver_offer, driver_assignment	24.18, 25.7	BR-050–053	TC-080–084
FR-051	Driver progresses trip status	UC-33	POST /driver/assignments/{id}/status	driver_assignment, booking	25.10–25.12	BR-055, BR-056	TC-090, TC-091
FR-052	Tourist tracks the driver	UC-17	WS trip.{id}; POST /driver/location	driver_location	24.26	BR-057	TC-092
FR-060	Tourist cancels a booking	UC-19	POST /bookings/{id}/cancel	booking, refund	24.24	BR-040–048	TC-100–103
FR-061	System refunds per policy	UC-20	POST /refunds	refund, ledger_entry	24.24	BR-044, BR-045	TC-101–103
FR-070	System accrues commission and settles	—	(internal)	commission_rule, ledger_entry, provider_balance	26.7	BR-070–073	TC-110, TC-111
FR-071	Finance releases payouts	UC-65	POST /admin/payouts/{id}/approve	payout, payout_item	27.10	BR-074–076	TC-111
FR-080	System notifies at every material event	—	(internal)	notification, notification_delivery	24.28	—	Event coverage suite
FR-081	Booking-scoped messaging	UC-18	GET/POST /conversations/…	conversation, message	24.27	BR-094	Messaging suite
FR-090	Tourist reviews a completed	UC-21	POST /reviews	review, rating_aggregate	24.29	BR-090–093	TC-120, TC-121

Req	Description	Use case	API	Data	UI	Rules	Tests
	booking						
FR-100	Provider onboards and is verified	UC-40, UC-41	POST /provider/register, /provider/documents	provider, provider_document	26.2	BR-080, BR-085	Provider suite
FR-101	Provider manages listings and calendar	UC-42, UC-43	/provider/…	accommodation, room_type, activity, availability	26.4, 26.5	BR-023, BR-082	Provider suite
FR-110	Administrator manages the catalogue	UC-61	/admin/…	country, region, destination, attraction	27.8	BR-084	Section 41.12
FR-111	Administrator configures commercial rules	UC-62	/admin/commission-rules, /admin/tariffs	commission_rule, transfer_tariff	27.11	BR-070	TC-110
FR-112	All administrative actions are audited	UC-69	/admin/audit-logs	audit_log	27.17	—	Audit coverage test
NFR-P03	Quote completes within 1.5 s at p95	—	POST /trips/{id}/quote	—	—	—	LT-03
NFR-A05	RPO ≤ 5 min, RTO ≤ 60 min	—	—	—	—	—	DR rehearsal
—	No AI features in the product	—	—	—	—	Section 3.5	TC-901

The full matrix covering every requirement is maintained as a living artefact in `documentation/srs/traceability.xlsx`, generated in part from test annotations so that it cannot silently drift from the code.

43 Technical Consistency Verification

This section records the internal validation performed on the specification before release. Each item states the check and its result.

#	Check	Result
1	Every major requirement has a corresponding system function	Verified. Sections 10–27 cover every capability named in Sections 2.2 and 5.3; the traceability matrix has no requirement without an implementing endpoint or screen.
2	Every major function has appropriate database entities	Verified. Each subsystem section names its owning tables, and each appears in Section 7.5 and in the ERD.
3	The ERD matches the written schema	Verified. The four ERD views in Section 7.3 were reconciled field by field against Section 7.5 and the relationship register in Section 7.4; the `booking` subtype pattern, the `itinerary_item` polymorphic nullable keys and the append-only ledger are represented identically in both.
4	APIs match backend services	Verified. Every endpoint in Section 9.3 maps to a module in Section 6.4; no endpoint crosses a module boundary except through a service interface.
5	UI screens map to APIs and backend functions	Verified. Every screen in Sections 24 and 25 names the endpoints it calls, and every endpoint in Section 9.3 has at least one consuming screen or is explicitly internal (webhooks, scheduler-driven routines).
6	Booking states are consistent everywhere	Verified. The ten states in Section 20.2 are the same set used in Sections 7.5.12, 24, 25, 26.6, 27.9 and the test cases; no other state name appears anywhere in the document.

#	Check	Result
7	Payment states are consistent with booking states	Verified. The compatibility table in Section 20.4 is exhaustive over both machines, and the nightly consistency job that enforces it is specified in Sections 8.8 and 29.14.
8	Driver assignment is consistent with transportation bookings	Verified. `driver_assignment` is 1:0..1 with `booking_transfer` (relationship R25); the assignment machine in Section 11.7 drives the booking transitions declared in Section 20.3; the exclusion constraint in Section 7.7 enforces BR-051.
9	Accommodation availability is consistent with booking logic	Verified. The sellable expression in Section 14.3, the hold routine in Section 17.3, the confirmation routine in Section 20.8 and the constraints in Section 7.5.8 all use the same three counters with the same semantics.
10	Activity capacity is consistent with booking logic	Verified. Identical structure to item 9, over `activity_departure`.
11	Commission calculations are consistent with the payment architecture	Verified. Commission is snapshotted at confirmation (BR-070), accrued at completion (BR-071), reversed on refund per the table in Section 22.6, and every movement is a ledger entry type declared in Section 22.3.
12	Notifications correspond to real system events	Verified. Every row in Section 19.1 is triggered by a domain event declared in Section 8.9 or by a scheduled job declared in Section 8.8. No notification exists without a producer.
13	Security requirements match the architecture	Verified. Every control in Section 30 has a named enforcement point in Sections 6, 8 or 9; the authorisation model in Section 5.2 is enforced by the two-check pattern in Section 30.3 and tested by the TC-200 series.
14	Admin permissions match administrative functionality	Verified. Every screen in Section 27 maps to a role in Section 5.2; no console function exists that no role can perform, and no role grants a capability with no interface.
15	Zanzibar remains the actual MVP scope	Verified. Section 4.1 fixes the market; Section 38 admits no non-Zanzibar feature to MUST HAVE; mainland and regional expansion appear only under FUTURE.
16	Future expansion does not require redesigning the architecture	Verified with one recorded limitation. Adding destinations, corridors, tariffs, currencies and listings is configuration (acceptance test 41.12). The known structural limitation is single-destination trips (`trip.destination_id`), whose migration path is specified in Sections 4.3 and 44.4.
17	No AI functionality has been introduced	Verified. Section 3.5 enumerates every behaviour that could have been delegated to a model and names its deterministic implementation: SQL ranking (16.5), rule-based dispatch scoring (11.6), constraint-based itinerary sequencing (10.4), tariff-table pricing (12.4), threshold-based fraud rules (30.14), and a structured help centre in place of a chatbot (24.32). TC-901 enforces this at build time.
18	No unnecessary features were added to lengthen the document	Verified. Each subsystem traces to a problem in Section 3.1 or to a regulatory, financial or operational obligation. Deferred capabilities are listed in Section 38 rather than specified speculatively.

Two residual risks are recorded rather than resolved, because they require commercial decisions outside this document: the choice of payment provider (Section 21.2), which affects settlement currency handling and mobile-money coverage; and the routing provider's caching terms (Section 12.3), which affect how long `route_cache` entries may be retained. Neither changes the architecture, and both are isolated behind ports.

44 Future Scalability and Roadmap

44.1 Scaling the Current Architecture

Pressure	First response	Second response
API request volume	Horizontal scaling of stateless API pods	Split read traffic to the replica; add a CDN edge cache for catalogue reads
Database write volume	Vertical scaling; tune indexes; move reporting to the replica	Partition `booking` and `payment` by year; extract the highest-volume module
Catalogue search complexity	PostgreSQL FTS with GIN indexes	OpenSearch behind a search port
Location ingest volume	Partition daily; batch writes; shorten retention	Extract the location gateway as the first standalone service
Notification volume	Dedicated workers per channel	Extract the notification dispatcher — the second extraction candidate

Pressure	First response	Second response
Real-time connections	Scale WebSocket pods with Redis pub-sub fan-out	Dedicated real-time service
Reporting load	Materialised nightly summaries	A read-optimised analytical store fed by change data capture

44.2 Service Extraction Sequence

The modular monolith is designed so that extraction is a deployment change, not a rewrite. The order is chosen by consistency requirement: extract what needs the least transactional coupling first.

```
1. Notification dispatcher   (no consistency requirement; event-driven already)
2. Location / tracking gateway (write-heavy, tolerant of eventual consistency)
3. Catalogue & search        (read-heavy, cache-friendly, no writes in the booking path)
4. Finance & payouts         (batch-oriented, reconciles asynchronously)
── boundary of easy extraction ──
5. Trip planning             (would require a distributed pricing contract)
X. Booking + inventory + payment  DO NOT EXTRACT
   These three share the atomic transaction that makes the basket correct.
   Splitting them converts a database transaction into a saga and is the
   single most expensive mistake available to this system.
```

44.3 Product Roadmap Beyond V1

Horizon	Capability	Principal architectural work
V1.1	Promotions, saved payment methods, additional languages, vehicle hire	Discount entity in the pricing engine; locale content rows
V1.2	Public web booking front end; provider bulk import	A second consumer of the same API; import pipeline with validation
V2.0	Mainland Tanzania	Catalogue data; new corridors and tariffs; park permit fees as a new line type; domestic flight legs as a new service class
V2.1	Flight status integration	Implement `FlightStatusPort`; automatic transfer re-timing and driver re-notification
V2.2	On-demand transfers	Real-time supply matching; a shorter offer TTL; live pricing within the existing tariff framework
V3.0	East African expansion	Multi-country trips (see 44.3); cross-border transfer compliance; a second tax jurisdiction model
V3.1	Partner and agency API	Public API with per-partner keys, quotas, and a partner commission dimension

44.4 Known Structural Changes Required for Multi-Destination Trips

The single-destination assumption is deliberate for V1 and its removal is bounded:

```
Today:   trip.destination_id  -> destination   (single FK)

Target:  trip.primary_destination_id  -> destination   (retained)
         trip_destination (trip_id, destination_id, sequence_no,
                           arrive_on, depart_on)       (new join table)

Consequential changes:
  - itinerary day skeleton keyed by (day, destination) rather than day
  - validation rule VR-03 must consider inter-destination transfers
    including ferry and domestic flight legs
  - currency rule VR-10 must permit conversion across destinations
    with a single presentment currency
  - catalogue queries filter by the destination set, not one id
  - transfer corridors already support any origin-destination pair,
    so no change is required there
```

Estimated at four to six engineering weeks, with no change to the booking, payment or finance modules — which is the point of having isolated them.

Appendices

Appendix A — Glossary of Status Values

Machine	Values
Trip	DRAFT, PRICED, PENDING_PAYMENT, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED
Booking	DRAFT, PENDING, AWAITING_PROVIDER, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, REFUNDED, NO_SHOW, FAILED
Payment	INITIATED, PENDING, AUTHORISED, CAPTURED, PARTIALLY_REFUNDED, REFUNDED, FAILED, EXPIRED, DISPUTED, CHARGEBACK_LOST
Refund	REQUESTED, APPROVED, PENDING, SETTLED, FAILED
Driver assignment	PENDING, OFFERED, ASSIGNED, EN_ROUTE, ARRIVED, STARTED, COMPLETED, CANCELLED, UNFULFILLED, INCIDENT
Driver availability	OFFLINE, ONLINE, ENGAGED
Inventory hold	HELD, COMMITTED, RELEASED, EXPIRED
Verification	DRAFT, SUBMITTED, UNDER_REVIEW, VERIFIED, REJECTED, SUSPENDED
Activity departure	OPEN, FULL, CANCELLED, CLOSED
Review	PENDING, PUBLISHED, REJECTED, WITHDRAWN
Payout	DRAFT, PENDING_APPROVAL, APPROVED, PAID, FAILED
User account	PENDING, ACTIVE, SUSPENDED, CLOSED

Appendix B — System Settings Register

Every value below is a `system_setting` row, editable by an administrator, with no deployment required. This register is the practical expression of NFR-M07.

Key	Default	Effect
`buffer.arrival_processing_minutes`	45	Delay between flight arrival and pickup
`buffer.airport_departure_minutes`	180	Required arrival before departure flight
`buffer.activity_minutes`	15	Slack before an activity start
`quote.ttl_minutes`	20	Quote and hold validity
`payment.window_minutes`	30	Extended hold during payment
`dispatch.lead_hours`	72	When scheduled dispatch begins
`offer.ttl_seconds.scheduled`	3600	Offer validity for future work
`offer.ttl_seconds.imminent`	90	Offer validity within 24 hours
`assignment.disclosure_hours`	24	Deadline to disclose driver details
`dispatch.weights`	0.40/0.25/0.20/0.10/0.05	Proximity, rating, acceptance, experience, utilisation
`dispatch.max_radius_m`	60000	Proximity normalisation ceiling
`geofence.pickup_m`	300	Arrival geofence
`geofence.approach_m`	1500	Nearby notification
`wait.airport_minutes`	60	No-show threshold at a gateway
`wait.standard_minutes`	15	No-show threshold elsewhere
`platform_fee_rate`	0.05	Tourist-facing service fee
`commission.default_percent`	15	Global fallback commission
`fx.markup_percent`	2.0	Conversion protection margin
`settlement_hold_days`	2	Delay before balance becomes available

Key	Default	Effect
`payout.minimum`	50 USD equivalent	Threshold below which balance rolls forward
`refund.auto_approve_limit`	200 USD equivalent	Above this, finance approval required
`limit.items_per_day`	5	Warning threshold VR-13
`limit.travel_minutes_per_day`	240	Warning threshold VR-14
`trip.max_days`	30	Maximum trip length
`stay.max_nights`	30	Maximum accommodation stay
`availability.horizon_days`	400	Calendar auto-extension
`departures.horizon_days`	180	Schedule materialisation window
`location.retention_days`	30	Raw point retention
`provider_response_hours`	24	On-request activity response window
`review.window_days`	30	Review submission window

Appendix C — Zanzibar Seed Data Summary

The seed set delivered in `database/seeds/` and loadable through the admin console:

Entity	Count	Notes
Country	1	Tanzania (TZ), TZS, Africa/Dar_es_Salaam
Region	5	Zanzibar Urban/West, North, Central/South, Pemba North, Pemba South
Destination	10	Section 4.1, with Pemba inactive at launch
Gateway destination	1	ZNZ, with meeting point text, coordinates and photograph
Attraction	~25	Stone Town sites, Jozani Forest, Prison Island, Nungwi turtle sanctuary, spice farms, Mnemba Atoll, and others
Transfer corridor	~30	All gateway-to-destination pairs plus principal inter-destination pairs, per vehicle class
Transfer tariff	4	One per vehicle class, region-scoped fallback
Cancellation policy	4	FLEX_48H, MODERATE_7D, STRICT_14D, NON_REFUNDABLE
Commission rule	4	Global default plus one per booking type
Vehicle class	4	STANDARD, COMFORT, VAN, MINIBUS
Notification template	~25 events × 3 channels	English

Accommodation and activity records are **not** seeded; they are created by verified providers through the portal, which is the correct source of truth for price, availability and capacity.

Appendix D — Open Decisions Requiring Commercial Input

#	Decision	Owner	Blocking
D1	Payment provider selection and settlement account structure	Finance / Product Owner	Phase 8 start
D2	Routing provider selection and its caching terms	Architecture / Finance	Phase 4 completion
D3	Hosting region and data residency position	Legal / Architecture	Phase 14
D4	Tanzanian tourism tax and VAT treatment of platform fees and commission	Tax adviser	Phase 8
D5	Provider agreement terms: commission bands, chargeback allocation, cancellation compensation	Legal / Commercial	Phase 11
D6	Consumer terms of use and privacy notice	Legal	Phase 13
D7	Insurance and liability position for transfers	Legal / Insurance	Phase 14

#	Decision	Owner	Blocking
D8	Support operating hours and escalation contacts	Operations	Phase 13

None of these decisions changes the architecture; each is isolated behind a port, a configuration row or a content record, which is the property that allows development to proceed in parallel with the commercial workstream.

Appendix E — Document Conventions

“Must” denotes a mandatory requirement; “should” denotes a strong recommendation whose deviation requires a recorded architecture decision; “may” denotes an option. All times are stored in UTC and displayed in the destination’s local timezone. All monetary examples use United States Dollars unless stated. Code fragments are illustrative of structure and contract, not literal implementations. Diagrams are normative where they define states, transitions, relationships or sequences, and indicative where they describe deployment topology.